

OLIMPO: Fake News AI

Documento técnico consolidado: projeto, metodologia, dados, modelos, resultados e MVP

Equipe Olimpo, PUC-Campinas e Instituto de Pesquisas ELDORADO



Sumário

1. Projeto	4
1.1 Situação encontrada	4
1.2 Solução proposta	4
Evolução do escopo	4
Princípio central e posicionamento	4
1.3 Escopo e priorização (MoSCoW)	5
1.4 Indicadores de desinformação (parâmetros)	6
1.5 Mapa de ideiação	7
2. Metodologias	8
2.1 Metodologia organizacional	8
2.2 Frameworks	9
2.3 Ferramentas	9
Jira: Plataforma para organização com quadro de tarefas	9
GitHub: Plataforma para os repositórios e versionamento do desenvolvimento do projeto	10
Google Colab: Plataforma para o desenvolvimento de notebooks de maneira compartilhada	10
2.4 Metodologia de modelagem de Machine Learning	10
Separação e engenharia de dados	10
Modelo baseline	10
Algoritmo de classificação escolhido	10
Validação e prevenção de vazamentos	10
2.5 Governança de IA (AIPGF)	10
2.6 Mineração de dados (CRISP-DM)	13
2.7 Metodologia de testes	13
2.8 MLOps	14
MLOps	14
2.9 Metodologia de pesquisa	15
2.10 Protocolo comum de comparação	15
3. Dataset	16
3.1 Fake.br-Corpus	16
3.2 Seleção de campos	16
3.3 Corpus externo: FakeRecogna	18

3.4 Extração morfossintática própria (spaCy)	18
4. Análise exploratória de dados (EDA)	18
4.1 Introdução	18
4.2 Correlação entre variáveis	20
Pairplot	21
Matriz de correlação	21
Fator de inflação de variância (VIF)	21
4.3 Manipulação de dados	22
4.4 Visualizações	23
Boxplot	23
4.5 Evidências de artefatos de fonte	24
4.6 EDA das features morfossintáticas (spaCy: POS e DEP)	24
Extração e preparação	24
Matriz de correlação de Spearman	24
VIF por família	25
Relevância univariada por AUC	26
Pair grid, histogramas e boxplots por classe	27
5. Aprendizado supervisionado	29
5.1 Classificadores candidatos	29
5.2 Preparação dos dados	31
5.3 Representação: TF-IDF e dimensionalidade	32
5.4 Modelos treinados	34
SVM (Support Vector Machine)	34
Por que funciona no nosso caso?	35
Notebook com o código do treinamento e cross validation da SVM	35
Random Forest	36
Regressão Logística	37
Modelo baseline	39
5.5 Resultados no protocolo atual: regularização (C) e poda de vocabulário	39
Regularização (06_regularizacao_C)	40
Poda de vocabulário (05_feature_dim_analysis, min_df)	40
5.6 Metadados: do corpus e morfossintáticos (spaCy)	41
Metadados do corpus	41
Extração morfossintática com spaCy (branch fix-metadados)	42
Metadados morfossintáticos por notícia (spaCy; notebooks 10 a 12)	42
5.7 Interpretabilidade (SHAP)	46
Desenvolvimento	47
Visualização	47
5.8 Dimensionalidade e kernels	49
Redução de dimensionalidade (08_reducao_dim, 05)	49
Kernels não lineares (09_kernels_svm)	50
Redução de dimensão no modelo com spaCy (notebooks 11 e 12)	50
5.9 Generalização fora do corpus (teste externo no FakeRecognia)	52
5.10 Validação, curva de aprendizado e dependência de convenções	53
Dados de treino, validação e teste	53
Hold-out versus K-Fold (03_SVM_training)	53
Curva de aprendizado	54
Dependência de convenções de redação	55
5.11 Modelo de produção	55
6. Aprendizado não supervisionado	56
6.1 Análise de classificadores não supervisionados	56

6.1.1 Isolation Forest: funcionamento	57
6.1.2 Por que a detecção de anomalias foi a primeira abordagem	58
6.2 Resumo do relatório	58
6.3 Introdução e objetivo	59
6.4 Dados e protocolo experimental	59
6.5 Jornada dos modelos avaliados	59
6.5.1 Resultados quantitativos no protocolo canônico	60
6.5.2 Índice de Jaccard como medida complementar	60
6.6 Revisão crítica das acurácias	60
6.7 Mineração FP-Growth e extração de regras	61
6.7.1 Preparação e execução	61
6.7.2 Como ler as regras	61
6.8 Avaliação externa e controles	61
6.8.1 Controle da autoria	62
6.9 Discussão e aplicação pelo método socrático	62
6.10 Conclusão	62
Referências do relatório não supervisionado	63
7. Resultados e discussão integrada	63
7.1 Quadro-resumo	63
7.2 O que os resultados sustentam	64
7.3 Limitações	64
8. MVP: plataforma Olimpo	65
8.1 Visão do produto	65
8.2 Arquitetura	65
Decisões de arquitetura e trade-offs	65
8.3 Módulos implementados	69
8.4 Estado do componente de IA	70
Modelo supervisionado	70
Observação de escrita (FP-Growth)	71
Validação da integração	71
8.5 Como o MVP constrói sobre as descobertas	71
9. Conclusão	72
Referências	72
Apêndices	73
Apêndice A: Feature extraction e atributos derivados propostos	73
Atributos derivados propostos	74
Apêndice B: Capturas dos resultados do FP-Growth	74
Apêndice C: Trechos do código do FP-Growth	75
Apêndice D: Catálogo completo das regras extraídas	77
Apêndice E: Planejamento (roadmap) e cumprimento	78

Parceria: PUC-Campinas + Instituto de Pesquisas ELDORADO · Repositório: `olimpofake-news-ai`
Estado conferido em 09/10/2026 nesta branch `docs/entregavel-final`: pesquisa em `machine-learning/`
e execução integrada em `model-engine/`. Os experimentos anteriores permanecem em `history/`.

Como ler este documento. A ordem é a do raciocínio técnico (problema → método → dados → exploração → modelos → resultados → produto), e não a cronológica. Dois tipos de aviso aparecem no texto: **Observação** registra uma correção ou uma decisão que mudou o que o texto anterior afirmava; **[PENDENTE]** marca o que ainda não tem evidência no repositório e

precisa ser executado ou confirmado antes da entrega. Todo [PENDENTE] deve ser resolvido ou removido na versão final.

1. Projeto

1.1 Situação encontrada

O excesso de informação e a perda da confiabilidade Atualmente as plataformas digitais costumam priorizar conteúdos que geram cliques e engajamento rápido, muitas vezes deixando a precisão dos fatos em segundo plano. Com isso, a fronteira entre o jornalismo sério e a desinformação acabou se misturando, gerando uma crise global de confiança nas notícias.

A queda do senso crítico e o efeito das "bolhas" Nesse cenário de consumo acelerado, o senso crítico das pessoas acaba enfraquecendo. A sobrecarga de notícias, somada aos algoritmos, nos coloca dentro de "bolhas" de conteúdo. Nelas, somos expostos quase exclusivamente a opiniões que confirmam o que já pensamos. Isso desestimula a checagem das fontes e nos torna mais vulneráveis a aceitar narrativas apelativas (ou falsas) como verdades absolutas, sem qualquer questionamento.

O impacto na sociedade Esse fenômeno não fica restrito à internet; os efeitos atingem o nosso dia a dia. A desinformação alimenta a polarização extrema e destrói a confiança nas instituições, na educação, na ciência e na própria democracia. Quando a sociedade perde a capacidade de separar o fato da ficção, tomamos decisões piores sobre assuntos vitais, como nossa saúde e nossos direitos, transformando uma falha tecnológica em uma verdadeira crise social.

1.2 Solução proposta

Para combater o problema da desinformação na raiz, vamos desenvolver um site avaliador de notícias. Diferente das ferramentas tradicionais de checagem de fatos, que se limitam a carimbar um texto como "verdadeiro" ou "falso", nossa plataforma foi pensada para ser um ambiente interativo. O objetivo não é apenas classificar a informação inserida, mas reeducar o olhar do leitor e devolver a sua autonomia intelectual.

Por que usaremos perguntas socráticas? Se o sistema apenas ditar que algo é mentira, um usuário pode simplesmente rejeitar a ferramenta por entrar em conflito com suas crenças. Para contornar essa barreira, escolhemos integrar o Método Socrático. Em vez de entregar um veredito impositivo, a plataforma estimula o pensamento crítico fazendo perguntas guiadas, como: "O termo "URGENTE" apareceu 10 vezes, isso é confiável?" ou "Existe uma referência da fonte?". Essa aprendizagem ativa ajuda o indivíduo a perceber as falhas de lógica por si mesmo, quebrando o viés de confirmação.

Evolução do escopo

O documento de concepção descreve um **avaliador individual de notícias** com perguntas socráticas. O produto em desenvolvimento no repositório evoluiu para uma **plataforma gamificada e colaborativa** (estilo Kahoot): jogadores em salas multiplayer ou solo recebem manchetes e votam entre *Fake*, *Fato real* e *Tenho dúvida*; ao fim de cada rodada o sistema revela o gabarito e uma justificativa pedagógica. O componente de IA continua sendo o diferencial, pois é ele que fornece a classificação e a explicação usadas como feedback educativo. A seção 8 detalha o que já foi implementado.

Princípio central e posicionamento

A IA **não substitui o julgamento humano**. O produto deve estimular o pensamento crítico, apresentar indícios e deixar claras as limitações do modelo. Por isso a solução é posicionada como uma plataforma educacional que identifica padrões associados à confiabilidade de informações, e **não** como um "detector definitivo de fake news": o modelo trabalha com padrões aprendidos de um dataset e está sujeito a erros,

vieses e mudanças de contexto (seção 7.3). Esse posicionamento é coerente com os resultados: a generalização fora do Fake.br é fraca, e a saída do modelo é apresentada como indício.

1.3 Escopo e priorização (MoSCoW)

Para viabilizar essa solução, estruturamos as prioridades de desenvolvimento da plataforma da seguinte maneira, utilizando a metodologia MoSCoW:

Requisitos Essenciais (*Must Have*):

- **Plataforma web (Site):** O ambiente principal e acessível onde a aplicação funcionará.
- **Modo de uso individual (Solo):** Foco inicial na jornada de aprendizagem de um único usuário.
- **Input do usuário:** Área dedicada para que a pessoa insira diretamente a notícia ou afirmação que deseja avaliar.
- **Sistema de semáforo:** Classificação visual e intuitiva utilizando *red flags* (alertas de desinformação) e *green flags* (indícios de confiabilidade) nas afirmações testadas.
- **Método socrático:** O núcleo do sistema, responsável por devolver a análise em formato de questionamentos reflexivos.

Requisitos Desejáveis (*Should Have*):

- **Marcação pelo usuário:** o próprio leitor será desafiado a destacar as palavras que considera como *red flags* no texto.
- **Gamificação:** Uso de mecânicas de jogos para tornar o processo de checagem mais engajador e recompensador.
- **Dinâmicas coletivas:** Expansão para uso em grupo e análise de padrões de comportamento de múltiplos usuários.

Requisitos Potenciais (*Could Have*):

- **Integração do Método SIFT:** Possibilidade de aliar a técnica SIFT (*Stop; Investigate the Source; Find Better Coverage; Trace Claims, Quotes, and Media to the Original Context*) às perguntas socráticas.
- **Faixa etária e localidade dos usuários:** Durante o cadastro na plataforma, solicitaremos a data de nascimento e a localização do usuário. A finalidade dessa coleta é agrupar o público demograficamente para identificar possíveis padrões de comportamento no consumo de informações dentro de cada região e geração. Dessa forma, o sistema poderá analisar e validar métricas específicas cruzando esses dados. Será possível verificar, por exemplo, a hipótese de que 95% dos jovens tendem a clicar mais em notícias com gatilhos de *clickbait* (textos apelativos como "Urgente", "Veja" ou "Saiba mais"), ou avaliar se os usuários de uma cidade específica, como Campinas, interagem de maneira diferente e apresentam outros vieses de confirmação ao lerem notícias de impacto local.
- **Tooltips Socráticos:** Exibição interativa de perguntas reflexivas geradas pela socratic-llm ao interagir com os trechos destacados, instigando o senso crítico do leitor sem exigir resposta.

Observação. Situação dos requisitos *Must Have* em 06/10/2026, verificada no código: **plataforma web:** implementada (Next.js); **input do usuário:** implementado como extração de notícia a partir de URL (`/api/news/extract`); **semáforo:** implementado como votação em três níveis (*reliable / uncertain / unreliable*); **método socrático: parcial:** a tela de jogo exibe três cartões socráticos **fixos** (fonte, tom emocional e evidência) antes da revelação do gabarito, mas as perguntas não são geradas pelo modelo nem por LLM; **modo solo:** backend de salas pronto, interface em andamento. O modelo de ML **ainda não está integrado** ao sistema: o serviço de análise é um *mock* por palavras-chave (seção 8.4).

1.4 Indicadores de desinformação (parâmetros)

Como a inteligência artificial não sabe instintivamente o que é verdade ou mentira, nossa finalidade foi fornecer a ela um conjunto claro de indicadores baseados no formato e no comportamento do texto (parâmetros). Ao invés de tentar checar cada fato isoladamente, ensinaremos o modelo a identificar a estrutura da desinformação (vícios de linguagem, sensacionalismo e falta de transparência).

Algumas informações serão mais relevantes devido à grande disparidade entre as comparações de dados entre informações falsas e verdadeiras. No *paper* “Towards automatically filtering fake news in Portuguese” encontramos dados como 36% dos erros ortográficos para notícias falsas enquanto 3% dos erros estavam presentes em texto verídicas. Outro ponto relevante é a variedade linguística usando sinônimos para detalhar o conteúdo, notícias falsas possuem 119 em média enquanto notícias verdadeiras possuem uma média de 494.

Isso garante que a solução seja rápida, escalável e capaz de justificar de forma lógica e transparente a análise de confiabilidade gerada para o usuário através da lógica de semáforo (*red* e *green flag*).

Palavras 100% em maiúsculo (Caixa Alta): Na internet, escrever em maiúsculo equivale a gritar. O jornalismo profissional raramente usa esse recurso. Para o modelo, o excesso de palavras em CAIXA ALTA é um forte indicador de sensacionalismo, exagero e intenção de atrair cliques (*clickbait*).

Erros Ortográficos e gramaticais: Veículos de comunicação reconhecidos possuem processo de revisão e edição do conteúdo, para que não haja uma publicação com erros. Consequentemente, o modelo efetuará uma avaliação no texto, entendendo se há erros frequentes, concordâncias mal colocadas, frases construídas incorretamente, geralmente tendem a ser padrões em notícias falsas.

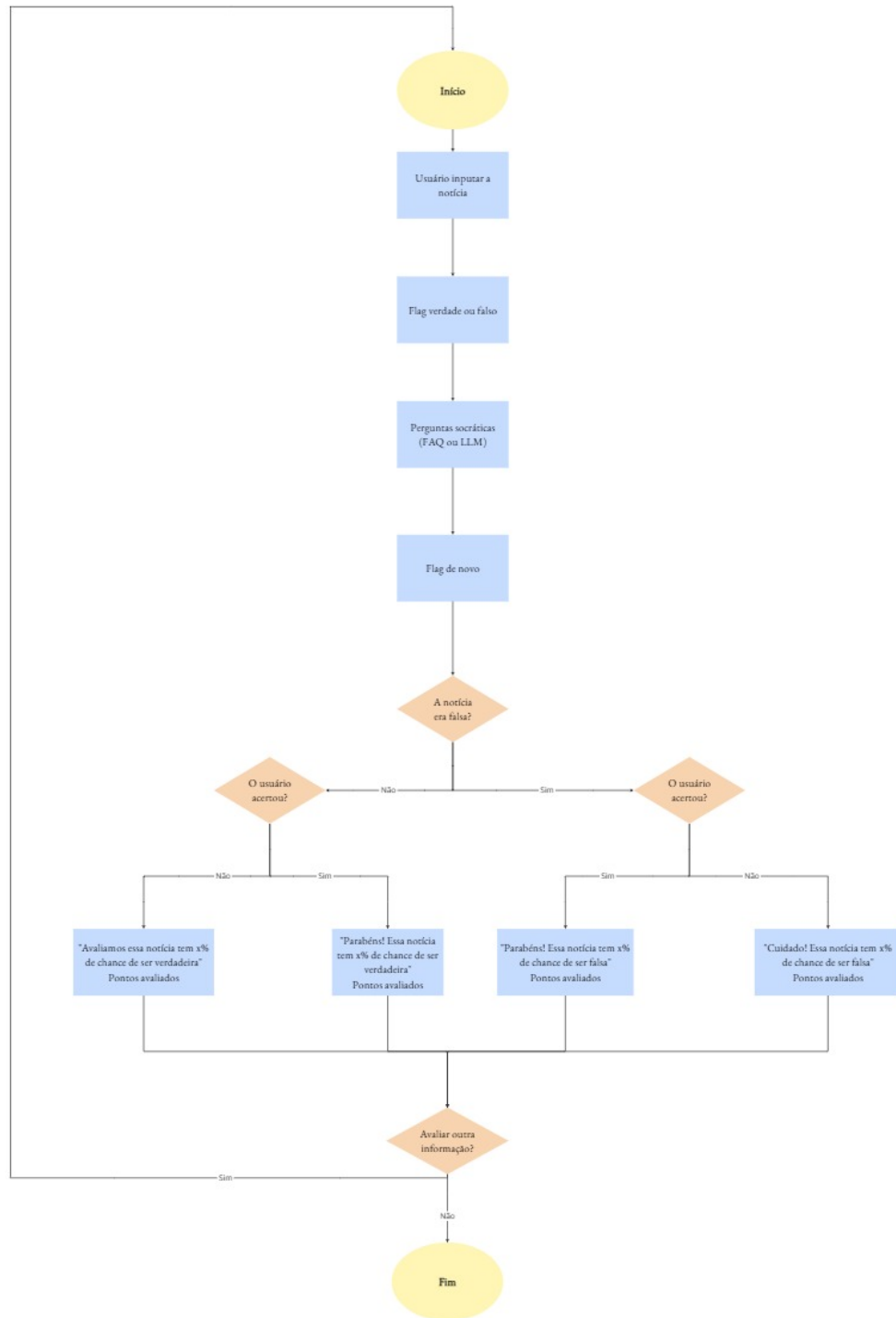
Verbos no imperativo: Expressões que enviessem os leitores, tais como “Compartilhe antes que apague”, “Clique aqui agora”, “Divulgue entre os familiares e amigos”, tendem a forçar uma ação, que por muitas vezes são tendenciosas. Com isso, o modelo aprenderá a identificar o imperativo como uma tentativa de manipulação, onde o foco é a viralização, e não a informação em si.

Emoções no texto e senso de urgência: A mentira viaja rápido porque mexe com nossos instintos. A desinformação usa o medo, a raiva ou a ansiedade para “desligar” o raciocínio lógico do leitor. Através da análise de sentimento, o modelo detecta gatilhos de urgência (“Alerta grave!”, “Cuidado!”) que tentam provocar reações impulsivas, e tendem a ser tendenciosas para o engajamento.

Identificação do responsável (Autoria, Empresa): A falta de um autor claro (um jornalista, uma agência de notícias ou uma empresa verificada) é um sintoma clássico de notícias falsas. O modelo procura pela autoria; se o texto não tem um “dono” para se responsabilizar pelo que foi dito, a confiabilidade cai. Alguns exemplos negativos: “relatado por alunos”, “comprovado por especialistas”. Exemplos positivos: “relatados por alunos da PUC Campinas”, “comprovado por especialistas do instituto Eldorado”.

Fora ou Falta de Contexto: Uma das táticas mais perigosas da desinformação não é inventar uma mentira do zero, mas tomar um fato verdadeiro e retirá-lo de contexto (como usar um vídeo de 2019 dizendo que aconteceu ontem). O modelo avalia a estrutura do texto para checar se ele responde às perguntas básicas (quem, quando, onde), penalizando conteúdos vagos ou descontextualizados.

1.5 Mapa de ideação



2. Metodologias

2.1 Metodologia organizacional

Escolhemos a metodologia *Crystal Clear* porque é eficiente para times pequenos e projetos rápidos. Essa metodologia também usa os principais conceitos da metodologia ágil, porém aplicados de forma mais flexível do que o Scrum ou o XP.

Além disso, o *Crystal Clear* não define cargos rígidos, permitindo que cada pessoa participe de várias frentes do projeto. Isso se relaciona diretamente com o método CBL da residência e seus conceitos de *group learning* e *individual learning*: todos nos interessamos por todas as partes do projeto e queremos manter contato constante para sustentar o fluxo de aprendizado coletivo.

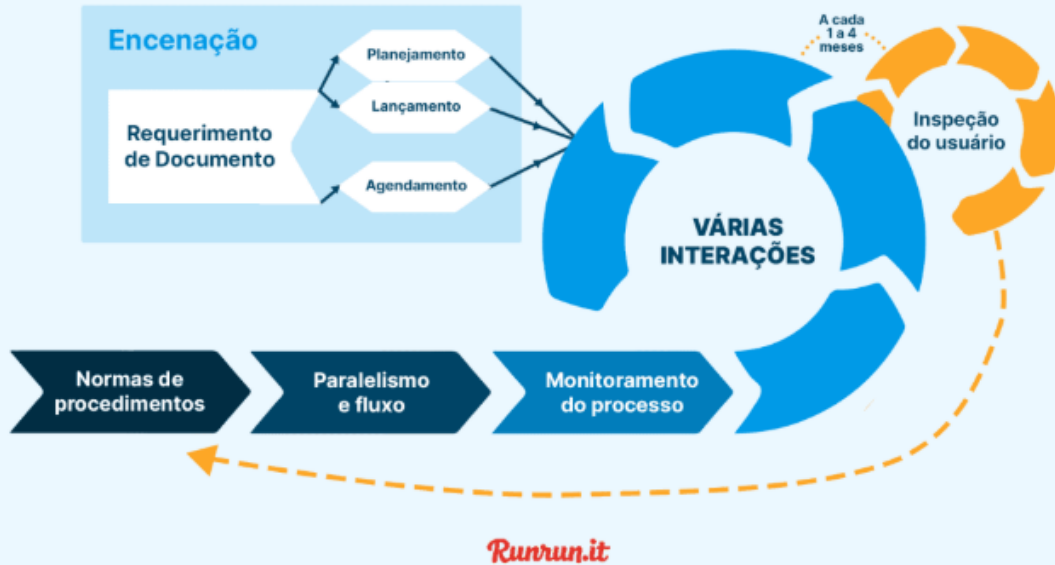
Também nos identificamos com as fases do *Crystal Clear*:

- *Reflection workshop*: Reunião periódica (geralmente ao final de cada incremento) para a equipe parar e analisar o que funcionou ou não no processo, equivalente à retrospectiva;
- *Iteration planning*: Reunião para definir as tarefas e funcionalidades do próximo ciclo de entrega;
- *Osmotic communication*: Mais do que uma reunião marcada, é a prática de manter o time no mesmo ambiente ou com canais abertos, para que as informações fluam de forma natural e contínua.

A partir disso, toda semana usaremos o que for necessário dessa metodologia, sempre seguindo esses conceitos, de forma a entregar o projeto e, ao mesmo tempo, permitir que cada integrante da equipe aprenda sobre as diferentes partes do sistema.

Para visualização, utilizaremos o quadro de tarefas no Jira, o qual organizaremos em Épicos, Histórias e Tarefas. Essa organização é uma das bases dos processos ágeis, dividindo os grandes objetivos em pequenas tarefas de curto prazo e menor esforço. Vamos usá-la para nos direcionar e conseguir dividir tarefas nas sprints, mantendo o grupo trabalhando de forma que o esforço seja equivalente e com prazos que fazem sentido.

CICLO DA METODOLOGIA CRYSTAL



2.2 Frameworks

- **RACI:** Responsável, Aprovador, Consultado e informado.
Vamos usar essa matriz RACI para conseguirmos nos organizar previamente qual é a responsabilidade de cada um em uma determinada tarefa, ajudando a organização da equipe, já que o *Crystal Clear* não demanda papéis fixos para cada pessoa durante o projeto.
- **MoSCoW:** *MUST HAVE*, *SHOULD HAVE*, *COULD HAVE*, *WON'T HAVE*

O MoSCoW vai ser utilizado para andar com o planejamento das sprints de desenvolvimento ao longo do projeto para definir o que vai ser mais necessário em cada parte do processo e o que deve ou não demandar esforço.

- **Planning Poker:** Definir média de esforço para cada uma das tarefas

Junto com o MoSCoW, o *Planning poker* vai definir a média de esforço esperada para as tarefas em grupo e com isso conseguir separar tarefas por pessoa e tempo que vai demandar.

2.3 Ferramentas

Jira: Plataforma para organização com quadro de tarefas

Integrantes do grupo já tiveram contato com a plataforma então decidimos usá-la pela familiaridade, além de ter possibilidade de vincular com as *branches* e os *pull requests* quando utilizarmos o GitHub.

GitHub: Plataforma para os repositórios e versionamento do desenvolvimento do projeto

Google Colab: Plataforma para o desenvolvimento de notebooks de maneira compartilhada

2.4 Metodologia de modelagem de Machine Learning

Separação e engenharia de dados

- Os dados do Fake.br-Corpus (7.200 notícias, 3.600 por classe) são divididos em **80% treino e 20% teste**, de forma estratificada e com `random_state=42`. O teste fica isolado de qualquer escolha de modelo, hiperparâmetro ou configuração.
- Os textos passam por **normalização** e por **truncamento em 100 palavras** (detalhes na seção 5.2). A avaliação durante o desenvolvimento é feita por **validação cruzada estratificada, k=5, com embaralhamento (random_state=42)**.
- A representação padrão é o **TF-IDF word+char**: unigramas e bigramas de palavras (`min_df=2`) combinados com n-gramas de caracteres `char_wb` de 3 a 5 (`min_df=3`), ambos com `sublinear_tf`.

Modelo baseline

- A **Regressão Logística** é o baseline do projeto: fornece o patamar mínimo de desempenho, tem custo computacional baixo em espaços esparsos de alta dimensão e coeficientes diretamente interpretáveis.
- Sob o pipeline atual, a LR com word+char obtém **F1-macro de $0,9182 \pm 0,0136$** na validação cruzada (`C=1`; `resultados_C.csv`).

Algoritmo de classificação escolhido

- O **LinearSVC** (SVM linear) com word+char e `C=1` é o classificador de referência: **F1-macro de $0,9295 \pm 0,0099$** na validação cruzada, contra 0,9182 da LR e 0,9285 do Random Forest com a configuração equivalente no protocolo anterior (seção 5.5).
- A justificativa é estrutural: o TF-IDF gera ~114 mil atributos para ~5,7 mil documentos de treino ($d \gg N$, ~99,5% de esparsidade), regime em que modelos lineares de margem máxima são eficientes e estáveis. Kernels não lineares não trouxeram ganho (seção 5.8).
- O LinearSVC não produz probabilidades. No pipeline de produção (seção 5.11) ele é envolvido em `CalibratedClassifierCV` (sigmoide/Platt), que entrega $P(\text{fake})$ ao sistema.
- **O modelo de produção é o M2 + χ^2 (10 mil) + SVD(500)**: word+char reduzido por χ^2 e SVD, somado ao bloco de metadados morfosintáticos spaCy, com LinearSVC calibrado (seções 5.6, 5.8 e 5.11). O word+char completo (B0) permanece como baseline de referência.

Validação e prevenção de vazamentos

- Todos os estimadores e transformações (vetorizadores, imputação, padronização, seleção de atributos) ficam dentro de um **Pipeline** com `ColumnTransformer` do scikit-learn, ajustados somente com o treino de cada fold.
- O desempenho no teste interno não deve ser lido como capacidade de generalização: o teste externo (seção 5.9) mostra queda de F1 de ~0,93 para ~0,61 fora do Fake.br.

2.5 Governança de IA (AIPGF)

Definimos que utilizaríamos o AI Project Governance Framework (AIPGF), devido ao fato dele ser um modelo estruturado e pragmático focado na governança do uso de Inteligência Artificial em projetos. Onde o objetivo é garantir uma colaboração ética, eficiente e eficaz entre humanos e IA, mitigando riscos específicos, como vieses algorítmicos e questões de privacidade. Em vez de substituir metodologias de gestão, o AIPGF integra-se de forma fluida a abordagens já consolidadas, como Agile. Na prática, ele operacionaliza normas globais de alto nível, como a ISO/IEC 42001 (ISO/IEC 42001 é a norma internacional para sistemas de gestão de

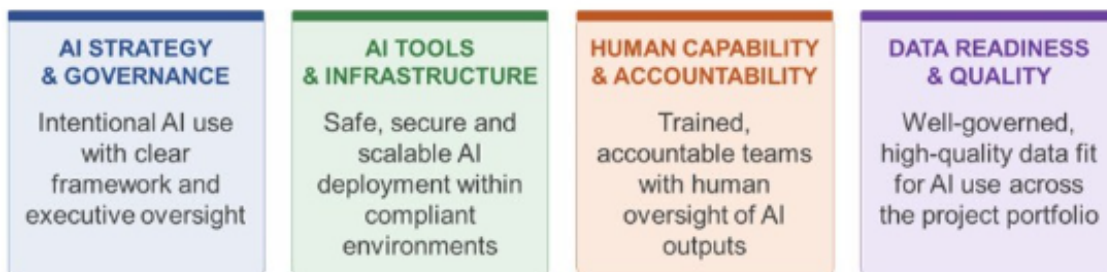
IA), Lei Geral de Proteção de Dados (LGPD) e o *National Institute of Standards and Technology* (NIST), aplicando suas exigências de conformidade diretamente à rotina dos projetos. Todo o framework é guiado por três princípios fundamentais: o centrismo humano, assegurando que a supervisão e a responsabilidade final sejam sempre das pessoas; a transparência, garantindo que o papel da IA seja explicável e auditável; e a adaptabilidade, permitindo que o modelo seja escalado e ajustado conforme o tamanho, a complexidade e o nível de risco de cada iniciativa.

What Does AI Governance Mean in a Project Context?

The Scope of AI Governance



AI Project Governance Pillars



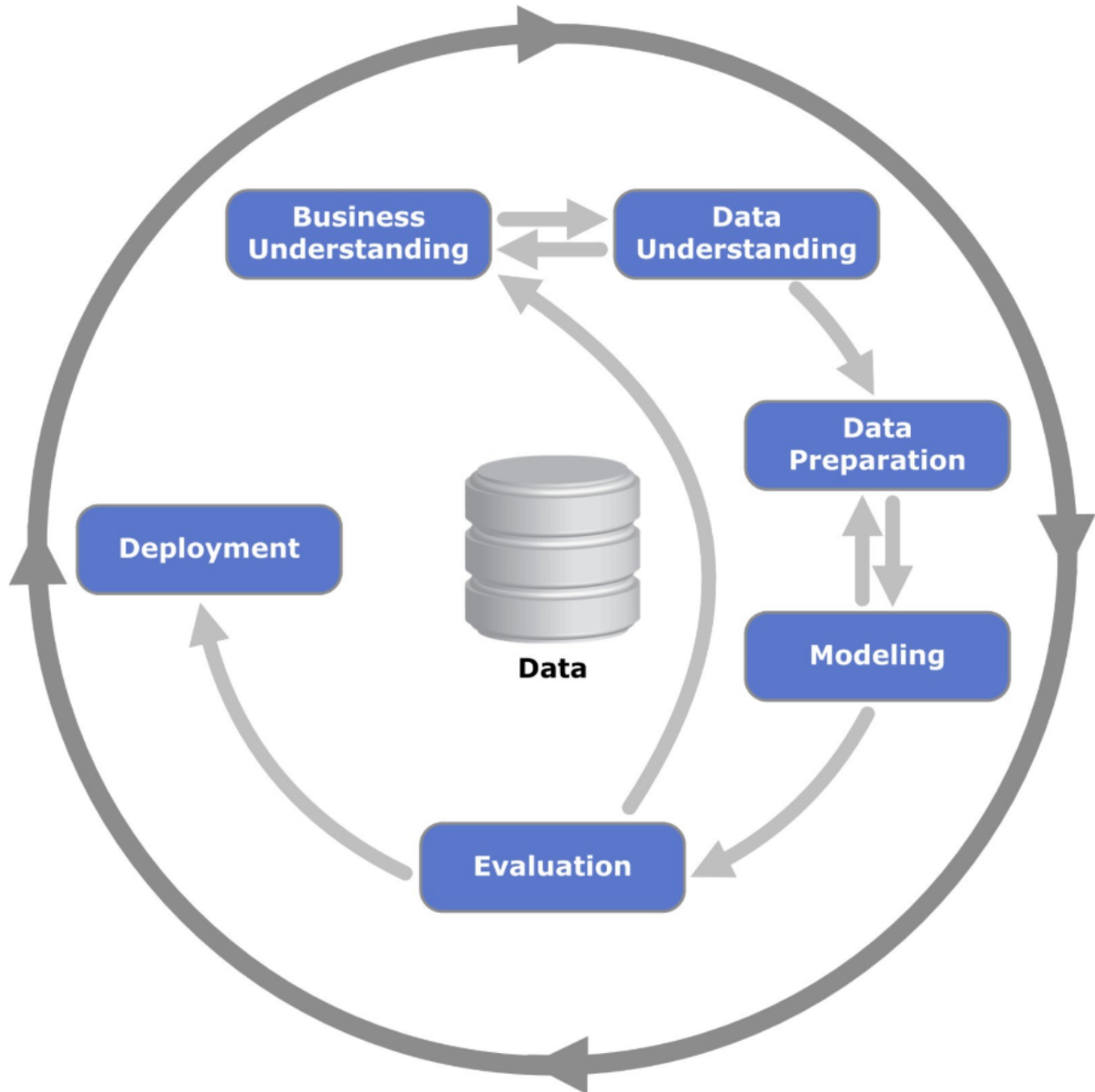
How mature is the governance of AI use in
YOUR projects and programmes?

Complete a free AI Governance Maturity
self-assessment at <https://aipgf.pro>



2.6 Mineração de dados (CRISP-DM)

O *Cross-Industry Standard Process for Data Mining* (CRISP-DM) é uma metodologia estruturada e amplamente adotada para orientar o desenvolvimento de projetos de ciência de dados e inteligência artificial. Seu foco principal é fornecer um ciclo de vida padronizado e iterativo, garantindo que as soluções analíticas estejam estritamente alinhadas aos objetivos estratégicos da organização desde o início. O modelo organiza o projeto em seis fases contínuas e flexíveis: Entendimento do Negócio, Entendimento dos Dados, Preparação dos Dados, Modelagem, Avaliação e Implantação, devido a isso nós utilizaremos CRISP-DM.



2.7 Metodologia de testes

- **Para testes de Dados:** Testes de Viés (Bias) e Equidade (Fairness)

Os testes de Viés e Equidade servem para manter o modelo de IA e *Machine Learning* com decisões justas,

éticas e não discriminatórias.

- **Para testes de código:** Testes Unitários, Teste de integração

Os testes unitários têm como intuito testar cada parte do sistema isoladamente. Esse tipo de teste demanda pouco tempo de elaboração; porém, para cobrir o sistema inteiro a suíte de teste tem que ser muito extensa.

Já os testes de integração, focam em testar a integração entre testes unitários e serviços externos do projeto, demandam mais tempo que os unitários, mais custo, porém aparecem em menor quantidade.

Escolhemos focar nesses dois tipos de teste para desenvolver uma base forte sobre o sistema, em vínculo com os testes de dados e modelos. Na prática, o web-app também ganhou testes de ações simuladas ponta a ponta (`*.e2e.test.ts`, com logs `pino`) e um smoke test de interface com Playwright, executados no CI (seção 8).

Observação. Os testes de viés e equidade previstos para os dados não existiram como suíte própria. As análises que cumprem esse papel são: o controle de autoria (Cramér $V = 0,960$ entre autoria e rótulo), a dependência de marcadores temporais (seção 5.10) e a queda de desempenho no corpus externo (seção 5.9).

2.8 MLOps

MLOps: Disciplina para manter as boas práticas no desenvolvimento de modelos Machine Learning (ML)

Dentro do MLOps, temos 3 níveis separados em:

MLOps

MLOps de nível 0 Cada etapa é manual, incluindo preparação de dados, treinamento de *Machine Learning* e performance e validação de modelos. Isso exige uma transição manual entre as etapas, e cada etapa é executada e gerenciada de maneira interativa. Os cientistas de dados normalmente entregam modelos treinados como artefatos que a equipe de engenharia implanta na infraestrutura da API.

O processo separa os cientistas de dados que criam o modelo e os engenheiros que o implantam, algo que em nosso caso, utilizando a metodologia Crystal não nos encaixou.

Observação. O estado real do projeto é o **nível 0**: preparação, treino e avaliação são feitos manualmente em notebooks, e o CI cobre apenas o web-app. A tentativa de versionar dados com DVC (*pull requests* #18–#21) foi encerrada sem merge. O nível 1 permanece como meta, condicionada à integração do modelo ao sistema (seção 8.5).

MLOps de nível 1 (meta do projeto) O MLOps nível 1 visa treinar o modelo continuamente, automatizando o *pipeline* de *Machine Learning*, onde a maturidade de nível 1 tem as seguintes características:

Etapas rápidas do experimento de *Machine Learning* que envolvem automação significativa

Treinamento contínuo do modelo em produção com dados atualizados como acionadores ativos do *pipeline*

Implementação do mesmo *pipeline* em ambientes de desenvolvimento, pré-produção e produção

MLOps de nível 2 MLOps de nível 2, é adequado em situações em que há atualizações em modelos em minutos, onde é necessário retreinamento a cada hora ou diariamente e os reimplantam simultaneamente em milhares de servidores.

MLOps nível 2, necessita de:

Um orquestrador de pipeline de *Machine Learning*

Um registro de modelos para rastrear vários modelos

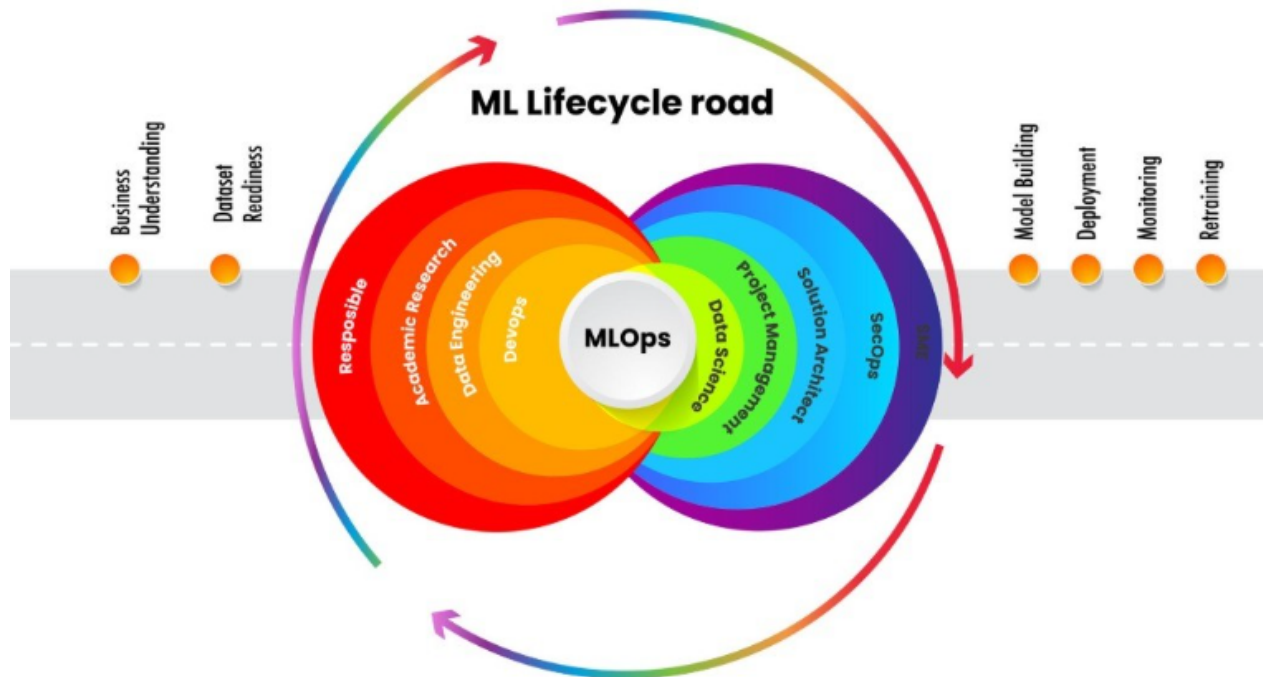
Os três estágios a seguir se repetem em grande escala para vários *pipelines* de *Machine Learning* a fim de garantir a entrega contínua de modelos.

Se divide em 3 etapas:

Construir o *pipeline*

Implantar o *pipeline*

Fornecer o *pipeline*



2.9 Metodologia de pesquisa

- **Levantamento de campo:** Obtenção de dados no ambiente real afetado, principalmente por meio de entrevistas.
- **Bibliográfica:** Pesquisa por meio da seleção e análise dos livros já publicados sobre o tema. Vai ser muito útil para fundamentar o problema.
- **Netnografia:** É uma adaptação da etnografia tradicional, e é voltada para investigação de interações sociais, comportamentos e culturas no meio digital. Vamos usá-la para estudar como o público-alvo é afetado por informações de baixa confiabilidade e como pesquisam sobre, analisando a melhora ou piora do pensamento crítico na internet.

2.10 Protocolo comum de comparação

O repositório define um protocolo comum para que métodos supervisionados e não supervisionados sejam comparáveis ([docs/machine-learning/comparacao-modelos/comparison-protocol.md](#)):

- Rótulos fixos (0 = True, 1 = Fake); IDs, pastas de origem e labels servem para auditoria e nunca

como feature.

- Imputação, normalização, vocabulário TF-IDF e qualquer redução de dimensão são ajustados **somente no treino**.
- Partição canônica com treino, validação e teste e `random_state=42`, sem que um mesmo registro ou grupo (par temático, duplicata) apareça em mais de uma partição. Escolha de hiperparâmetros, limiares e regras usa a validação; o teste só é aberto depois de congelá-las.
- Métricas principais: macro-F1 e balanced accuracy; também precisão e recall da classe Fake, taxa de falsos alertas em True, ROC-AUC e Average Precision. Scores de anomalia não são chamados de probabilidade de falsidade.
- Clusterings que não atribuem notícias novas entram como análise exploratória, fora do ranking de classificação.

Observação. Os dois lados do projeto não seguem a mesma partição. O supervisionado usa divisão aleatória estratificada 80/20 por notícia; o não supervisionado usa partição por grupos (4.320 treino / 1.440 validação / 1.440 teste). O próprio protocolo reconhece que os números supervisionados antigos **não são comparação principal justa** e pede que os baselines sejam reexecutados na partição canônica. Essa reexecução ainda não foi feita ([PENDENTE]). Além disso, o supervisionado trunca em 100 palavras e o protocolo ainda cita 200.

3. Dataset

3.1 Fake.br-Corpus

O corpus utilizado é o **Fake.br-Corpus** (NILC/USP, em revisão fixa 780f5516c4ae070761632d98ac3368f3ded09d35, validada por SHA-256 nos experimentos não supervisionados): **7.200 notícias em português, 3.600 falsas e 3.600 verdadeiras**, com texto completo, metadados e meta-informação linguística de cada uma. As falsas são, em geral, muito mais curtas (mediana de 157 palavras contra 918 nas verdadeiras), diferença que contamina qualquer atributo sensível a tamanho e motivou o truncamento descrito na seção 5.2.

3.2 Seleção de campos

Escolhemos esse dataset porque com ele teremos acesso a 7200 notícias já classificadas, sendo 3600 falsas e 3600 reais, além dos metadados de cada uma. Vamos fazer testes para achar a melhor combinação entre metadados e o texto da notícia, e a princípio já determinamos que alguns não serão necessários. Para maximizar a acurácia do modelo treinado com esse dataset tivemos que fazer alterações nos campos de metadados para trocar de números absolutos, nos campos de número de palavras, verbos, pronomes, substantivos, para taxa desses metadados a cada 100 palavras, a conta utilizada foi: `numero_absoluto_metadado / numero_total_de_palavras * 100`

Campos que não vamos utilizar

Autor: Campo onde cita o redator do texto. Não vamos utilizar

```
100*df_meta['autor'].value_counts()/len(df_meta['autor'])
```

autor	
None	49.652778
Por G1	4.597222
Por BBC	3.263889
José Nêumanne	2.402778
Por G1, Brasília	1.194444
...	
Por Camila Bomfim*, Jornal Hoje, Brasília	0.013889
Por Geisy Negreiros e Janine Brasil, G1 AC, Rio Branco	0.013889
G1 RR	0.013889
Ricardo Brandt. Fausto Macedo, Julia Affonso, Luiz Vassallo, de São Paulo, e Beatriz Bulla e Fábio Serapião, de Brasília	0.013889
Por Rafael Miotto, G1, São Paulo	0.013889
Name: count, length: 1422, dtype: float64	

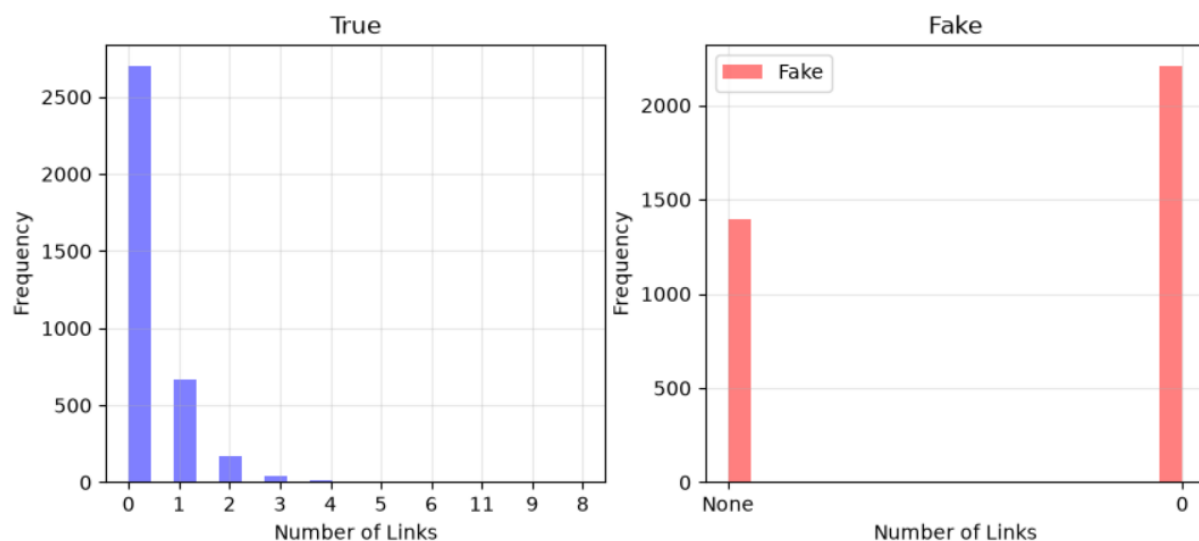
Categoria: Campo para direcionar o grupo de notícias, mas não identificamos como um dado aplicável no nosso caso, visto que os verdadeiros e falsos possuem a mesma quantidade de cada categoria.

```
categoria
politica          2090
tv_celebridades   772
sociedade_cotidiano 638
ciencia_tecnologia  56
economia          22
religiao          22
Name: count, dtype: int64
```

URL: Campo com link da notícia. Não vamos utilizar pois a maioria dos links direciona para 3 sites específicos o que pode acabar criando um viés ruim para o treinamento do modelo.

Label: Responsável por identificar se a notícia é verdadeira ou não então está presente em todas as linhas

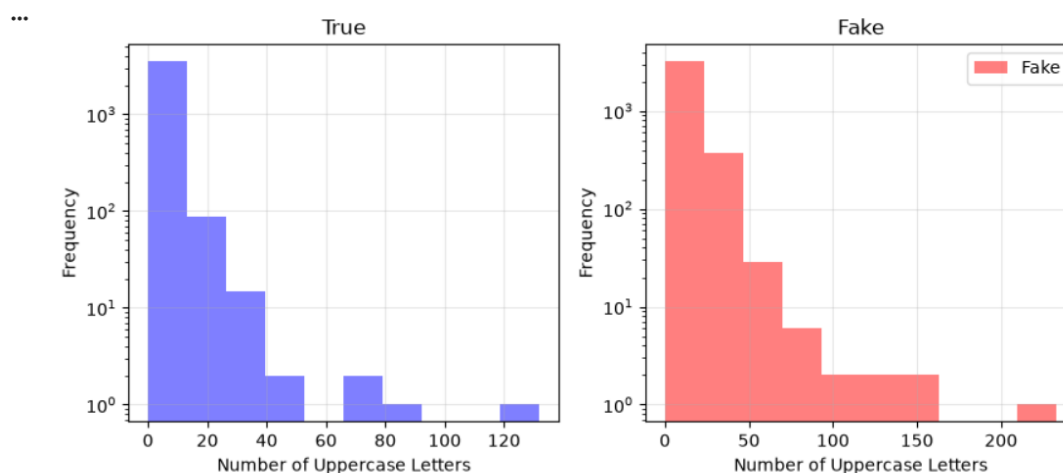
Num_links: Número de links contidos no texto



Campos para considerarmos

Texto: Campo responsável por apresentar o corpo do texto (mudança feita apenas a truncagem dos dados para limitar o número de palavras evitando viés).

Num_maiusculas: Conta o número de maiúsculas no texto. Em comparação o número de maiúsculas aparece muito mais considerando o tamanho do texto.



Campos ainda analisar

Num_verbos; num_verbos; num_verbos_subj_imp; num_substantivos; num_adjetivos; num_adverbios; num_verbos_modais; num_pron_1_2_sing; num_pron_1_plural;

num_pronomes; Pausalidade; pct_erros_ortograficos; Emotividade; Diversidade.

3.3 Corpus externo: FakeRecogna

Para medir generalização fora do Fake.br usa-se o **FakeRecogna** (HuggingFace `recogna-nlp/FakeRecogna`; 11.903 registros, 5.951 por classe). Como a coluna *Noticia* vem lematizada e sem stopwords, e boa parte dos textos "fake" são checagens que desmentem boatos, o teste externo usa **somente títulos**: os 2.484 títulos falsos do domínio `boatos.org` (sem a tag `#boato`) e a mesma quantidade de títulos reais amostrados (`random_state=42`), normalizados pela mesma função do `01_Data_Prep`. A mediana é de **13 palavras por título**, contra textos de até 100 palavras no treino: o teste mede, portanto, generalização de **domínio e de formato** ao mesmo tempo.

3.4 Extração morfossintática própria (spaCy)

Além dos metadados que acompanham o corpus (etiquetados pelo NILC sobre o texto completo), o projeto extrai seus próprios atributos por notícia com o spaCy (`pt_core_news_sm`, módulo `metadados_spacy.py`), sobre o mesmo texto normalizado e truncado que o TF-IDF vê. É a única fonte de metadados aplicável a notícias novas e ao FakeRecogna, que não possuem os metadados do corpus (resultados na seção 5.6).

4. Análise exploratória de dados (EDA)

4.1 Introdução

A análise exploratória de dados consiste em explorar, resumir e visualizar os dados através de uma série de etapas. Com essas etapas buscamos entender a estrutura, testar nossas suposições, identificar anomalias e encontrar padrões e relações entre *features*. A análise exploratória de dados tem algumas variações e cada uma possui objetivos claros a atingir, sendo elas: a análise univariada, que tem o objetivo de entender a distribuição e variação de uma única *feature*; bivariada, que serve para estudar o relacionamento entre duas variáveis; e a análise multivariada, que foca nas interações entre múltiplas variáveis. Neste documento, descreveremos quais análises foram realizadas e as informações obtidas.

Etapas da análise exploratória de dados (EDA)

1. Entender o objetivo e os dados

- O objetivo é treinar um algoritmo de classificação para determinar se uma notícia é verdadeira ou falsa, para nosso projeto.
- Os tipos de dados no dataset encontrado foram Texto, Números (float e inteiros).

2. Importar e inspecionar

Nessa etapa importamos o dataset escolhido (Fake.br corpus) do github para o notebook (.ipynb) e transformamos os arquivos de texto em Data Frames do pandas. Com o intuito de facilitar a exploração, análise e manipulação de dados. Em resumo, no notebook, seguimos os passos de LOAD (carregamento dos dados no ambiente), CHECK (Avaliar os dados), IDENTIFY MISSING (Encontrar valores faltantes), VERIFY DATA TYPE (Verificar os tipos de dados) e FIND ERRORS AND INVALID VALUES (Encontrar erros e valores inválidos no meio do dataset).

3. Lidar com dados faltantes ou inválidos

Para lidarmos com mais certeza, primeiramente precisamos entender o porquê de os dados estarem faltantes ou inválidos, e percebemos que na maioria das vezes era em relação com a fonte, autor.

Após isso partimos para a etapa de remover ou preencher as células destacadas, e nos campos de metadados numéricos, que estavam com o tipo None, preenchemos com 0, já nos campos de fonte e autor, como tinham alta quantidade de valores vazios ou inválidos, decidimos cortar essas features do dataset.

Com isso, o impacto foi de um dataset padronizado com features do mesmo tipo e preenchidas com valores válidos e sem as features que deixariam o modelo desbalanceado.

4. Exploração das características dos Dados

Essa etapa consistiu em entender a distribuição dos dados, para qual utilizamos o boxplot, barcharts, histograms, no ambiente do colab, a tendência central dos dados a partir do boxplot, a viabilidade do dado, a forma de distribuição e encontrar os outliers e anomalias. A etapa consistiu no tipo de EDA Univariada.

Juntamente com as análises de gráficos, também testamos treinar o modelo de regressão logística com diferentes features para explorar melhor e como a feature afeta no modelo.

5. Executar transformação nos dados

Após entender as características, transformar os dados para melhor atender nossa necessidade foi muito mais simples, na exploração havíamos nos alertado com o tamanho do texto, que enviesou o modelo a colocar extrema importância nisso, por isso cortamos os textos (inicialmente em 200 palavras; ver seção 5.2, em que o corte passou a 100 e foi acompanhado de normalização).

No entanto, os campos de metadados com números absolutos por texto por serem muito relativos ao tamanho do texto, isso também se relacionava com o quanto a notícia era falsa ou verdadeira, de forma que para o modelo, nos testes, era só uma notícia falsa ter maiores números absolutos para ser considerada como verdadeira e o modelo errar por isso. Por esse motivo decidimos transformar as features com valores absoluto relativos ao tamanho da notícia em taxas a cada 100 palavras, com a fórmula:

```
numero_absoluto / numero_palavras * 100.
```

Não precisamos usar o label encoding, já que o label estava como binário (0 é notícia verdadeira e 1 é notícia falsa).

Além disso não criamos features novas, estava nos planos adicionar a feature binária se tem autor ou não, porém, através da exploração, percebemos que o valor estava faltando em grande parte das notícias falsas, o que poderia enviesar o modelo.

Durante o processo de transformação, executamos diversos testes no modelo de regressão logística para conferir em tempo real o quanto a mudança fazia efeito, tanto para efeitos que melhoravam, tanto os que pioraram.

6. Visualizar a Relação dos dados

Nosso objetivo nessa etapa foi de descobrir e entender padrões, tendências e relações entre campos, que entra no tipo de EDA Bivariada e Multivariada. Para atingir melhores resultados, usamos gráficos de barra, histogramas, gráficos de dispersão como o pairplot e matriz de correlação, junto com heatmap para facilitar a visualização da matriz.

Com isso, percebemos que havia relações inesperadas entre os dados, como a taxa de maiúsculas ser mais presente em notícias com o label 1(falsas) a diversidade (número de palavras diferentes dividido pelo número de palavras) estar fortemente relacionada às notícias com o label 1 (falsas), efeito do comprimento do texto (ver a observação ao fim desta seção).

A visualização da relação entre dos dados foi ainda mais presente em conjunto com o modelo de regressão logística, que indicava o tamanho da relação ao omitir dados, incluir ou tratar o dado de forma diferente.

7. Lidar com Outliers

Para os Outliers, utilizamos principalmente o boxplot para identificá-los, no entanto, não executamos nenhuma ação de capping ou transformação nesses dados, já que durante as nossas pesquisas o que achamos foi que só outliers são excluídos apenas quando devidamente entendidos e incorretos.

8. Comunicar resultados

Estamos comunicando os resultados por meio desse documento sobre a nossa análise exploratória de dados com foco no dataset escolhido do Fake.br Corpus, nossos resultados até agora foi um treinamento do modelo de regressão logística de 92% de acurácia com apenas o texto cortado em 200 palavras e tratado pelo método TF-IDF, e 95% de acurácia com o texto (tratado pelo TF-IDF) junto com os metadados da notícia.

O FP-Growth principal descreve padrões recorrentes, enquanto o supervisionado estima uma classe. Seus protocolos e medidas não permitem afirmar que um seja pior que o outro.

Observação. Os resultados citados na etapa "Comunicar resultados" (92% de acurácia só com texto e 95% com metadados) são das primeiras rodadas, com corte em 200 palavras e sem normalização, na Regressão Logística. Em rodadas posteriores os metadados pioraram a LR (-2,4 pontos de F1 em word+meta, protocolo anterior), o que levou a não adotá-los como bloco principal; a evidência no pipeline atual é conflitante (seção 5.6).

Observação. A diversidade (razão entre tipos e tokens) correlaciona-se positivamente com o rótulo Fake ($r \approx 0,80$ nos textos completos): é maior nas notícias falsas, e não nas verdadeiras, como a etapa "Visualizar a relação dos dados" afirmava. A causa é o comprimento: a razão tipos/tokens decresce com o tamanho do texto e as falsas são cerca de 6× mais curtas (mediana de 157 contra 922 palavras). Recalculada sobre os 7.200 textos do corpus (minúsculas, tokens \w+), a correlação cai para $r \approx 0,17$ com truncamento em 100 palavras (o resíduo vem dos 16,8% de falsas com menos de 100 palavras) e para $r \approx -0,09$ com a MATTR de janela 100 sobre o texto completo. Com o tamanho controlado, a diversidade praticamente não separa as classes.

4.2 Correlação entre variáveis

A fim de compreender quais *features* possuem maior impacto nas *labels*, além verificar se existem colinearidade ou multicolinearidade em nosso banco de dados, fizemos uma análise bivariável, a matriz de correlação, e uma análise multivariável, por meio do cálculo do fator de inflação de variância (VIF).

Pairplot

Por meio do *pairplot*, fizemos uma inspeção visual das variáveis de metadados do *dataset*, antes da criação da matriz de correlação. Geramos esse gráfico com a função `.pairplot()` da biblioteca *seaborn*. Esse gráfico organiza as variáveis em uma grade onde a diagonal principal mostra a distribuição de cada variável, e as outras células o *pairplot* apresenta os gráficos de dispersão de variáveis comparadas em pares. Com isso, conseguimos analisar ao mesmo tempo o formato da distribuição e a relação entre os pares de variáveis. Além dessas duas formas de visualização, colorimos os pontos com laranja para notícias falsas e azul para notícias verdadeiras, o que permite verificar se alguma variável isoladamente já separa notícias falsas de verdadeiras, o que aconteceu em grande volume na variável de diversidade, onde os pontos laranjas estão mais separados dos pontos azuis.

Nos gráficos de dispersão, alguns pares de variáveis formam nuvens alongadas em torno de uma reta, ainda que ruidosas, como é o caso de taxa de adjetivos com taxa de advérbios e taxa de substantivos com taxa de adjetivos, o que reforça a viabilidade de aplicar o coeficiente de Pearson nessas relações. Essa parte do *pairplot* mostra a relação de dispersão de relação das variáveis de diversidade, taxa de advérbios e emotividade. A partir dele, identificamos que a diversidade se relaciona de forma que a dispersão de notícias falsas e verdadeiras é alta, já no gráfico de dispersão de emotividade com taxa de advérbios a dispersão é baixa e formam uma nuvem alongada em torno de uma reta, o que reforça ainda mais a viabilidade de aplicar o coeficiente de Pearson. Disponibilizamos abaixo os gráficos citados, para ver o *pairplot* completo acesse os anexos do documento.

Matriz de correlação

Para a criação da matriz de correlação, aplicamos a função do Pandas `.corr()` optando pelo método padrão de Pearson para calcular os coeficientes. Tomamos essa decisão pois como podemos analisar no gráfico anterior, o *Pair plot*, as variáveis possuem distribuição normal e quando comparadas dois a dois formam uma reta, apesar de ruidosa. Para melhor visualização utilizamos o *heatmap* de correlação e dessa forma comparação das variáveis ganha uma coloração, onde valores próximos do -1 são sinalizados com uma tonalidade azul, indicando variáveis correlacionadas inversamente; valores próximos de 0 indicam pouca correlação e recebem tons claros; já os quadrados avermelhados sinalizam as variáveis que estão correlacionadas positivamente, com valores próximos do 1. Na imagem abaixo disponibilizamos o *heatmap* de correlação.

Note que as variáveis com maior correlação positivas maiores que 0,5 e menores que 0,7, consideradas moderadas, são: taxa de verbos com taxa de verbos modais (0,52), taxa de erros ortográficos com taxa de advérbios (0,68), taxa de emotividade com taxa de advérbios (0,68) e taxa de pronomes com taxa de pronomes no singular (0,53); as variáveis com correlação alta, sendo maior que 0,7 são: emotividade com taxa de adjetivos (0,74), e a diversidade com as *labels* (0,8); já as variáveis com correlação negativa moderadas, estando entre -0,5 e -0,7, são: taxa de substantivos com taxa de verbos (-0,54), taxa de advérbios com taxa de substantivos (-0,57), taxa de pronomes com taxa de substantivos (-0,6), taxa de emotividade com taxa de substantivos (-0,57) e tamanho médio de sentença com diversidade (-0,52); e não encontramos correlação com valores menores que -0,7. Sendo assim, nas próximas análises nos atentaremos a essas variáveis, principalmente aquelas que em módulo são maiores que 0,7, pois podem estar apresentando informações redundantes ao nosso modelo e por consequência aumentando sua carga de processamento.

Fator de inflação de variância (VIF)

Feito o levantamento das variáveis que mereciam maior atenção, avaliamos o nível de multicolinearidade do nosso modelo por meio do método Fator de inflação de variância (VIF). Abaixo disponibilizamos a tabela de valores de VIF para cada variável, onde 3 atingiram um valor do acima de 10, o que é preocupante, pois indica altos níveis de multicolinearidade. São elas: emotividade (VIF = 106), taxa de adjetivos (VIF = 45) e taxa de advérbios (VIF = 41).

Esse resultado, apesar de preocupante, era esperado, pois a emotividade é uma variável criada a partir dos valores da taxa de adjetivos e taxa de advérbios, então há dependência entre elas. Por isso, decidimos remover

essas duas variáveis e calcular novamente o VIF. Segue abaixo a tabela com os valores finais, note que todos se mantiveram abaixo de 5, um resultado satisfatório.

[PENDENTE] As tabelas de VIF (antes e depois de remover emotividade, adjetivos e advérbios) e o pairplot completo são citados, mas não estão no documento.

4.3 Manipulação de dados

- **Texto:** Campo utilizado para as demais validações.
- **Autor:** Identificamos que nas notícias falsas o campo autor não estão preenchidos, devido a isso esse campo pode acabar influenciando o modelo negativamente. Abaixo extração da informação vindo das notícias falsas.
- **URL:** No documento muito dos links se encontram como "quebrados" logo não tornando relevante a sua utilidade, outro ponto importante.
- **Categoria:** O campo categoria também não apresenta uma informação relevante pois ambos os grupos possuem categorias espelhadas.
- **Número de links dentro do texto:** Dentro do texto o número de informações é muito baixo logo a informação apresentada não apresenta relevância para o treinamento do modelo.
- **Data de publicação:** Para o nosso modelo este campo não é relevante pois quando o usuário inserir a notícia não estaremos usando essa informação, além disso o fator período de amostragem do *dataset* pode acabar confundindo o modelo uma vez que nenhuma nova notícia inserida será desse mesmo período.
- **Números de tokens:** (verificar)
- **Número de caracteres:** O número de caracteres por fazer a contagem individual de cada letra acaba usando de muito processamento além de ser um campo com informações relativamente repetitivo, treinar o modelo sem ele não fez alteração relevante no nível de acurácia.
- **Número de tipos:** Esse campo está sendo utilizado para armazenar um dado útil para um segundo atributo, optamos por não armazenar esse dado e fazer o cálculo direto em linha de código futuramente para evitar armazenamento redundante.
- **Diversidade:** Este campo corresponde a variação de palavras dentro do texto. Usando a *feature importance* foi apresentado que esse atributo gera informações relevantes.
- **Pausalidade:** Este campo apresenta a pontuação dentro do texto e em nossa extração, verificamos que notícias verdadeiras possuem mais pausalidade do que notícias falsas.
- **Tamanho médio da sentença:** No tamanho médio, identificamos também diferença relevante entre notícias verdadeiras e falsas.
- **Tamanho médio de palavra:** Em comparação ao campo informado anteriormente o valor médio de palavras se encontra bem menos relevante.
- **Número de palavras sem pontuação:** Este campo não se apresentou relevante uma vez que durante o passar do tempo a mudança textual é muito comum Ex: a queda da trema.
- **Número de verbos subjuntivos e imperativos:** Transformamos esse número de verbos subjuntivos e imperativos na taxa dos mesmo a cada 100 palavras, porque como já havíamos determinado que o modelo ficaria enviesado com o tamanho da notícia, já que nosso dataset tem notícias verdadeiras, em geral, seis vezes maiores que falsas, e com os números absolutos, a correlação entre tamanho da notícia e número absoluto de verbos é direta. Além disso, identificamos que o campo da taxa de verbos subjuntivos e imperativos tem correlação de aproximadamente 0.1 com a label (se é falsa ou verdadeira), e por isso decidimos manter essa variável no dataset.

- **Número de substantivos:** Também transformamos esse campo em taxa a cada 100 palavras, ao invés do número absoluto, e não tinha uma correlação tão alta com a label, porém decidimos manter no dataset porque afetava positivamente no treinamento do modelo. A seguir é a frequência de taxa de substantivos em notícias falsas e verdadeiras.
- **Número de adjetivos:** Também transformamos esse campo em taxa a cada 100 palavras, ao invés do número absoluto, e não tinha uma correlação tão alta com a label, porém decidimos manter no dataset porque afetava positivamente no treinamento do modelo.
- **Número de verbos:** Esse campo foi transformado em taxa de verbos, e apresentou alta correlação com o campo de label (0.258), além de alta importância no treinamento do modelo.
- **Número de palavras em letra maiúscula:** Esse campo foi transformado em taxa de palavras em letra maiúscula, e apresentou alta correlação com o campo de label (0.259), além de alta importância no treinamento do modelo.
- **Número de advérbios:** Esse campo foi transformado em taxa de advérbios, e apresentou baixa correlação com o campo de label (0.05).
- **Número de verbos modais (principalmente verbos auxiliares), Número de pronomes:** Os três próximos campos estão sendo utilizados por serem dados complementares em emotividade, consideramos que mesmo tendo baixo fator de impacto testando no modelo eles possuem informações relevantes para criar o campo emotividade.
- **Número de caracteres:** Aqui é feita a contagem de cada caractere do texto e verificamos que os outros campos já possuem as informações necessárias logo esse campo se torna um armazenamento redundante.
- **Porcentagem de erros ortográficos:** Esse campo se apresenta muito coerente na identificação uma vez que texto com diversos erros ortográficos apresentam pouca confiabilidade.

Observação. Lista final de metadados no 01_Data_Prep: taxa_maiusculas, taxa_verbos, taxa_verbos_subj_imp, taxa_substantivos, taxa_verbos_modais, taxa_pron_1_2_sing, taxa_pron_1_plural, taxa_pronomes, pausalidade, pct_erros_ortograficos, tam_medio_sentenca, tam_medio_palavra e diversidade_relativa (MATTR, janela de 100 tokens, calculada sobre o texto truncado). **Adjetivos, advérbios e emotividade ficaram de fora**, em coerência com o VIF; a descrição do campo "emotividade" acima (modais e pronomes) deve ser revista, pois no corpus ela deriva de adjetivos e advérbios.

4.4 Visualizações

Boxplot

O painel de Análise Exploratória de Dados (EDA) está estruturado em uma grade de nove boxplots que comparam a distribuição de nove métricas linguísticas e estruturais extraídas do FakeCorpus BR entre notícias verdadeiras (classe 0, representadas em verde) e falsas (classe 1, em vermelho). O objetivo dessa visualização é examinar assimetrias, tendências centrais e o potencial discriminatório individual de cada atributo antes da modelagem preditiva, com os valores extremos ocultados para privilegiar a clareza da densidade central. A leitura de cada gráfico revela a mediana por meio da linha central da caixa, enquanto a extensão e as hastes mapeiam a dispersão dos dados para cada categoria.

A análise detalhada evidencia comportamentos distintos entre os indicadores. A métrica de **diversidade** apresenta uma das separações mais marcantes, onde as notícias falsas exibem uma mediana consideravelmente superior. Atributos como a taxa de letras maiúsculas e o percentual de erros ortográficos também se concentram em patamares mais elevados no grupo desinformativo, apontando para o uso frequente de ênfases visuais e menor rigor revisional. Por outro lado, variáveis como a taxa de verbos e de verbos modais mostram elevações sutis, enquanto métricas como a emotividade, o uso de adjetivos, advérbios e os verbos no subjuntivo ou imperativo apresentam distribuições bastante sobrepostas entre as classes.

Essa sobreposição generalizada nos atributos isolados demonstra que, de forma individual, a maioria das métricas não é suficiente para isolar perfeitamente os grupos.

Notebook da EDA: <https://colab.research.google.com/drive/1Al1scCzAe9uQWYXc2L8bfyXeo4ZYPvGi?usp=sharing>

4.5 Evidências de artefatos de fonte

Três achados independentes mostram que parte do sinal do corpus vem da fonte, e não da linguagem da desinformação; eles orientam as decisões de pré-processamento e a leitura de todas as métricas:

1. **Autoria:** 3.528 de 3.601 notícias sem autor são Fake e 3.527 de 3.599 com autor são Real (Cramér $V = 0,960$). O campo foi excluído do modelo supervisionado e da descoberta de padrões.
2. **Formatação:** no texto bruto, 2,93% dos tokens das falsas são espaços/quebras (SPACE) e 3,21% recebem a relação **dep** do parser, contra 0,38% e 0,73% nas verdadeiras (treino, notebook 10). Também a tipografia (travessão, aspas curvas, [...], caracteres cp1252) só aparece nas verdadeiras. **Depois da normalização e do truncamento, SPACE cai a 0,00% nas duas classes e dep fica em 0,40% contra 0,41%:** o artefato foi eliminado (seção 5.6).
3. **Convenções de redação:** marcadores temporais ("nesta", "quarta", "feira") concentram-se nas verdadeiras e afetam a classificação (seção 5.10).

4.6 EDA das features morfofssintáticas (spaCy: POS e DEP)

Esta seção documenta a exploração das features extraídas com o spaCy (seção 3.4), que antecede a seleção feita nos notebooks 10 a 12 (seção 5.6).

Extração e preparação

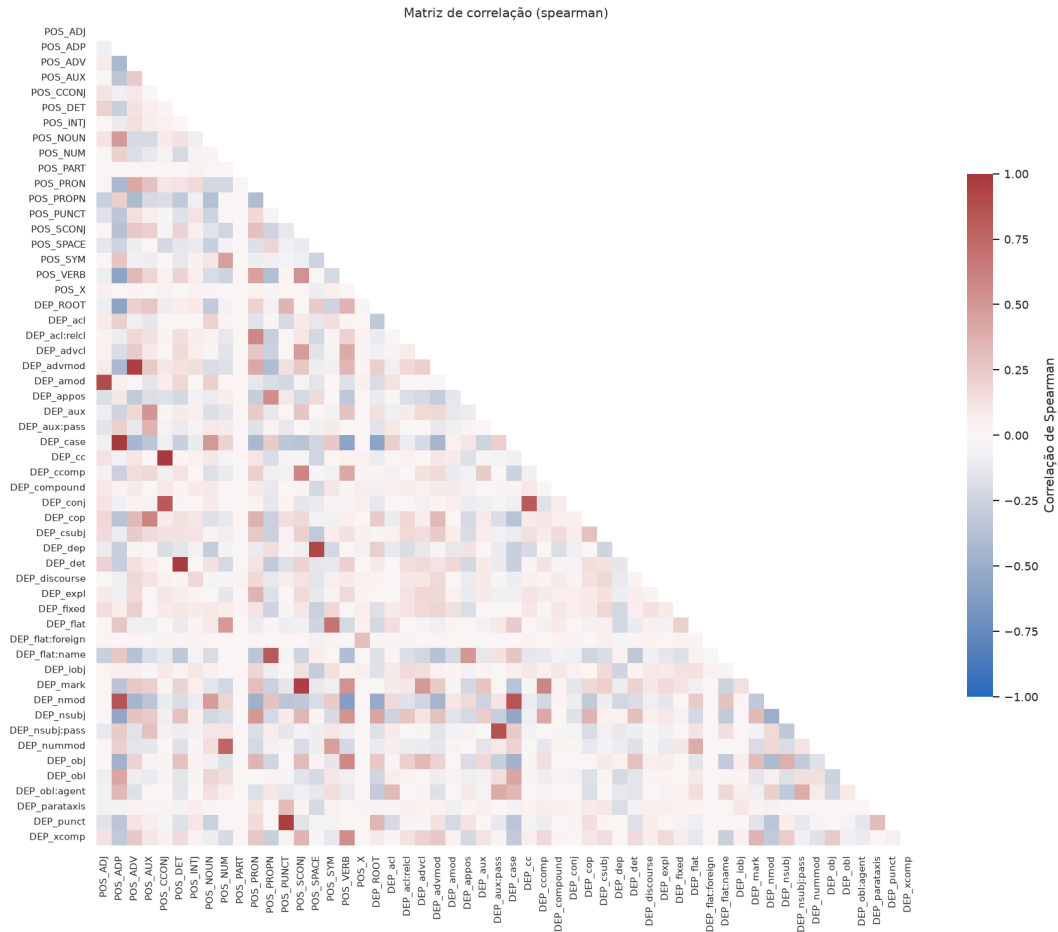
Após carregar o Fake.br Corpus, os textos foram processados com o spaCy. Como o volume era grande, o dataset foi particionado em blocos de 200 textos e, como ainda era pesado para a extração, em partes menores de 16 textos. De cada trecho extraíram-se, para cada palavra, a classe gramatical (**POS**, *Part Of Speech*) e a relação de dependência sintática (**DEP**) na sentença:

- **POS (17 classes):** ADJ, ADP, ADV, AUX, CONJ, DET, INTJ, NOUN, NUM, PART, PRON, PROPN, PUNCT, SCONJ, SYM, VERB e X (mais SPACE, usada como diagnóstico de formatação).
- **DEP (35 relações):** entre elas ROOT, nsubj, obj, iobj, obl, nmod, amod, advmod, det, case, conj, cc, acl, advcl, appos, compound, flat, fixed, mark e punct.

Todos os valores foram contabilizados como **proporção em relação ao total de tokens do texto**, e não como números absolutos, pelo mesmo motivo do tamanho do texto discutido na seção 4.1.

Matriz de correlação de Spearman

Medimos a correlação de Spearman entre todas as features POS e DEP. Várias características gramaticais se repetem entre as duas famílias ou têm correlação muito alta (por exemplo, POS_DET e DEP_det; POS_ADP e DEP_case), o que já indicava candidatas à remoção.



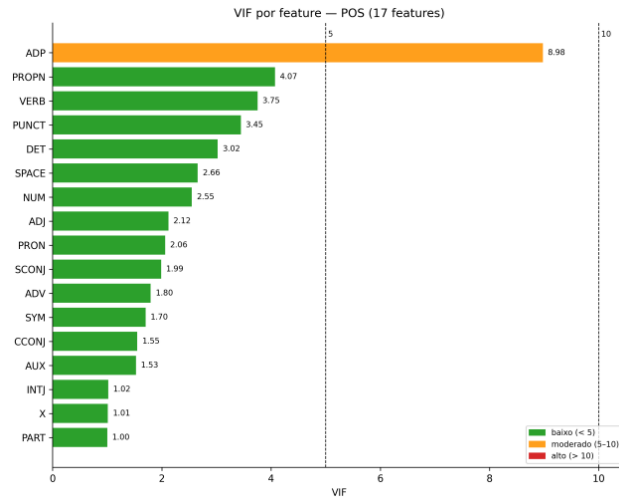
Matriz de correlação de Spearman entre as features POS e DEP. Fonte: relatório de EDA (EDA_V1_06).

VIF por família

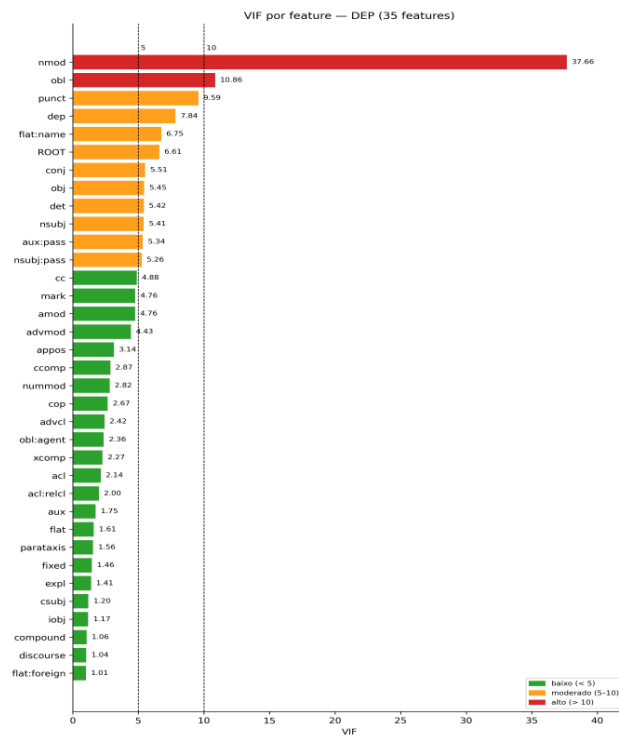
O VIF foi calculado separadamente para POS e DEP:

- **POS:** todas abaixo de 5, exceto ADP (VIF = 8,98, faixa moderada).
- **DEP:** nmod (VIF = 37,66) e obl (10,86) ficam acima de 10; punct (9,59), dep (7,84), flat:name (6,75) e ROOT (6,61) ficam na faixa moderada (5 a 10).

nmod é o ponto fora da curva: sua relevância e comportamento foram examinados à parte antes de decidir pela remoção.



VIF por feature POS. Fonte: relatório de EDA (EDA_V1_06).

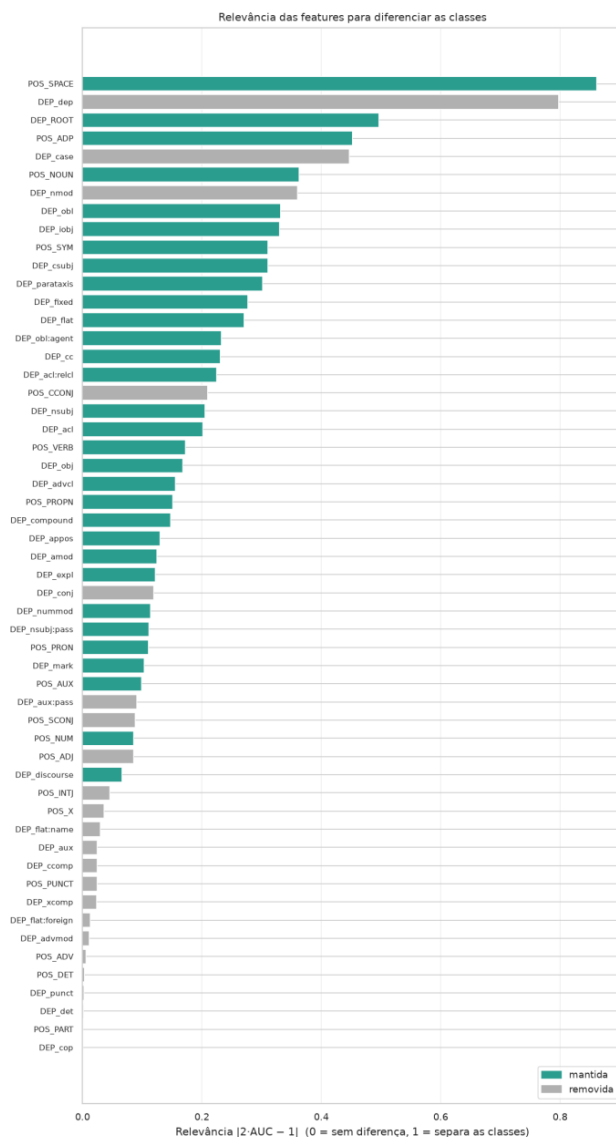


VIF por feature DEP. Fonte: relatório de EDA (EDA_V1_06).

Relevância univariada por AUC

Para cada feature calculou-se a relevância $|2 \cdot \text{AUC} - 1|$ (0 = não diferencia as classes; 1 = separa por completo) e marcou-se se ela foi mantida ou removida. Destaques:

- Mais relevantes: POS_SPACE ($\approx 0,86$), DEP_dep ($\approx 0,80$), DEP_ROOT ($\approx 0,50$), POS_ADP ($\approx 0,45$), DEP_case ($\approx 0,45$), POS_NOUN e DEP_nmod ($\approx 0,36$).
- Relevância praticamente nula: POS_PART, POS_DET, DEP_det, DEP_punct e DEP_cop, removidas.
- Também removidas as redundantes (DEP_det, DEP_case, DEP_nmod) e as de pouco poder discriminativo (POS_INTJ, POS_X, POS_PUNCT, POS_ADV, DEP_advmod, entre outras).



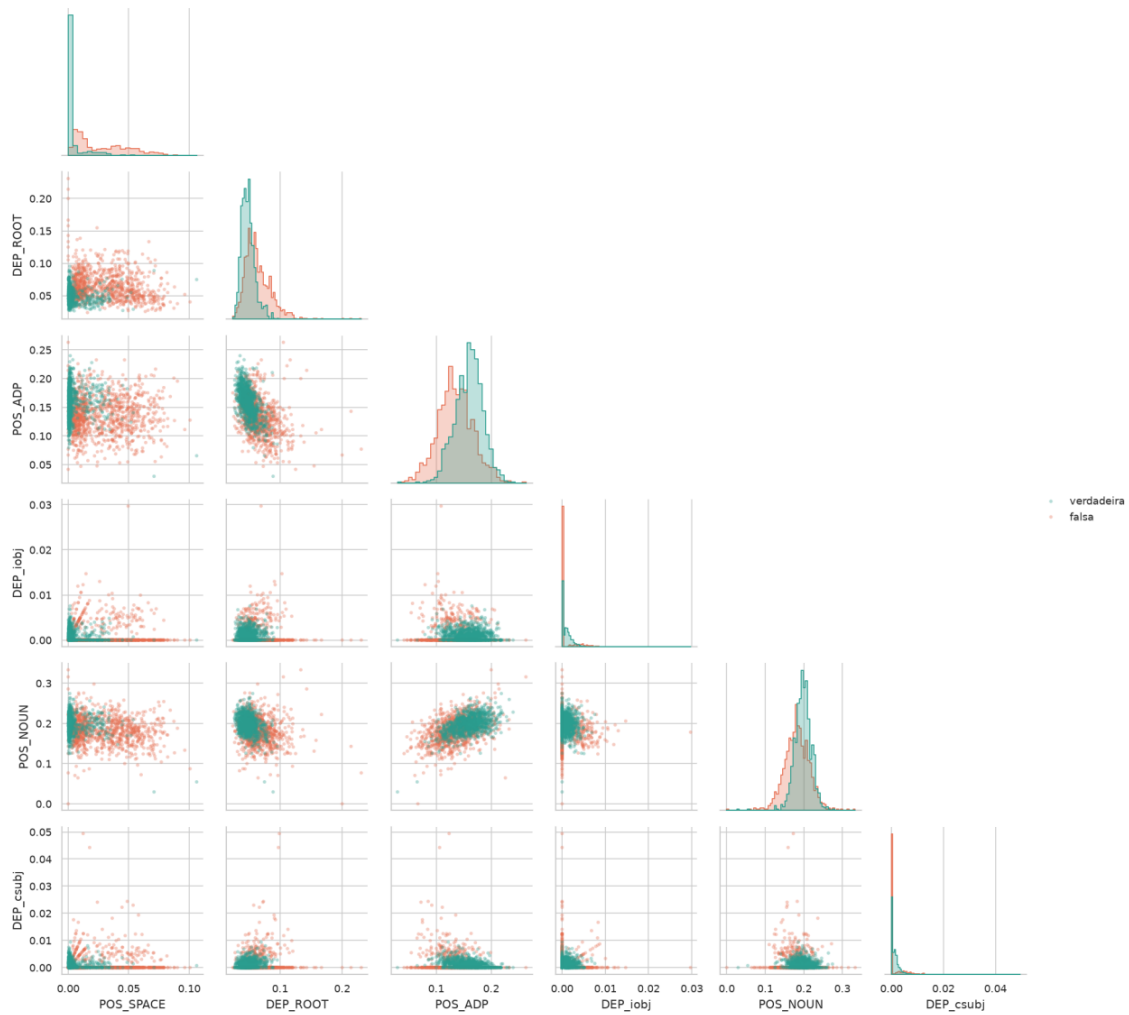
Relevância univariada $|2 \cdot AUC - 1|$ das features (mantidas e removidas). Fonte: relatório de EDA (EDA_V1_06).

Atenção. POS_SPACE e DEP_dep lideram a relevância porque medem **formatação do texto bruto** (quebras de linha e espaços, mais frequentes nas falsas), e não a linguagem da desinformação (seção 4.5, item 2). Na EDA a POS_SPACE aparece como “mantida”, mas no pipeline final elas viram colunas de diagnóstico (**diag_***) fora do modelo, e o artefato desaparece após normalização e truncamento (seção 5.6).

Pair grid, histogramas e boxplots por classe

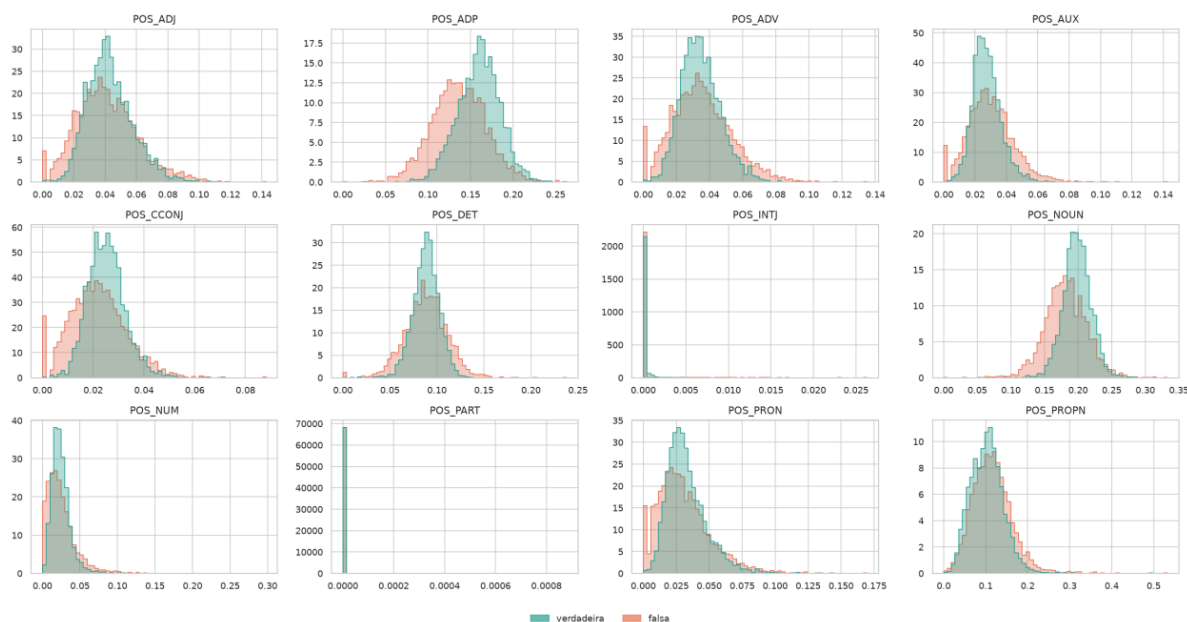
- **Pair grid** (POS_SPACE, DEP_ROOT, POS_ADP, DEP_iobj, POS_NOUN, DEP_csubj): permite ver agrupamentos e outliers e checar as hipóteses. Em sua grande maioria, as notícias verdadeiras têm comportamento mais previsível (nuvens mais concentradas) do que as falsas, que se espalham mais.
- **Histogramas por classe:** confirmam estrutura gramatical diferente entre verdadeiras e falsas em várias features (por exemplo POS_ADP, POS_NOUN, POS_CCONJ, POS_PRON). POS_PART e POS_INTJ não têm comportamento significativo (praticamente só zeros), por isso saíram dos dados de treino.
- **Boxplots por classe:** reforçam os histogramas e mostram os outliers e a densidade central. Notícias

verdadeiras têm maior mediana de POS_ADP e POS_NOUN; POS_INTJ e POS_NUM têm outliers apenas nas falsas.



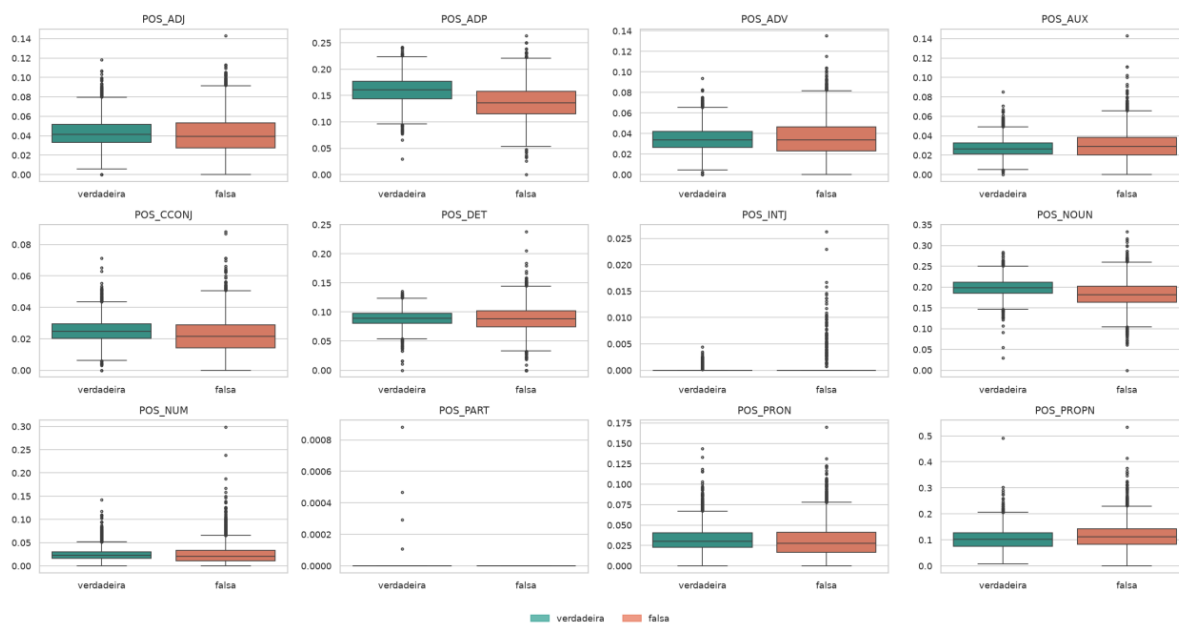
Pair grid das features mais relevantes, por classe. Fonte: relatório de EDA (EDA_V1_06).

Histogramas por classe (página 1)



Histogramas por classe (página 1). Fonte: relatório de EDA (EDA_V1_06).

Box plots por classe (página 1)



Boxplots por classe (página 1). Fonte: relatório de EDA (EDA_V1_06).

5. Aprendizado supervisionado

5.1 Classificadores candidatos

Tendo em vista um problema de classificação binária e os indicadores definidos na seção 1.4, foram levantados, por pesquisa bibliográfica, os principais classificadores supervisionados. O quadro abaixo documenta vantagens e desvantagens; foram treinados Regressão Logística (baseline interpretável), SVM (candidato principal)

e Random Forest (contraponto não linear baseado em árvores).

Analisando a tabela, os prós e contras de cada aprendizado e os problemas que se adequam melhor, decidimos treinar os modelos Regressão logística, SVM e *Random forest*.

Regressão logística: esse modelo se destacou devido a facilidade de encontrar quais características mais impactaram o modelo; ser adequado para classificar em problemas como nosso, onde o resultado é discreto; e retornar um percentual, que podemos fornecer aos usuários para que tenham noção do nível de confiabilidade que nossa informação possui. Além disso, durante a limpeza dos dados, faremos o tratamento para evitar ruídos e *outliers*, assim como já separamos os parâmetros de tal forma que não sejam dependentes entre si.

Random forest: este aprendizado tem qualidades que supomos que irão se destacar em nosso projeto. Entre essas qualidades, as principais são: menor probabilidade de ocorrer *overfitting*, o que é de suma importância para reduzir o risco de retornarmos informações errôneas; ser fácil encontrar quais variáveis geram maior impacto para o modelo por meio de diferentes métodos, sendo assim podemos comparar facilmente com nossas hipóteses de parâmetros mais importantes; além de poder ser utilizado em modelos com relações lineares ou não, sendo mais flexível nesse sentido.

SVM: Como o *random forest*, um dos principais atrativos do SVM são o baixo risco de *overfitting*. Além disso, sua diversidade de modelos aplicáveis a dados linearmente separáveis ou não traz uma flexibilidade para o desenvolvimento do modelo uma vez que teremos apenas que fazer uma pré-análise dos dados para ver como se agrupam e qual formato de treinamento será mais adequado.

<i>Classificador</i>	<i>Descrição</i>	<i>Vantagens</i>	<i>Desvantagens</i>
Regressão logística	É um tipo de algoritmo de classificação que prevê um resultado discreto ou categórico.	· Permite analisar o peso de cada variável de entrada no resultado com clareza; · Retorna a chance percentual de um dado pertencer a uma classe, e não apenas uma resposta fixa;	· Assume uma relação linear entre as variáveis; · Sensibilidade a ruídos e <i>outliers</i>; · Pressupõe que as variáveis preditoras sejam independentes entre si, sofrendo quedas de desempenho em casos de forte multicolinearidade.
Random forest	É um algoritmo que combina resultados de saída de múltiplas <i>decision trees</i> para alcançar um único resultado.	· Redução do risco de <i>overfitting</i>; · Facilita a avaliação da importância ou contribuição das variáveis para o modelo.	· Processo demorado; · Mais complexo de interpretar do que a <i>decision tree</i>.
SVM	É um algoritmo que classifica dados encontrando uma linha ou hiperplano ótimo que maximiza a distância entre cada classe em um espaço N-dimensional.	· Pode ser utilizado para dados linearmente separáveis ou não; · Menos sensível ao <i>overfitting</i>.	· Pode ser mais custoso em termos computacionais que o Naive Bayes e a regressão logística; · Ajuste complexo na escolha do <i>kernel</i>; · Dados com muitas sobreposições de classes ou ruídos prejudicam bastante o desempenho do modelo.

<i>Classificador</i>	<i>Descrição</i>	<i>Vantagens</i>	<i>Desvantagens</i>
KNN	É um classificador não paramétrico, que usa a proximidade para fazer classificações ou previsões sobre o agrupamento de um determinado ponto de dados. Principalmente utilizado para pré-processamento de dados, mecanismos de recomendação e reconhecimento de padrões.	· Fácil de implementar; · Adapta-se facilmente a novos dados; · Requer apenas um valor de k e uma métrica de distância.	· Consome mais memória e armazenamento de dados em comparação com outros classificadores; · Propenso a excesso de ajuste: valores mais baixos de k podem ajustar demais os dados e, se o valor de k for muito alto, ele poderá não se ajustar aos dados.
Árvore de decisão	Algoritmo não paramétrico que possui estrutura hierárquica em árvore, que consiste em um nó raiz, ramificações, nós internos e nós de folha.	· Fácil de interpretar; · É necessária pouca ou nenhuma preparação de dados.	· Propenso a <i>overfitting</i>; · Pequenas variações nos dados podem produzir uma árvore de decisão muito diferente; · O treinamento pode ser mais caro do que o de outros algoritmos.
Naive Bayes	É um algoritmo que usa princípios de probabilidade para realizar tarefas de classificação.	· É considerado um classificador mais simples, já que os parâmetros são mais fáceis de estimar; · Baixos requisitos de armazenamento; · É considerado rápido e eficiente, com precisão razoável quando a suposição de independência condicional é válida.	· Embora a suposição de independência condicional funcione bem no geral, ela nem sempre é válida; · Sujeito à frequência zero quando uma variável categórica não existe no conjunto de treinamento.

5.2 Preparação dos dados

O pipeline oficial está em `01_Data_Prep.ipynb` e gera os artefatos `dados_preparados.pkl` e `transformers.pkl` (ignorados pelo git; reproduzíveis a partir do notebook). As etapas são:

1. **Carga e rotulagem:** textos completos de `full_texts/fake` e `full_texts/true`, unidos às colunas de meta-informação.
2. **Normalização (normalizar):** remove marcas de formatação específicas de cada fonte (caracteres de controle do cp1252, travessões e aspas curvas, reticências e [...], reduzidos a um ponto colado à palavra anterior) e converte todos os dígitos em 0, pois datas e valores diferem entre as fontes. Sem isso o modelo aprende a fonte em vez do conteúdo.
3. **Truncamento em 100 palavras (truncar).** Deixa as classes com comprimento comparável e impede que o tamanho do texto, que o modelo explorava, domine a classificação.
4. **Metadados em taxa:** contagens absolutas viram taxas a cada 100 palavras (`absoluto / num_palavras * 100`); a diversidade é recalculada sobre o texto truncado (MATTR).
5. **Split 80/20** estratificado, `random_state=42`: 5.760 textos de treino e 1.440 de teste, em partes iguais entre as classes.
6. **Stop words:** o modelo as mantém, pois carregam estilo de escrita e melhoram o F1; uma versão sem stop words (`texto_limpo`) existe apenas para análise (heatmaps e SHAP).

Observação. Até 25/09 o pipeline usava truncamento em **200 palavras sem normalização**. A mudança (commit `cc6f2ad6`) reduziu os números de validação. As tabelas de desempenho por

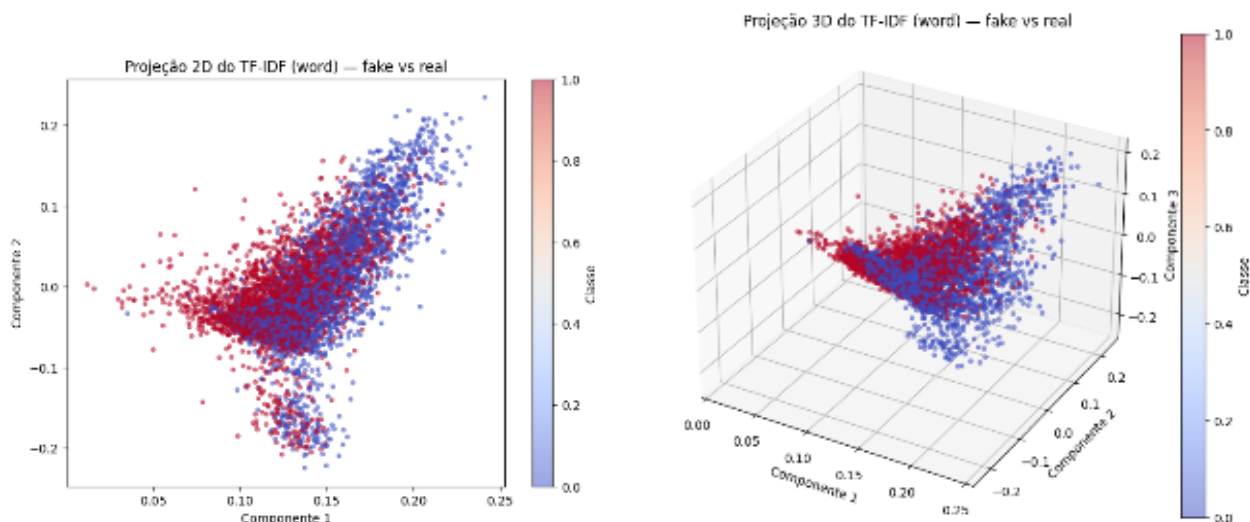
modelo da seção 5.4 foram produzidas **no protocolo anterior** e estão sinalizadas como tal; os números do protocolo atual são os das seções 5.5 a 5.9.

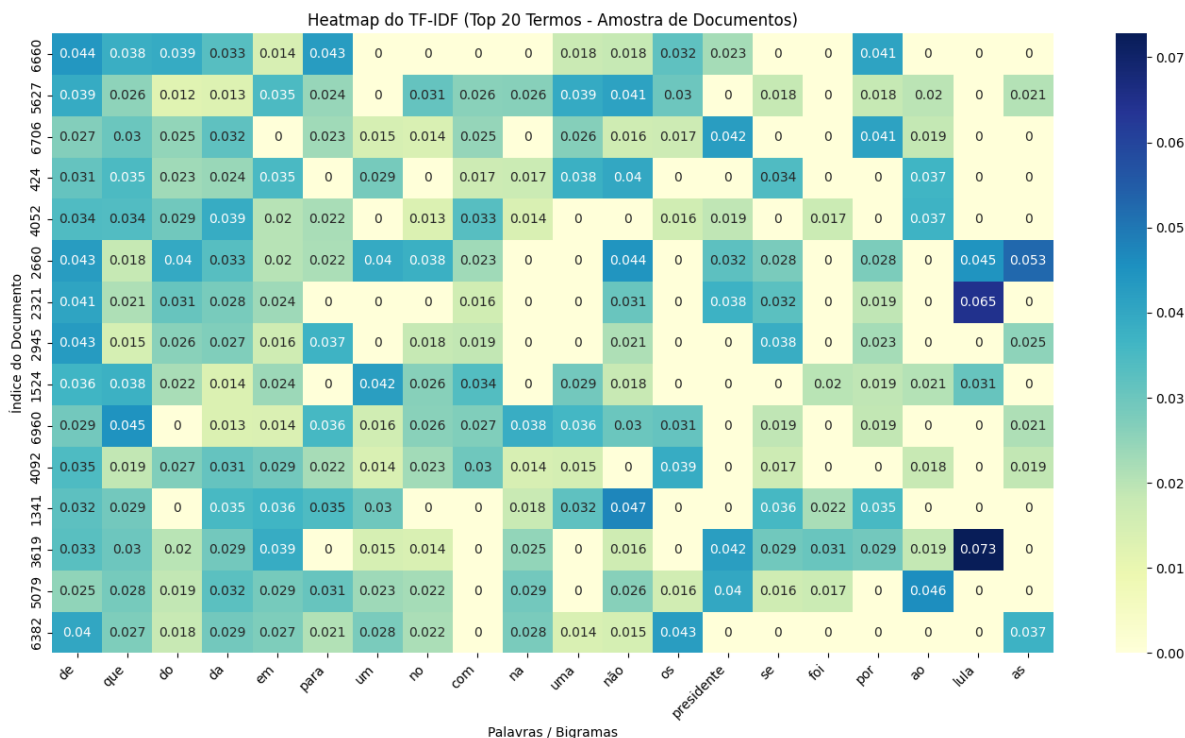
5.3 Representação: TF-IDF e dimensionalidade

Calcula a frequência do termo multiplicada pelo inverso da frequência do documento. Com unigramas + bigramas de palavras, os termos são separados assim:

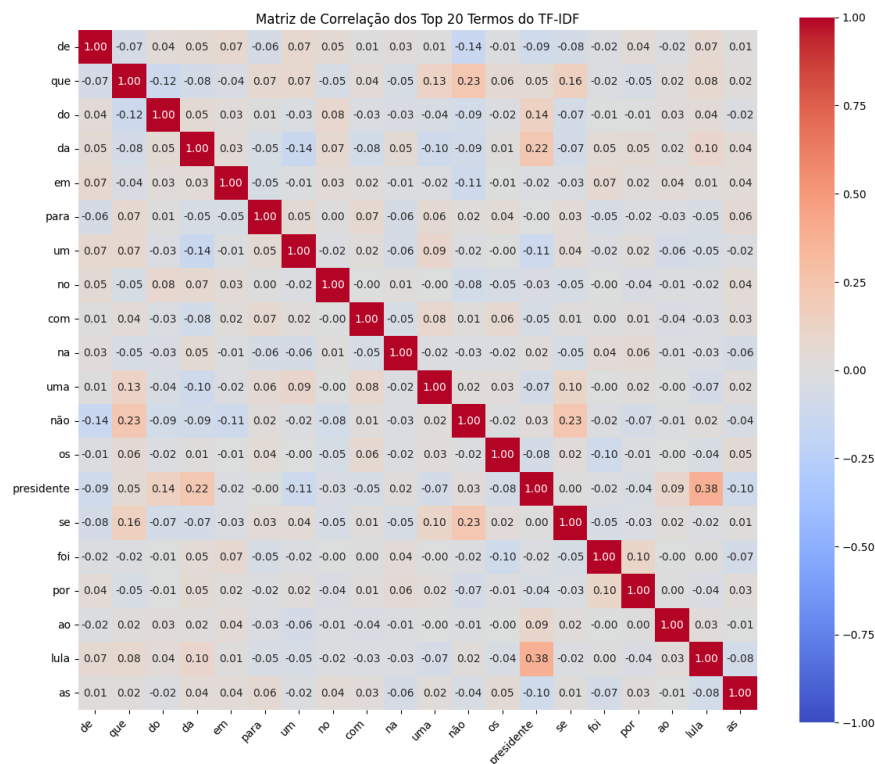
"não boa" -> "nao" | "boa" | "nao boa"

Para visualizar o TF-IDF em um plano, utilizamos o TruncatedSVD, que mostra o TF-IDF consegue diferenciar bem as duas classes para o nosso modelo supervisionado de ML, além disso, observamos que no centro, se aglomeram tanto notícias falsas, quanto verdadeiras, por compartilharem uma série de palavras, principalmente as stop-words, porém, por ser uma representação com dimensões reduzidas, as features ficam mais separadas do que no gráfico, mas já ajuda a entender como estão dispostas, em uma primeira aproximação.





Heatmap do TF-IDF dos 20 termos mais frequentes em uma amostra de documentos. Fonte: notebook 02_LR_training.



Matriz de correlação (Pearson) entre os 20 termos mais frequentes do TF-IDF. Fonte: notebook 02_LR_training.

A configuração padrão (txt_word + txt_char) produz 114.301 atributos para 5.760 documentos de treino,

com cerca de 99,5% de esparsidade. O vetorizador de palavras não remove acentos (o exemplo acima ilustra apenas a decomposição em n-gramas). A projeção 2D do TF-IDF com **TruncatedSVD** serve apenas para visualização: as duas primeiras componentes explicam ~0,75% da variância e misturam as classes (seção 5.8).

5.4 Modelos treinados

Observação. As tabelas de validação cruzada e os resultados de teste desta seção vêm do **protocolo anterior** (truncamento em 200 palavras, sem normalização, 5 folds sem embaralhamento). Servem para comparar configurações entre si, mas **não são comparáveis** aos números das seções 5.5 a 5.9. Os notebooks 02, 03 e 04 precisam ser reexecutados no pipeline atual para atualizar estas tabelas.

[PENDENTE] Reexecutar `02_LR_training`, `03_SVM_training` e `04_RF_training` com o pipeline atual. No 04, o título da seção diz "Regressão Logística" e a célula final chama `decision_function`, que o RandomForest não possui; no 02/04 as células de CV estão comentadas, então os outputs não correspondem ao código.

SVM (Support Vector Machine)

Racional de Escolha do Algoritmo A seleção do SVM, sob a variante de otimização linear (**LinearSVC**), fundamenta-se nas propriedades matemáticas inerentes ao domínio de Processamento de Linguagem Natural (PLN):

- **Alta Dimensionalidade e Esparsidade:** A representação vetorial do texto via TF-IDF gera matrizes com elevado volume de atributos (features), caracterizadas por alta densidade de zeros. O SVM é reconhecido pela robustez e eficiência computacional na navegação em espaços amostrais de alta dimensão.
- **Maximização de Margem de Decisão:** O algoritmo delimita um hiperplano ótimo de separação, focado exclusivamente nos vetores de suporte, cujo as instâncias limítrofes entre as classes (Notícia Verdadeira vs. Notícia Falsa), mitigando o risco de *overfitting* inerente a distribuições complexas.

Arquitetura de Implementação A construção da solução adotou uma abordagem modular, encapsulada e escalável via scikit-learn.

Configuração e Hiperparâmetros do Estimador O núcleo de classificação foi parametrizado com as seguintes diretrizes:

- **class_weight='balanced':** Ajusta a função de perda inversamente proporcional às frequências das classes. Como o conjunto é balanceado (3.600/3.600), o parâmetro não tem efeito prático; o grid search confirmou resultados idênticos com e sem ele.
- **max_iter=2000:** Expansão do teto de iterações para assegurar a convergência do solucionador numérico sobre matrizes de alta complexidade.
- **random_state=42:** Fixação de semente estocástica para garantir a reprodutibilidade integral dos experimentos.

Pipeline de Extração de Atributos Para mitigar vazamentos de dados (*data leakage*), o estimador foi integrado a um pipeline alimentado por um **ColumnTransformer**, avaliado em quatro configurações incrementais de extração:

- **word:** Vetorização n-grama baseada estritamente em palavras.
- **word+char:** Combinação de n-gramas de palavras e n-gramas de caracteres.
- **word+meta:** Vetorização de palavras acoplada a metadados linguísticos e estilísticos (devidamente normalizados).
- **word+char+meta:** Integração unificada de todas as representações vetoriais.

Calibração de probabilidades O `LinearSVC` retorna apenas a distância ao hiperplano. Para entregar uma pontuação de 0 a 1 ao sistema, o pipeline de produção (`modelo_olimpico.py`, seção 5.11) envolve o estimador em `CalibratedClassifierCV` com método **sigmoide (Platt)**, `StratifiedKFold(5)` e `ensemble=False`. Os notebooks 02 a 12 usam o `decision_function` (ROC-AUC) e não a calibração.

Vantagens Se comporta bem em dados de alta dimensionalidade, muito utilizado para classificação de texto e imagem, com boa regularização por margem e consegue construir fronteiras lineares e não lineares.

Desvantagens Depende de um poder computacional mais robusto que os outros modelos mais simples, o ajuste de kernel só é necessário nas variantes não lineares (`SVC`); o `LinearSVC` adotado não usa kernel. Os kernels RBF e polinomial foram testados no notebook 09 (seção 5.8).

Avaliação Empírica de Desempenho A validação foi conduzida utilizando Validação Cruzada Estratificada ($k=5$), avaliando o poder de generalização do modelo diante de dados não observados.

Configuração de Features	F1-Macro	Acurácia	Precisão Macro	Recall Macro
word + char + meta	0.9554 ($\pm$ 0.0155)	0.9554 ($\pm$ 0.0154)	0.9568 ($\pm$ 0.0135)	0.9554 ($\pm$ 0.0154)
word + char	0.9516 ($\pm$ 0.0184)	0.9517 ($\pm$ 0.0183)	0.9538 ($\pm$ 0.0158)	0.9517 ($\pm$ 0.0183)
word + meta	0.9384 ($\pm$ 0.0153)	0.9385 ($\pm$ 0.0152)	0.9404 ($\pm$ 0.0133)	0.9385 ($\pm$ 0.0152)
word	0.9352 ($\pm$ 0.0142)	0.9353 ($\pm$ 0.0140)	0.9380 ($\pm$ 0.0114)	0.9353 ($\pm$ 0.0140)

Avaliação dos resultados com os dados de testes O desempenho no teste ficou dentro de um desvio-padrão da validação cruzada, o que indica que a escolha da configuração não gerou *seleção* excessiva sobre o treino. Isso não implica generalização para outros corpora (seção 5.9). No protocolo anterior, o `LinearSVC` teve o melhor desempenho entre os três modelos.

[PENDENTE] A contagem original ("64 erros em 1.440: 49 falsas→verdadeiras e 15 verdadeiras→falsas") não foi localizada em nenhum notebook do repositório; o único teste de SVM com output salvo (word+char+meta) tem F1-macro de 0,9465 (acurácia 0,9465, ROC-AUC 0,9906). Reexecutar e registrar a matriz de confusão.

Por que funciona no nosso caso?

O SVM com kernel linear obteve o melhor desempenho no nosso modelo de detecção de Fake News porque é algoritmicamente desenhado para lidar com a extrema alta dimensionalidade e esparsidade dos dados gerados pelo TF-IDF. Ao combinarmos *n-grams* de palavras, caracteres e metadados, criamos um espaço com milhares de variáveis onde as classes (Verdadeiro e Falso) tornam-se naturalmente linearmente separáveis. Enquanto modelos baseados em árvores sofrem de *overfitting* ao tentar memorizar ruídos e regras muito específicas nesse cenário, o SVM foca estritamente em traçar um hiperplano que maximize a margem de separação entre as classes, ignorando os exemplos óbvios e ajustando-se apenas aos textos mais difíceis de classificar (vetores de suporte). Isso torna o modelo altamente robusto, permitindo que ele some centenas de sutis evidências linguísticas de forma estável para determinar a veracidade de uma notícia dentro do corpus.

Notebook com o código do treinamento e cross validation da SVM

<https://colab.research.google.com/drive/1g3B2vDnUg7f7ENvulFNFP0ygQ1jzoG5F?usp=sharing>

Random Forest

Racional de Utilização do Algoritmo A inclusão do *Random Forest* (`RandomForestClassifier`) no escopo experimental do projeto fundamenta-se na avaliação de estimadores baseados em *Ensemble Learning* (Aprendizado em Conjunto) e no particionamento não-linear do espaço de atributos:

- **Arquitetura de Ensemble por Bagging (*Bootstrap Aggregating*):** O algoritmo combina múltiplos estimadores fracos (árvores de decisão individuais) treinados sobre subamostras do conjunto de dados. Essa votação reduz drasticamente a variância do modelo, mitigando o risco de *overfitting* comum em árvores de decisão isoladas diante de ruídos textuais.
- **Mapeamento de Regras Não-Lineares e Interações Complexas:** Diferente de modelos lineares, o *Random Forest* divide o espaço de busca por meio de cortes ortogonais nos nós das árvores. Essa propriedade permite identificar dependências condicionais complexas entre variáveis (ex.: a combinação simultânea de alta taxa_maiusculas com baixa diversidade) sem a necessidade de engenharia manual de termos de interação.
- **Invariância à Escala de Atributos:** O processo de divisão dos nós depende estritamente da ordem relativa dos valores e não de sua magnitude absoluta. Essa característica confere robustez ao modelo ao processar vetores heterogêneos que agrupam TF-IDF (valores entre 0 e 1) e metadados numéricos não-normalizados.

Arquitetura de Implementação A construção do modelo seguiu o mesmo padrão modular e auditável, garantindo isonomia na comparação de desempenho através do scikit-learn.

Configuração e Hiperparâmetros do Estimador O modelo núcleo foi instanciado com a seguinte parametrização:

- `n_estimators=100`: Definição de um *ensemble* composto por 100 árvores de decisão independentes, assegurando estabilidade estatística e convergência na distribuição das votações.
- `random_state=42`: Fixação da semente estocástica para garantir a reprodutibilidade integral dos segmentos e ramificações geradas.
- `max_features='sqrt'`: Restrição do número de atributos avaliados em cada nó à raiz quadrada do total de *features*, promovendo a desconexão entre as árvores da floresta.

Pipeline de Extração de Atributos Para prevenir o vazamento de dados, o *Random Forest* atuou como o estimador final de um Pipeline acoplado ao `ColumnTransformer`, avaliado sob quatro arranjos incrementais de vetorização:

- `word`
- `word+char`
- `word+meta`
- `word+char+meta`

Suporte nativo à explicabilidade (SHAP / TreeExplainer) A estrutura baseada em árvores do *Random Forest* permite a integração com `shap.TreeExplainer` e o cálculo exato dos valores SHAP de interação.

[PENDENTE] Os notebooks do repositório não contêm esse cálculo para o Random Forest (o `TreeExplainer` aparece apenas em um notebook histórico não supervisionado, e o SHAP supervisionado usa `LinearExplainer` sobre a Regressão Logística). Os gráficos da seção 5.7 estão versionados em `02_LR_training` (células 20 e 25).

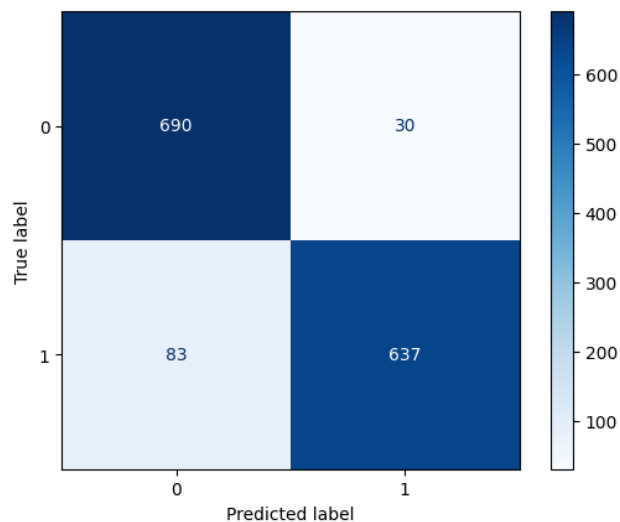
Vantagens Possui um bom desempenho com variedade dos dados, bom comportamento com dados não lineares, baixo risco de overfitting.

Desvantagens Entre os modelos selecionados não é o que menos precisa de recurso computacional, a importância das variáveis não significa causalidade obrigatoriamente.

Avaliação Empírica de Desempenho A validação do modelo foi conduzida via Validação Cruzada Estratificada (k=5), medindo o poder de generalização diante de dados não observados.

Configuração de Features	F1-Macro	Acurácia	Precisão Macro	Recall Macro
word + char + meta	0.9320 ($\pm$ 0.0140)	0.9321 ($\pm$ 0.0138)	0.9345 ($\pm$ 0.0122)	0.9320 ($\pm$ 0.0138)
word + char	0.9285 ($\pm$ 0.0162)	0.9286 ($\pm$ 0.0160)	0.9302 ($\pm$ 0.0145)	0.9286 ($\pm$ 0.0160)
word + meta	0.9150 ($\pm$ 0.0155)	0.9151 ($\pm$ 0.0153)	0.9180 ($\pm$ 0.0130)	0.9151 ($\pm$ 0.0153)
word	0.9112 ($\pm$ 0.0135)	0.9113 ($\pm$ 0.0132)	0.9142 ($\pm$ 0.0110)	0.9113 ($\pm$ 0.0132)

Avaliação dos resultados com os dados de testes O resultado no teste foi cerca de 0,7 ponto percentual abaixo da validação cruzada, dentro de um desvio-padrão, o que é consistente. O modelo errou 113 textos: 83 falsas classificadas como verdadeiras e 30 verdadeiras classificadas como falsas. Foi o modelo com mais erros dos três, e o que mais deixou notícias falsas passarem (recall da classe falsa de 0,8847).



Matriz de confusão do Random Forest no teste (1.440 textos; protocolo anterior). Fonte: notebook 04_RF_training.

Notebook com o código do treinamento e cross validation da random forest <https://colab.research.google.com/drive/12TiiqYNQwJJ5eN6Se8psifMT08N0NADp?usp=sharing>

Regressão Logística

Racional de Utilização do Algoritmo A adoção da *Regressão Logística* (LogisticRegression) no pipeline experimental teve como propósito estabelecer a **linha de base (baseline)** do projeto. Em Processamento de Linguagem Natural (PLN), estimadores lineares probabilísticos fornecem o patamar mínimo de desempenho a ser superado por arquiteturas mais complexas:

- **Interpretabilidade Direta e Probabilidades Nativas:** O modelo aplica a função sigmoide $\sigma(\mathbf{z}) = 1 / (1 + e^{-z})$ sobre a combinação linear dos atributos, mapeando a saída diretamente para o intervalo entre 0 e 1 (0% a 100%). Ao contrário do LinearSVC, a Regressão Logística entrega pontuações de probabilidade nativas sem a necessidade de métodos de calibração secundários. Além disso, a inspeção direta dos coeficientes (pesos *beta*) permite auditar com clareza a influência de cada n-grama ou métrica na classificação do texto como desinformação.
- **Eficiência Computacional e Navegação em Espaços Esparsos:** A otimização convexa por algoritmos de gradiente opera com extrema rapidez em matrizes esparsas de alta dimensão geradas pelo TF-IDF, oferecendo um excelente balanço entre custo computacional e acurácia preditiva.

Arquitetura de Implementação A validação da Regressão Logística seguiu rigorosamente a mesma esteira de pré-processamento e partição utilizada nos demais estimadores, assegurando isonomia nos testes do scikit-learn.

Configuração e Hiperparâmetros do Estimador O modelo núcleo foi parametrizado com as seguintes diretrizes:

- `class_weight='balanced'`: penaliza os erros de forma inversamente proporcional à frequência das classes; sem efeito prático aqui, pois o conjunto é balanceado.
- `max_iter=1000`: Expansão do teto de iterações do solver de otimização para garantir a convergência dos coeficientes numéricos no espaço multidimensional de features.
- O `random_state` não é fixado no solver `lbfgs` (determinístico), diferentemente do LinearSVC e do Random Forest.

Pipeline de Extração de Atributos Para mitigar o risco de vazamento de dados, o estimador atuou como o nó final de um Pipeline acoplado ao ColumnTransformer, avaliado sob quatro arranjos incrementais:

- `word`
- `word+char`
- `word+meta`
- `word+char+meta`

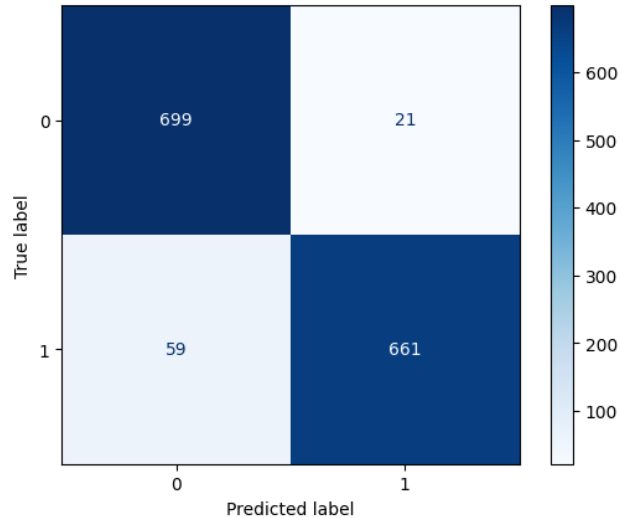
Vantagens Precisa de pouco desempenho computacional, muito bom com dados lineares, fácil compreensão.

Desvantagens Limitado para fronteiras não lineares. Quanto ao ajuste, o gap treino-validação com $C=1$ é de 0,067 (LinearSVC: 0,070), ou seja, tão sujeito a superajuste no espaço TF-IDF quanto o SVM.

Avaliação Empírica de Desempenho

Feature Set	f1_macro	accuracy	precision_macro	recall_macro
word	0.9243 ± 0.0048	0.9243 ± 0.0048	0.9250 ± 0.0044	0.9243 ± 0.0048
word+char	0.9474 ± 0.0031	0.9474 ± 0.0031	0.9483 ± 0.0029	0.9474 ± 0.0031
word+meta	0.9006 ± 0.0082	0.9007 ± 0.0081	0.9020 ± 0.0079	0.9007 ± 0.0081
word+char+meta	0.9326 ± 0.0016	0.9326 ± 0.0016	0.9336 ± 0.0016	0.9326 ± 0.0016

Avaliação dos resultados com os dados de teste O F1-Macro no teste ficou 0,3 ponto percentual abaixo da validação cruzada, dentro de um desvio-padrão. O modelo errou 80 textos: 59 falsas classificadas como verdadeiras e 21 verdadeiras classificadas como falsas.



Matriz de confusão da Regressão Logística no teste (1.440 textos; protocolo anterior). Fonte: notebook 02_LR_training.

Notebook com o código do treinamento e cross validation da Regressão Logística <https://colab.research.google.com/drive/1sAjVrmOaKkYbtNSGiiHIP6WV745ySN1P?usp=sharing>

Modelo baseline

Adotamos como baseline a Regressão Logística com TF-IDF de palavras (unigramas e bigramas) aplicado ao texto truncado, com `class_weight='balanced'`, `max_iter=1000` e `random_state=42`. Esse modelo estabelece o patamar mínimo de desempenho a ser superado: F1-Macro de 0.9243 em validação cruzada ($k=5$, protocolo anterior; [PENDENTE] recalculado no pipeline atual).

Um modelo mais complexo só se justifica se superar esse patamar de forma consistente, considerando o custo computacional e a perda de interpretabilidade.

Escolha do modelo como baseline **Simplicidade e Interpretabilidade:** os coeficientes podem ser analisados diretamente, o que permite explicar quais n-gramas contribuem para a classificação de uma notícia como desinformação.

Probabilidades Nativas: a saída é uma probabilidade entre 0 e 1 sem etapa de calibração, ao contrário do LinearSVC.

Eficiência em Espaços Esparsos: treina rapidamente sobre as matrizes de alta dimensão geradas pelo TF-IDF.

Dois Níveis de Referência

Piso (word): 0.9243, a Regressão Logística com o menor conjunto de features.

Melhor configuração do baseline (word+char): 0.9474, o teto que a Regressão Logística atinge nos experimentos.

Os n-gramas de caractere elevaram o F1 em +0.0231 sobre o piso. Já os metadados estilométricos reduziram o desempenho na Regressão Logística: -0.0237 sobre word e -0.0148 sobre word+char.

5.5 Resultados no protocolo atual: regularização (C) e poda de vocabulário

Todos os experimentos desta seção e das seguintes usam os mesmos dados (X_{tr}), os mesmos folds (`StratifiedKFold(5, shuffle=True, random_state=42)`) e a configuração word+char. O teste (X_{te})

não é usado. O **gap** é F1 de treino – F1 de validação.

Regularização (06_regularizacao_C)

modelo	C	f1_treino	f1_cv	f1_cv_std	gap
LinearSVC	0.01	0.9211	0.8802	0.0123	0.0409
LinearSVC	0.03	0.9564	0.9024	0.0150	0.0540
LinearSVC	0.1	0.9866	0.9203	0.0142	0.0663
LinearSVC	0.3	0.9997	0.9264	0.0111	0.0733
LinearSVC	1	1.0000	0.9295	0.0099	0.0705
LinearSVC	3	1.0000	0.9283	0.0074	0.0717
LinearSVC	10	1.0000	0.9283	0.0066	0.0717
Regressão Logística	0.01	0.8703	0.8417	0.0101	0.0285
Regressão Logística	0.03	0.8942	0.8577	0.0122	0.0365
Regressão Logística	0.1	0.9261	0.8837	0.0129	0.0425
Regressão Logística	0.3	0.9576	0.9019	0.0146	0.0556
Regressão Logística	1	0.9851	0.9182	0.0136	0.0669
Regressão Logística	3	0.9987	0.9241	0.0125	0.0746
Regressão Logística	10	1.0000	0.9281	0.0109	0.0719

- **LinearSVC**: com $C \leq 0,03$ o modelo entra em *underfitting* (F1 0,8802 em $C=0,01$). O máximo está em **$C=1$ (0,9295 $\pm$ 0,0099)**; $C=3$ e $C=10$ saturam o treino em 1,0 sem ganho na validação. O C padrão é o ponto ótimo.
- **Regressão Logística**: o F1 cresce de forma monotônica até **$C=10$ (0,9281 $\pm$ 0,0109)**, praticamente empatando com o LinearSVC. Em espaços esparsos de alta dimensão a LR se beneficia de regularização mais fraca que o padrão.
- O gap mínimo útil gira em torno de 0,07 e não se fecha com regularização sem perda de F1.

Poda de vocabulário (05_feature_dim_analysis, min_df)

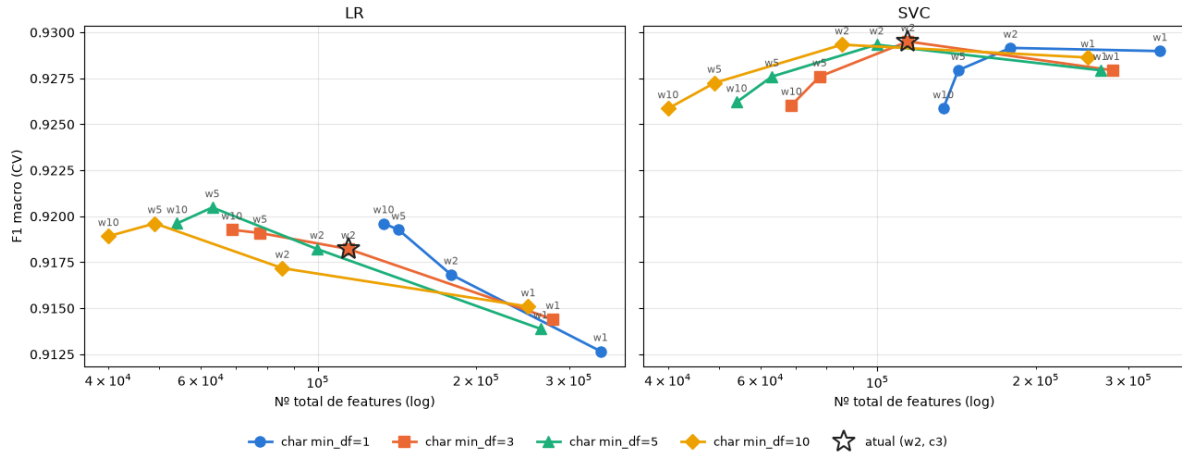
F1 de validação cruzada do **LinearSVC** para cada par (**min_df** de palavras $\times$ **min_df** de caracteres); a configuração atual é word=2, char=3.

word	char	char=1	char=3	char=5	char=10
1		0.9290	0.9279	0.9279	0.9286
2		0.9292	0.9295	0.9293	0.9293
5		0.9279	0.9276	0.9276	0.9272
10		0.9259	0.9260	0.9262	0.9259

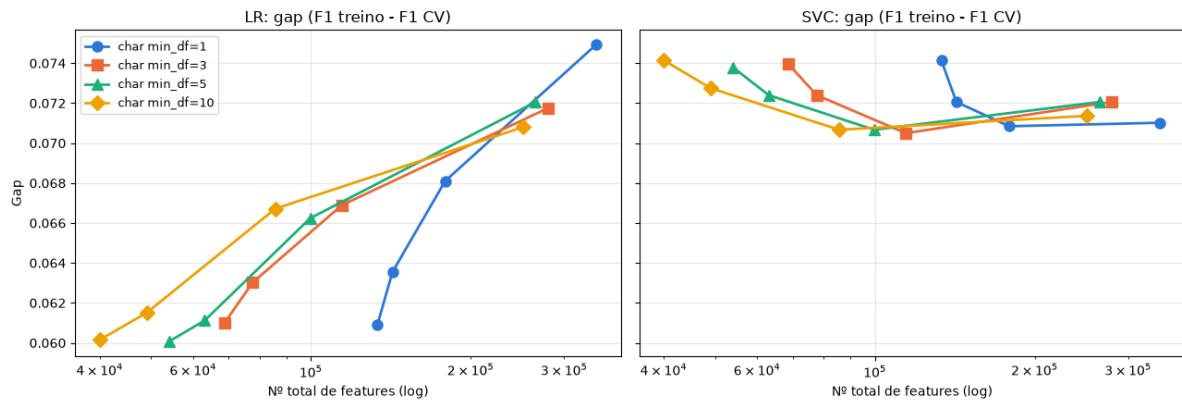
Número médio de atributos por combinação:

word	char	char=1	char=3	char=5	char=10
1		344.416	279.670	265.043	251.039
2		179.046	114.301	99.673	85.670
5		142.415	77.669	63.042	49.039
10		133.414	68.668	54.041	40.037

Com o LinearSVC, reduzir de 114 mil para ~40 mil atributos (**min_df** 10/10) custa apenas ~0,003 de F1 (0,9259 contra 0,9295), dentro do desvio entre folds (~0,010). Na Regressão Logística a poda melhora levemente o F1 (0,9182 $\rightarrow$ 0,9205 em 5/5) e reduz o gap, também dentro do ruído.



F1 macro (CV) em função do número total de atributos, por combinação de min_df. A estrela marca a configuração atual. Fonte: notebook 05_feature_dim_analysis.



Gap (F1 treino - F1 CV) em função do número total de atributos. Fonte: notebook 05_feature_dim_analysis.

5.6 Metadados: do corpus e morfossintáticos (spaCy)

Metadados do corpus

Os atributos do corpus (num_verbos, pausalidade, diversidade etc.) são calculados pelo NILC sobre o **texto completo**, enquanto o TF-IDF vê apenas as 100 primeiras palavras; as taxas são normalizadas pelo num_palavras do texto completo. Esses atributos **não existem** para notícias novas inseridas pelo usuário nem para o FakeRecogna.

Resultados registrados:

- Na **Regressão Logística** (curva de aprendizado, protocolo anterior): word+meta fica 2,4 pontos abaixo de word e word+char+meta fica 1,5 ponto abaixo de word+char. No LinearSVC e no Random Forest a diferença é de no máximo ~0,4 ponto, dentro do desvio. A hipótese em investigação é a diferença de escala entre o bloco denso padronizado e o TF-IDF sob regularização L2.
- No **grid search** da branch feat/taxa_stop_words (ainda não integrada; 3 folds, $C \in \{0,1; 1; 10\}$, bloco de metadados acrescido de taxa_stop_words), o LinearSVC com word+char+meta e $C=10$ obteve **F1 0,9415 $\pm$ 0,0026**, contra 0,9276 $\pm$ 0,0024 do word+char ($C=1$) e 0,9094 do word. Esses valores estão em protocolo de 3 folds e não são comparáveis aos de 5 folds.

Observação. As duas evidências não são comparáveis entre si: uma é a LR no protocolo anterior e a outra é o SVM em 3 folds com um bloco de metadados diferente. O bloco avaliado

no protocolo atual, em 5 folds e com teste externo, é o de metadados spaCy (abaixo).

Extração morfossintática com spaCy (branch `fix-metadados`)

A branch `fix-metadados` (Ana Laura Alves Cordeiro) conta, com o `pt_core_news_sm`, as classes gramaticais (POS) e as relações de dependência sintática (DEP) de cada classe do Fake.br, sobre o texto completo, e grava a proporção sobre o total de tokens (799.162 tokens falsos e 4.582.307 verdadeiros). Principais diferenças:

Atributo	Falsas	Verdadeiras	Leitura
POS SPACE	2,88%	0,29%	artefato de formatação da fonte
DEP dep	3,21%	0,64%	mesmo artefato (tokens de espaço sem relação sintática)
DEP ROOT	5,94%	4,93%	frases mais curtas nas falsas
POS PROPN	10,94%	9,75%	mais nomes próprios nas falsas
POS VERB	11,10%	10,57%	mais verbos nas falsas
POS NOUN	18,21%	19,71%	mais estrutura nominal nas verdadeiras
POS ADP / DEP case	13,68% / 13,49%	15,66% / 15,42%	mais preposições nas verdadeiras
DEP nmod	7,69%	8,95%	idem
ADJ, ADV, DET	≈ iguais	≈ iguais	não discriminam

A maior diferença (SPACE/dep, $\sim 10\times$ e $\sim 5\times$) é um artefato de fonte e não um sinal linguístico, o que dá uma segunda evidência independente a favor da normalização da seção 5.2. Os sinais linguísticos reais são modestos.

Limitações dessa extração: é **agregada por classe** (não por notícia, portanto não serve como feature nem permite teste estatístico), roda sobre o texto bruto e completo (o modelo vê texto normalizado e truncado) e não foi usada em nenhum modelo. O script de gráficos (`eda.py`) foi desativado (`eda-deprecated.py`) por usar nomes de colunas que não existem nos CSVs gerados. A branch também versiona o corpus inteiro, o que deve ser removido antes do merge.

Metadados morfossintáticos por notícia (spaCy; notebooks 10 a 12)

O módulo `machine-learning/supervised-learning/metadados_spacy.py` transforma a ideia acima em atributos utilizáveis pelo modelo. Decisões de projeto:

- **Entrada:** o mesmo `texto_trunc` que o TF-IDF vê (normalizado, 100 palavras), para que bloco textual e bloco de metadados descrevam o mesmo texto.
- **Taxas, não contagens:** cada atributo é dividido pelo número de tokens (ou de verbos), para não depender do tamanho do texto.
- **Atributos candidatos (28):** proporção de 13 classes POS e de 10 relações DEP (`nsubj`, `obj`, `obl`, `nmod`, `case`, `advmod`, `amod`, `ccomp`, `xcomp`, `mark`); taxa de sentenças por token e tamanho médio de sentença; proporção de verbos no imperativo e no subjuntivo; proporção de tokens de 1ª ou 2ª pessoa.
- **Atributos finais (24):** removidos `verbo_imperativo` (**constante:** o `pt_core_news_sm` não marcou nenhum imperativo) e três redundâncias por $|r| > 0,9$, escolhidas por uma regra que não usa o rótulo (`dep_case~pos_ADP` 0,98; `pos_ADV~dep_advmod` 0,95; `pos_SCONJ~dep_mark` 0,95). Os indicadores "imperativo" e "pessoalidade" da seção 1.4 ficam, portanto, só com a personalidade.

- **Diagnóstico de fonte:** SPACE, X, SYM, INTJ e DEP dep são calculados em colunas `diag_*` e não entram no modelo.
- **Sem vazamento:** a extração é por documento e não tem fit; é pré-calculada e cacheada (`*.parquet`, ignorado pelo git). Imputação (mediana) e padronização ficam dentro do pipeline, por fold. Textos sem verbos (comuns em títulos) geram NaN tratado pelo imputador.
- **Peso do bloco:** o bloco denso é multiplicado por um peso w (`transformer_weights`) para não dominar a margem do TF-IDF.

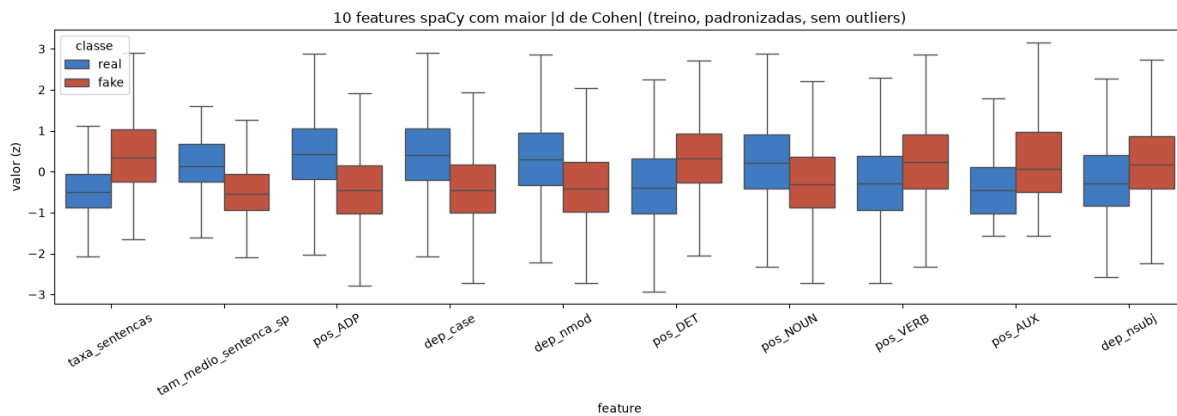
Protocolo executado. LinearSVC (`class_weight="balanced"`, `dual=False`; o notebook mostra que o B0 reproduz o notebook 06 fold a fold, $0,9295 \pm 0,0099$) e os mesmos folds dos notebooks 05–09. Comparação **pareada por fold** contra o B0; diferença menor que 0,01 sem 5/5 folds melhores é tratada como empate. O teste interno foi aberto uma vez. Configurações: **B0** (word+char, referência), **M2** (word+char + spaCy) e **S1** (só spaCy). Busca: $C \in \{0,1; 0,3; 1; 3; 10\} \times w \in \{0,03; 0,1; 0,3; 1; 3\}$. O motor por blocos dá scores idênticos ao Pipeline real do scikit-learn (diferença máxima 0,00000).

Artefato de fonte: eliminado pela normalização

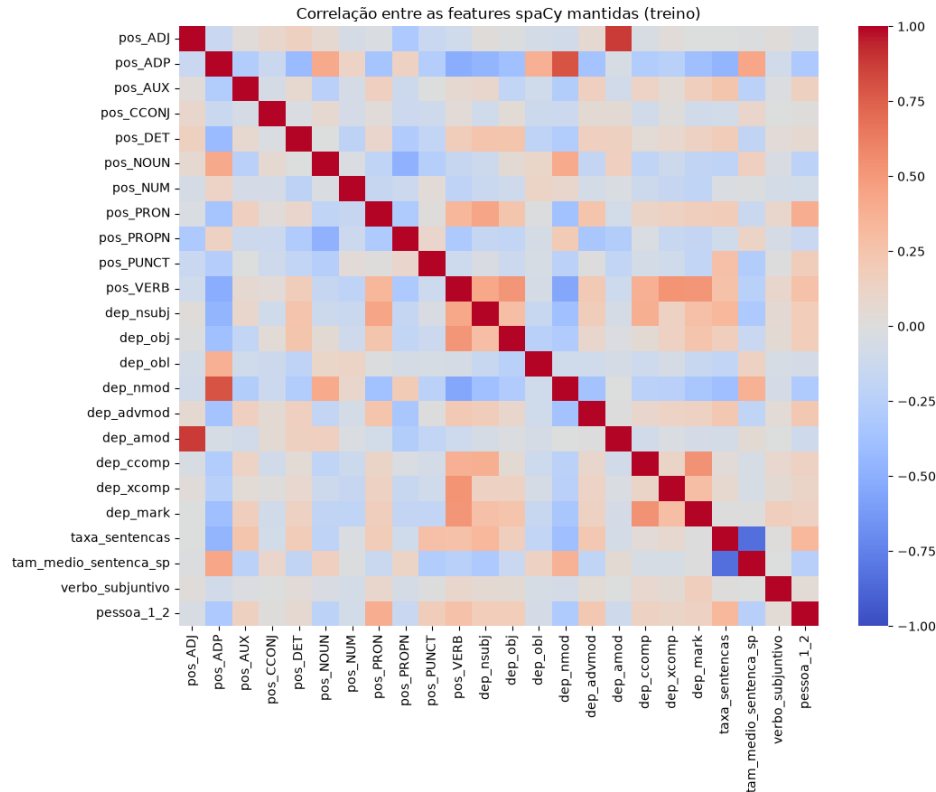
Diagnóstico	Bruto: falsas	Bruto: verdadeiras	Truncado: falsas	Truncado: verdadeiras
POS SPACE	2,93%	0,38%	0,00%	0,00%
DEP dep	3,21%	0,73%	0,40%	0,41%

A diferença que a `fix-metadados` encontrou é formatação de cada site (quebras de linha), e a `normalizar()` do `01_Data_Prep` já a remove. Isso responde à pergunta deixada em aberto: não resta vazamento residual dessas colunas. (Os valores da tabela vêm do treino; a `fix-metadados` usa o corpus inteiro, daí 2,88% e 0,29%.)

Sinal linguístico real (d de Cohen no treino): as falsas têm sentenças mais curtas (13,0 contra 18,7 palavras; $d = 1,1$) e mais DET, VERB, AUX, `nsubj` e 1ª/2ª pessoa; as verdadeiras têm mais ADP/case ($d \approx -0,93$), `nmod` e NOUN, isto é, frases nominais típicas do jornalismo.



As 10 features spaCy com maior $|d$ de Cohen entre as classes (treino, padronizadas, sem outliers). Fonte: notebook `10_svm_metadados_spacy`.

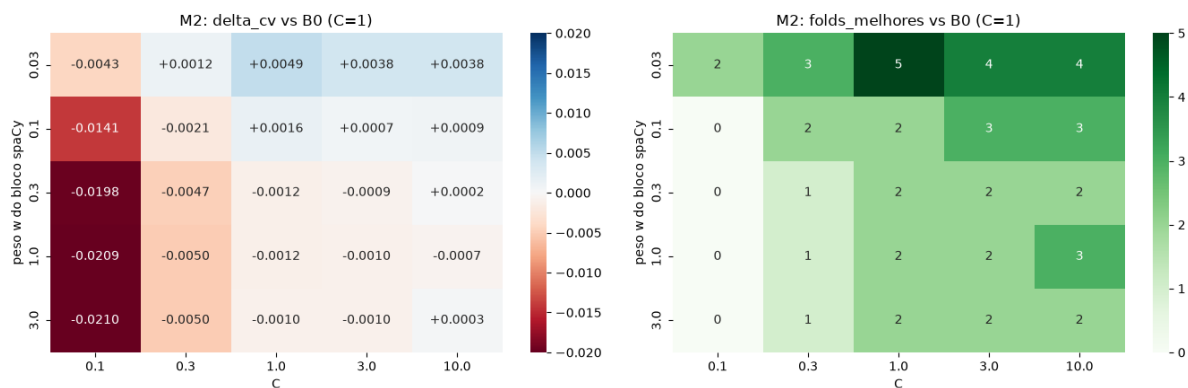


Correlação entre as features spaCy mantidas (treino). Fonte: notebook 10_svm_metadados_spacy.

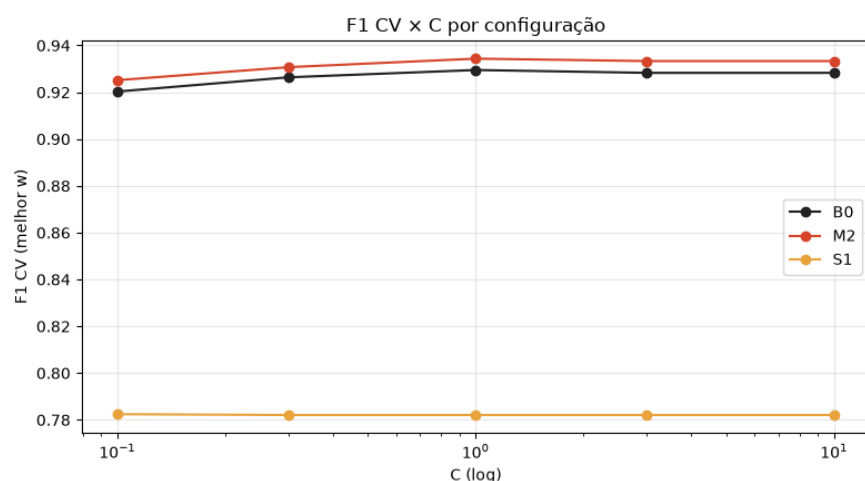
Validação cruzada interna: ganho pequeno, só com peso baixo

config	C	w	texto	F1 treino	F1 CV $\pm$ std	gap	Δ vs B0	folds melhores
B0	1	n/a	n/a	1.0000	0.9295 $\pm$ 0.0099	0.0705	n/a	n/a
M2	1	0,03	todo o texto	1.0000	0.9344 $\pm$ 0.0103	0.0656	+0.0049	5/5
M2 + χ^2 k=50 mil	1	0,03	50 mil	0.9999	0.9397 $\pm$ 0.0066	0.0601	+0.0102	5/5
M2 + χ^2 k=20 mil	1	0,03	20 mil	0.9952	0.9358 $\pm$ 0.0061	0.0595	+0.0062	3/5
S1 (só spaCy)	0.1	n/a	24 atributos	0.7852	0.7824 $\pm$ 0.0082	0.0028	-0.1471	0/5

- **O peso importa.** Com $w \geq 0,3$ o Δ é ≤ 0 em quase todo C; com C=0,1, qualquer w piora (até -0,021). O bloco denso, se não atenuado, toma a margem do TF-IDF. É consistente com a hipótese de escala, mas ela não foi testada na Regressão Logística, onde a piora tinha aparecido.
- **O ganho é frágil.** Só a célula C=1, w=0,03 chega a 5/5 folds; as vizinhas ficam em 2-4/5. O ótimo está na borda do grid de w e é o máximo entre 25 candidatos (viés de seleção).
- χ^2 + **spaCy**. Com k=50 mil o M2 chega a 0,9397, mas o mesmo χ^2 no B0 empatava (0,9288, notebook 08) e o número vem de duas rodadas de seleção, sem ter sido avaliado no teste.
- **S1**. Há sinal (F1 0,78), mas modesto, com gap ≈ 0 por *underfitting*.



M2 vs B0 na CV: Δ F1 (esquerda) e número de folds melhores (direita) por C e peso w do bloco spaCy. Fonte: notebook 10_svm_metadados_spacy.



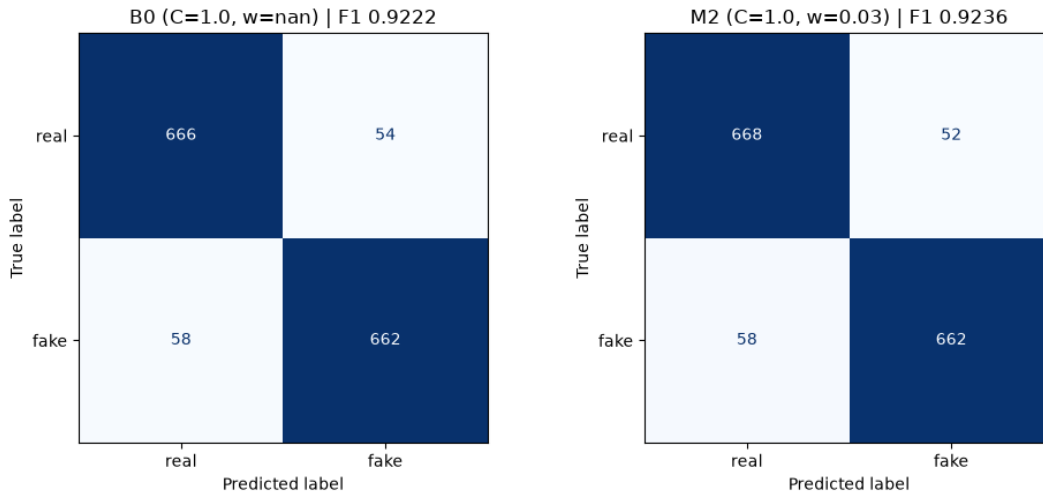
F1 CV $\times$ C por configuração (B0, M2, S1). Fonte: notebook 10_svm_metadados_spacy.

Teste interno e teste externo

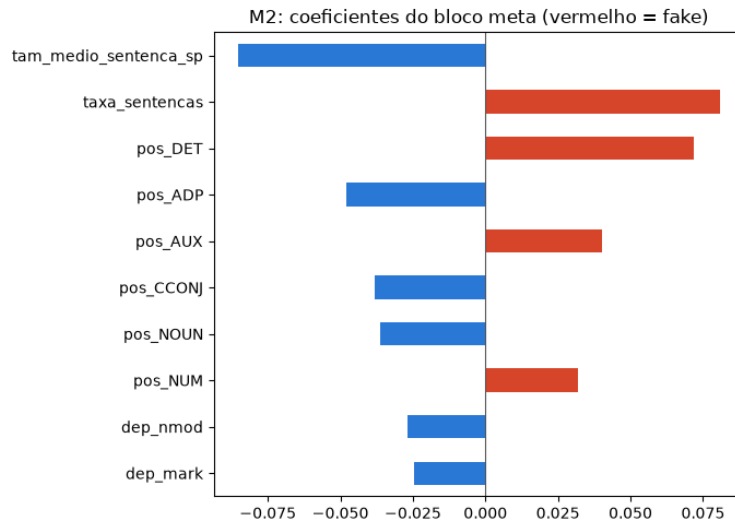
conjunto	modelo	F1 macro	ROC-AUC	% previsto fake	falsos positivos	falsos negativos
interno (Fake.br)	B0	0.9222	0.9830	49.7%	54	58
interno (Fake.br)	M2	0.9236	0.9846	49.6%	52	58
externo (FakeRecogna, títulos)	B0	0.6119	0.7072	70.8%	1438	407
externo (FakeRecogna, títulos)	M2	0.6134	0.6622	59.3%	1184	720
externo (FakeRecogna, títulos)	S1	0.5577	0.5845	57.9%	1287	897

- **Interno:** a direção é a da CV, mas +0,0014 de F1 está dentro do erro padrão ($\sim \pm 0,007$ com 1.440 notícias).

- **Externo:** o spaCy desloca o limiar (menos "fake") mas **ordena pior:** o AUC cai de 0,707 para 0,662. Os títulos têm mediana de 13 tokens contra 114 no treino, e `taxa_sentencas` e `tam_medio_sentenca`, as features de maior peso, saem da distribuição vista no treino. Isso é um risco direto para o app, que receberá textos de tamanhos e formatos variados.



Matrizes de confusão no teste interno: B0 (esquerda) e M2 (direita). Fonte: notebook `10_svm_metadados_spacy`.



Coeficientes do bloco de metadados spaCy no M2 (vermelho = fake). Fonte: notebook `10_svm_metadados_spacy`.

Observação. O desfecho difere do que o grid search sugeria (metadados ajudando, 0,9415 em 3 folds). Aqui, no bloco spaCy, o ganho é de +0,005 na CV, sem ganho comprovado no teste interno e com perda de generalização externa.

5.7 Interpretabilidade (SHAP)

Para a análise de importância das variáveis, a técnica SHAP (*SHapley Additive exPlanations*) se integra à arquitetura do projeto por sua capacidade de avaliar os parâmetros não de forma isolada, mas sim como eles atuam em conjunto. Baseado na Teoria dos Jogos, esse método permite ao sistema compreender a complementaridade matemática entre as variáveis, interpretando cenários complexos onde a presença de uma fonte reconhecida precisa estar necessariamente aliada a um contexto ou bibliografia consistente para atestar

a confiabilidade da notícia. Como algoritmos baseados em árvores de decisão conseguem aprender essas regras condicionais de forma orgânica, o SHAP interpreta as predições do modelo e gera uma matriz de interação que mede a força exata da união dessas variáveis. Isso contorna as limitações de outras técnicas explicativas diante de parâmetros altamente correlacionados, como o excesso de emoção e o uso de verbos no imperativo. Ou seja, o fluxo consiste em treinar o modelo de classificação com o dataset, processar a notícia pelo explicador do SHAP e extrair os atributos de maior impacto para aquele veredito específico. Esses ofensores matemáticos são então filtrados e enviados como contexto para o modelo socrático gerar a pergunta reflexiva correspondente, garantindo que o alerta exibido no sistema de semáforo possua uma justificativa rastreável e diretamente baseada na estrutura da notícia.

Beeswarm plot. O gráfico de *beeswarm plot* exibe o impacto e a direção de cada atributo na predição de notícias falsas, ordenando as variáveis por importância no eixo vertical. No eixo horizontal, os valores SHAP positivos indicam que o indicador empurra o veredito para a desinformação, enquanto os negativos apontam para a autenticidade, com a escala de cores representando o valor real de cada métrica (do azul para valores baixos ao rosa para valores altos). O resultado demonstra que a **diversidade** lidera o impacto preditivo, tal qual o boxplot representou, evidenciando de forma direta como intensidades específicas de cada variável linguística moldam a decisão do modelo e sustentam as intervenções analíticas do sistema.

Desenvolvimento

A implementação do cálculo de interações globais via SHAP expande a interpretação para além dos atributos isolados, mapeando como o modelo de aprendizado de máquina cruza duplas de variáveis para fundamentar suas predições. O procedimento inicia-se com a extração da matriz tridimensional de interações, que passa por um tratamento de dimensionalidade para isolar especificamente a classe de interesse voltada às notícias falsas. A partir disso, o sistema calcula a média dos valores absolutos dessa matriz em todo o conjunto de teste, condensando a complexidade estatística em uma métrica objetiva que quantifica a força de sinergia entre cada par de características textuais.

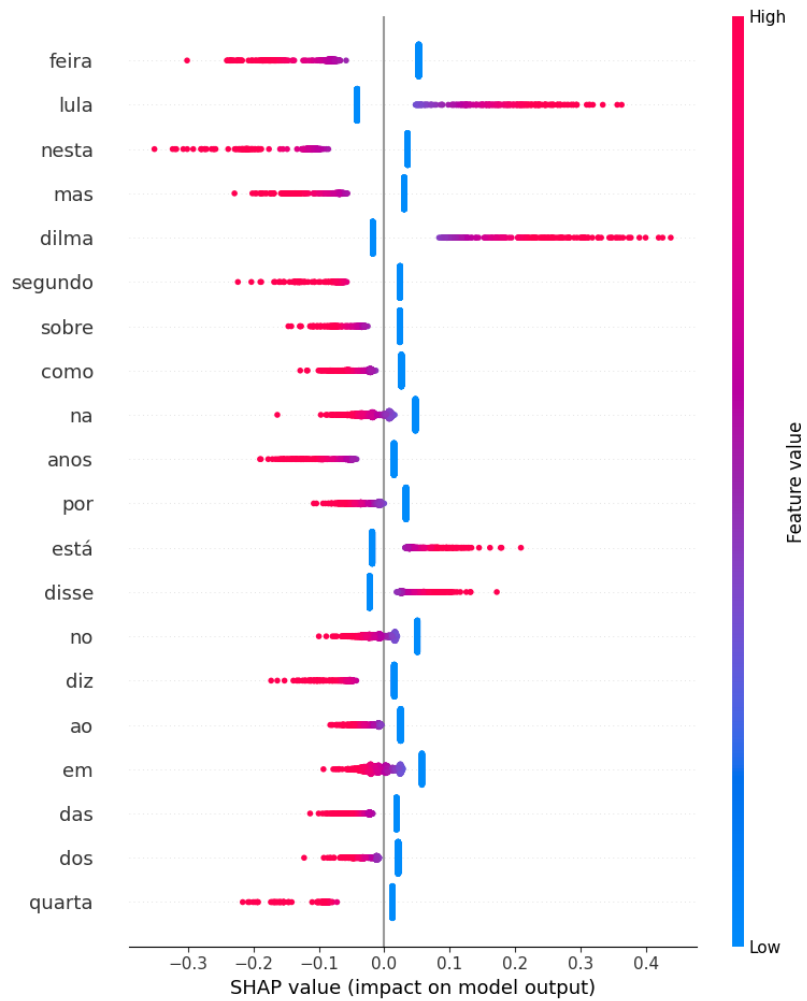
Para estruturar o resultado, o código varre o conjunto de atributos de maneira combinatória, ignorando comparações repetidas e a identidade própria das variáveis, a fim de consolidar os dados em um ranking ordenado de forma decrescente. Esse arranjo exibe as combinações mais potentes identificadas pelo algoritmo, destacando no topo duplas como a **taxa_verbos_subj_imp** combinada com a **diversidade**. Essa visão matricial comprova que o classificador opera por meio de regras lógicas conjuntas, fornecendo a base analítica essencial para que o sistema compreenda a dinâmica estrutural da desinformação e fundamente suas intervenções socráticas na interface.

Visualização

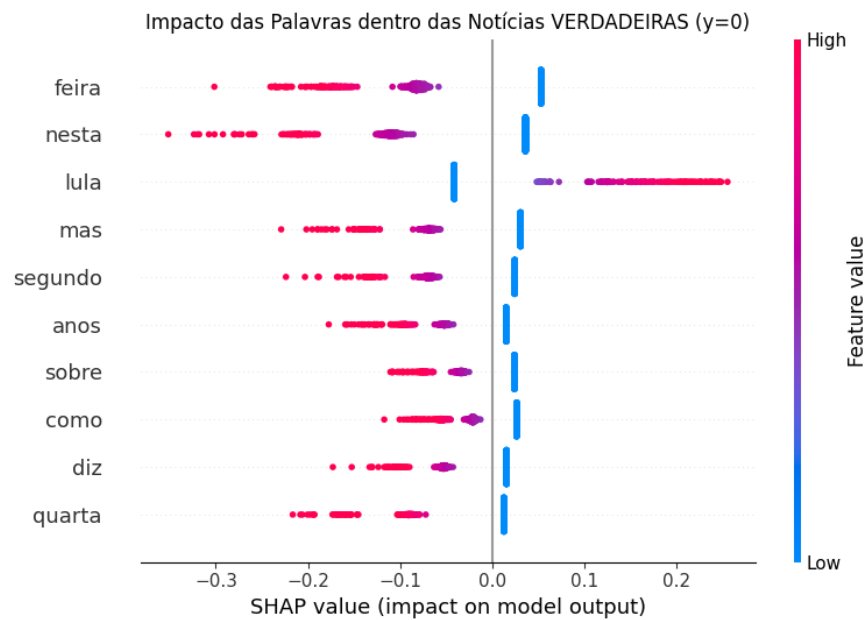
O gráfico de dispersão bivariado ilustra a interação entre as duas variáveis mais importantes **taxa_verbos_subj_imp** e **diversidade**, posicionando no eixo horizontal e eixo vertical, onde o impacto marginal corresponde o veredito de desinformação. A escala de cores lateral representa o nível de **diversidade** de cada instância, permitindo observar diretamente como essa segunda variável modula e altera o comportamento da primeira. Os pontos demonstram que o peso preditivo atribuído aos verbos modais e imperativos varia conforme o contexto da riqueza de vocabulário do texto, fornecendo a evidência visual da sinergia entre os fatores estruturais e fundamentando a lógica complexa que sustenta as intervenções analíticas da plataforma.

Observação. O gráfico de interação aponta **taxa_verbos_subj_imp** (verbos no **subjuntivo** ou **imperativo**; o texto original dizia "modais e imperativos") e **diversidade** como o par mais forte. Os notebooks de origem (SHAP de interação e *beeswarm*) não estão versionados e o modelo escolhido (LinearSVC sobre TF-IDF) não tem SHAP implementado.

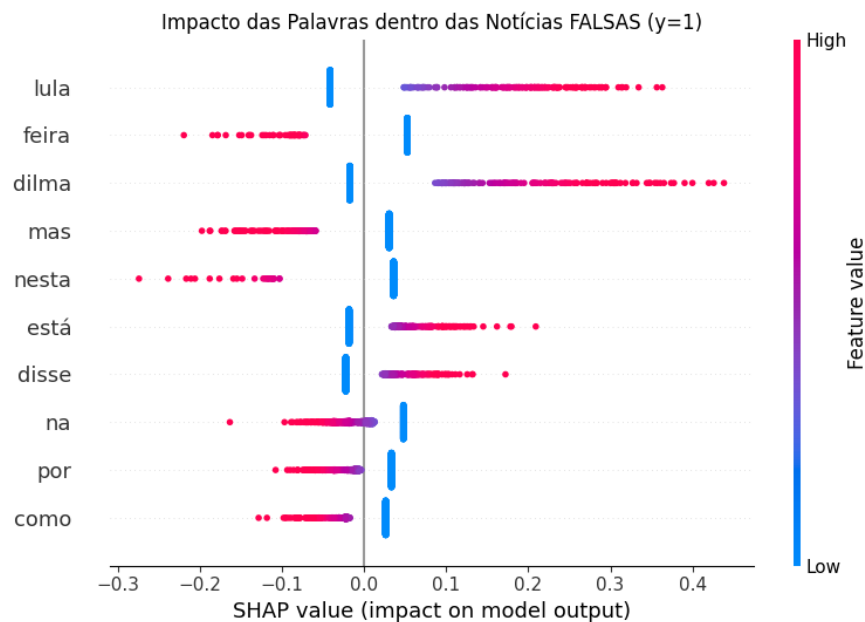
Complementarmente, o notebook **02_LR_training** traz gráficos *beeswarm* do SHAP (**LinearExplainer**) calculados sobre a Regressão Logística com TF-IDF, que mostram as palavras de maior impacto na decisão:



SHAP summary plot: 20 termos de maior impacto (Regressão Logística). Fonte: notebook 02_LR_training.



SHAP: impacto das palavras nas notícias verdadeiras ($y=0$). Fonte: notebook 02_LR_training.



SHAP: impacto das palavras nas notícias falsas ($y=1$). Fonte: notebook 02_LR_training.

Observação. O SHAP supervisionado está versionado em 02_LR_training (células 20 e 25, LinearExplainer sobre a Regressão Logística). No modelo de produção a explicação não depende de SHAP: o score se decompõe exatamente em contribuições por n-grama e por atributo spaCy (seção 5.11).

5.8 Dimensionalidade e kernels

Redução de dimensionalidade (08_reducao_dim, 05)

Pergunta: reduzir o número de atributos (~114 mil para ~5,7 mil documentos) melhora o F1 ou diminui o gap? Todas as reduções são ajustadas dentro do Pipeline, no treino de cada fold. O PCA do scikit-learn não aceita matriz esparsa (centralizá-la a tornaria densa, ~5,3 GB), então usa-se **TruncatedSVD (LSA)** com **Normalizer**. Técnicas comparadas: poda por `min_df`, TruncatedSVD (d de 2 a 300), seleção univariada χ^2 (`SelectKBest`, k de 500 a 80.000) e seleção por modelo com penalidade L1 (`SelectFromModel`).

método	features	F1 CV	Δ vs base	gap	folds melhores
base	114.301	0.9295	+0.0000	0.0705	0
chi2 (k=80000)	80.000	0.9291	-0.0004	0.0700	2
chi2 (k=50000)	50.000	0.9288	-0.0007	0.0676	2
chi2 (k=20000)	20.000	0.9241	-0.0054	0.0601	1
chi2 (k=10000)	10.000	0.9189	-0.0106	0.0509	1
chi2 (k=2000)	2.000	0.8951	-0.0344	0.0297	0
l1 (C=1)	7.854	0.9214	-0.0081	0.0668	1
svd (pca) (d=300)	300	0.9062	-0.0233	0.0326	0
svd (pca) (d=50)	50	0.8663	-0.0632	0.0059	0
svd (pca) (d=10)	10	0.7812	-0.1483	-0.0013	0
svd (pca) (d=2)	2	0.5482	-0.3813	-0.0001	0

- **Nenhuma técnica superou a baseline** na validação cruzada. O desvio do F1 é ~0,010, então χ^2 com $k \geq 50.000$ empata com ela.

- **O gap só se aproxima de zero com *underfitting*:** com SVD $d \leq 20$ o F1 é baixo e o gap ≈ 0 . Gap baixo aqui significa que o modelo não ajusta nem o treino.
- **Com o mesmo orçamento de atributos, χ^2 supera SVD:** com $k=2.000$ já supera o SVD de 100 componentes (0,8951 contra 0,8838). Os n-gramas específicos carregam o sinal e a projeção densa o dilui.
- **Visualização 2D ($d=2$):** as duas componentes explicam $\sim 0,75\%$ da variância, captam padrões genéricos (tamanho, palavras funcionais) e misturam as classes; o F1 de $\sim 0,55$ é praticamente o acaso.
- **Conclusão:** para texto, o regime esparso com $d \gg N$ é favorável a modelos lineares; reduzir demais descarta informação útil. Se for reduzir, preferir seleção supervisionada (χ^2).

Kernels não lineares (09_kernels_svm)

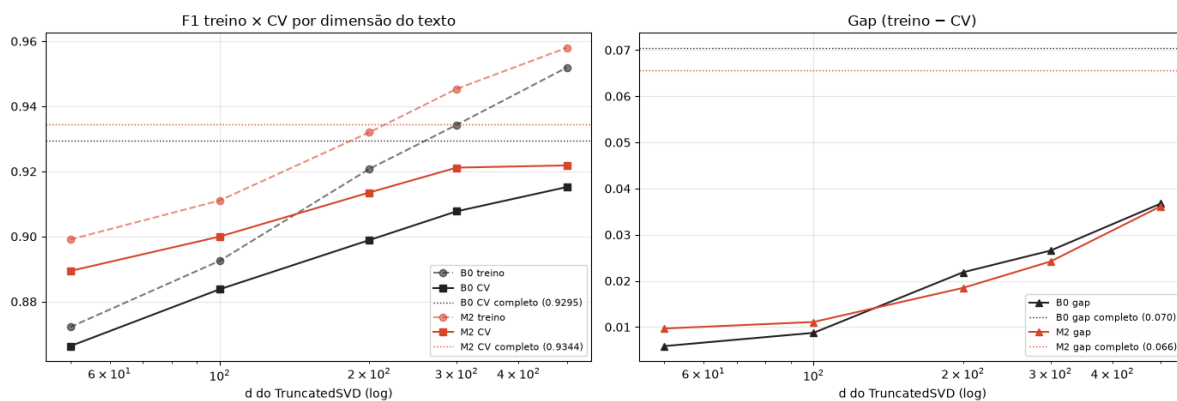
Hipótese: um kernel RBF ou polinomial valeria mais que o LinearSVC? Pelo teorema de Cover o espaço de ~ 114 mil dimensões já é praticamente linearmente separável, e o SVC com kernel exige matriz de Gram (memória $O(n^2)$), muito mais custosa que o **liblinear**. O notebook conclui que o LinearSVC é equivalente ou superior em F1 e muito mais barato.

Redução de dimensão no modelo com spaCy (notebooks 11 e 12)

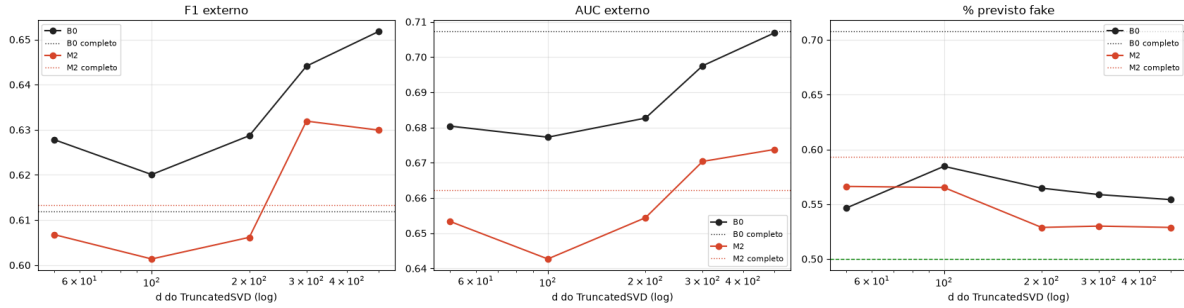
Repetida a pergunta com o M2 ($C=1$, $w=0,03$ fixos). Primeiro, **TruncatedSVD(d) + Normalizer** no bloco de texto (o bloco spaCy não é reduzido):

d (texto)	F1 treino (M2)	F1 CV (M2)	gap (M2)	Δ vs M2 completo	ganho do spaCy (M2 – B0)
completo	1.0000	0.9344	0.0656	n/a	+0.0049
500	0.9580	0.9218	0.0361	-0.0125 (0/5)	+0.0067
300	0.9454	0.9212	0.0242	-0.0132 (0/5)	+0.0134
100	0.9111	0.9000	0.0111	-0.0344 (0/5)	+0.0162
50	0.8991	0.8894	0.0097	-0.0450 (0/5)	+0.0231

O gap cai porque o treino piora, não porque a validação melhora: o F1 CV perde em todo d e em todos os folds, e nenhum d é escolhido pela regra fixada antes da execução (menor gap com $\Delta \geq -0,01$). O spaCy pesa mais com o texto comprimido (+0,023 com $d=50$), mas nenhum M2 reduzido alcança o texto completo. O SVD só foi até $d=500$, por limite de memória.



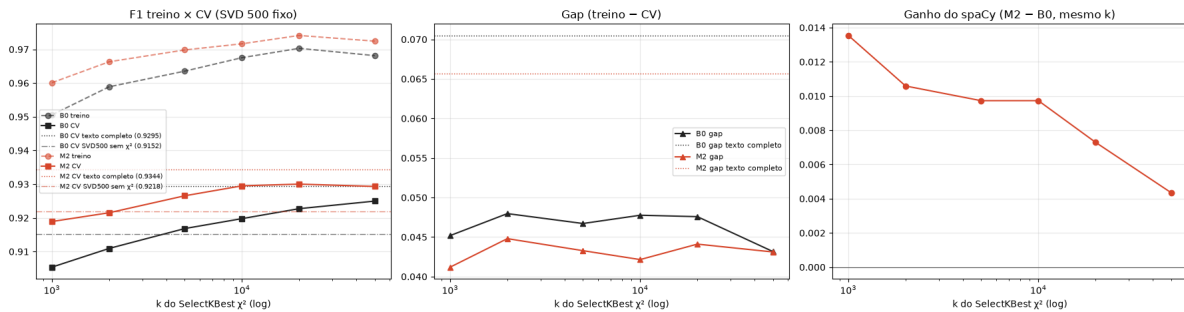
SVD no modelo M2: F1 de treino e CV por dimensão d (esquerda) e gap (direita). Fonte: notebook 11_reducao_dim_svm_spacy.



SVD no modelo M2: desempenho externo (FakeRecogna) por dimensão d . Fonte: notebook 11_reducao_dim_svm_spacy.

Depois, TF-IDF $\rightarrow$ SelectKBest(χ^2 , k) $\rightarrow$ TruncatedSVD(500) $\rightarrow$ Normalizer, ajustado por fold, variando só k :

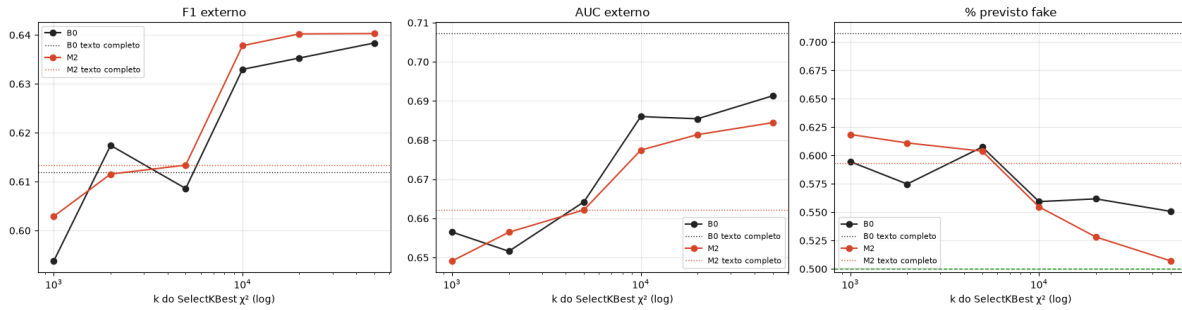
texto	F1 treino	F1 CV (M2)	gap	Δ vs M2 completo	ganho do spaCy	AUC externo (M2 / B0)
completo	1.0000	0.9344	0.0656	n/a	+0.0049	0.662 / 0.707
SVD500 sem χ^2	0.9580	0.9218	0.0361	-0.0125 (0/5)	+0.0067	0.674 / 0.707
χ^2 1 mil + SVD500	0.9601	0.9189	0.0412	-0.0154 (0/5)	+0.0135	0.649 / 0.657
χ^2 5 mil + SVD500	0.9698	0.9266	0.0433	-0.0078 (1/5)	+0.0097	0.662 / 0.664
χ^2 10 mil + SVD500	0.9717	0.9295	0.0422	-0.0049 (1/5)	+0.0097	0.677 / 0.686
χ^2 20 mil + SVD500	0.9741	0.9300	0.0441	-0.0043 (0/5)	+0.0073	0.681 / 0.685
χ^2 50 mil + SVD500	0.9724	0.9293	0.0431	-0.0050 (2/5)	+0.0043	0.684 / 0.691



M2 com SelectKBest χ^2 + SVD(500): F1 treino/CV, gap e ganho do spaCy por k . Fonte: notebook 12_selectk_svd_svm_spacy.

- **O χ^2 recupera o que o SVD sozinho perdia.** Com $k=10-50$ mil o M2 fica em $\sim 0,930$, o mesmo F1 CV do SVM atual (0,9295), e o gap cai de 0,0705 para $\sim 0,043$. Diferente do SVD sozinho, o gap cai **sem** a validação cair frente ao B0.
- **Escolha pela regra do notebook 11: $k = 10$ mil** (F1 CV $0,9295 \pm 0,0081$, gap 0,042). É empate, não vitória: perde para o M2 completo em 4/5 folds. O spaCy contribui o dobro nesse regime (+0,010 contra +0,005).
- **Teste interno** do M2 com χ^2 10 mil + SVD(500): F1 0,9243 e AUC 0,9789, contra 0,9236 e 0,9846 do M2 completo e 0,9222 e 0,9830 do B0, tudo dentro do ruído.

- **Fora do domínio (espiada):** o χ^2 **piora a ordenação**; o AUC do B0 cai de 0,707 para $\leq 0,691$ e o do M2 com k=10 mil fica em 0,677, porque o χ^2 escolhe os n-gramas que separam o Fake.br e especializa o modelo no corpus.



SelectKBest χ^2 + SVD(500): desempenho externo (FakeRecogna) por k. Fonte: notebook 12_selectk_svd_svm_spacy.

Observação. Pela regra "melhor média de CV", a configuração escolhida seria M2 + χ^2 k=50 mil (0,9397), mas o teste avaliou o M2 com todo o texto, e o χ^2 foi aplicado depois de escolher C e w, então essa estimativa é otimista. As avaliações externas são "espiadas" (uma vez, sem intervalo de confiança).

[PENDENTE] O notebook 09 não tem outputs salvos e `resultados_kernels.csv` não existe no repositório; a conclusão acima está escrita no notebook, mas sem evidência numérica versionada. Executar o notebook e versionar o CSV gerado.

5.9 Generalização fora do corpus (teste externo no FakeRecogna)

Treino no Fake.br, avaliação em títulos do FakeRecogna (seção 3.3), sem novo ajuste.

modelo	n_dim	f1_externo	auc_externo	pct_prev_fake
Base (word+char)	114301	0.6122	0.7072	70.7%
PCA/SVD d=2	2	0.4004	0.4822	89.1%
PCA/SVD d=5	5	0.5799	0.6238	66.8%
PCA/SVD d=10	10	0.6005	0.6412	48.4%
PCA/SVD d=20	20	0.6166	0.6622	46.3%
PCA/SVD d=50	50	0.6278	0.6804	54.6%
PCA/SVD d=100	100	0.6201	0.6773	58.5%
PCA/SVD d=200	200	0.6287	0.6827	56.5%
PCA/SVD d=300	300	0.6439	0.6975	55.9%
PCA/SVD d=500	500	0.6518	0.7069	55.4%
Chi2 k=5000	5000	0.6347	0.6976	60.0%
Chi2 k=10000	10000	0.6448	0.7101	57.8%
Chi2 k=20000	20000	0.6451	0.7115	60.2%
Chi2 k=50000	50000	0.6364	0.7110	63.4%
Chi2 k=80000	80000	0.6231	0.7132	69.5%

- **O F1 interno não se mantém:** de ~0,93 para **0,6122** (AUC 0,7072), e o modelo prevê "fake" em **70,7%** dos casos, o que indica viés de domínio.
- Uma redução moderada (SVD d=300–500 ou χ^2 k=10–20 mil) sobe o F1 externo em ~0,03–0,04 e aproxima a proporção prevista de fake de 50%. **O AUC quase não muda (~0,70–0,71):** grande parte do ganho vem de um limiar menos enviesado, não de uma ordenação melhor. O ganho é real, porém modesto.

- Variar `min_df` e `C` no teste externo também mexe pouco (F1 0,61–0,62 para `min_df`; `C` baixo piora por enviar para fake). SVD com $d \leq 5$ fica no nível do acaso.
- **O gargalo é o viés de domínio** entre Fake.br e FakeRecogna, que a redução de dimensionalidade não resolve. Os metadados spaCy (seção 5.6) e o χ^2 (seção 5.8) também não resolvem e, por ordenarem pior, reduzem o AUC externo.

Ressalvas: cada configuração externa foi avaliada uma única vez, sem intervalo de confiança; o conjunto externo é de títulos (mediana de 13 palavras) contra textos de 100 palavras no treino; e o SVD só foi avaliado até $d=300$ na validação interna ($d=500$ aparece apenas no externo).

Observação. O notebook 07 afirma que a redução "generaliza muito melhor". A formulação defensável é a deste documento (ganho modesto, AUC praticamente igual).

5.10 Validação, curva de aprendizado e dependência de convenções

Dados de treino, validação e teste

Para treino, utilizamos 80% do dataset tratado, separados com `random_state=42`. Todos os experimentos de comparação entre configurações de features foram avaliados por validação cruzada (k-fold, $k=5$) sobre esse conjunto de treino, ou seja, cada fold representa 16% do dataset total, 80% do dataset dividido em 5 partes iguais. Em cada uma das 5 iterações, 4 folds são usados para treinar o modelo, equivalente a 64%, e o fold restante é usado para validação, 16% do dataset. Alternando qual fold desempenha esse papel a cada passada, garantindo que cada amostra de treino seja validada exatamente uma vez ao longo do processo.

Testamos e comparamos o uso de metadados estilométricos para o treinamento do modelo contra o uso apenas do texto truncado em 100 palavras e vetorizado via TF-IDF. A métrica de referência até agora foi obtida com os vetores de TF-IDF aplicado ao texto truncado (unigramas e bigramas de palavras) combinado ao TF-IDF aplicado a caracteres (trigramas a quinigramas).

Optamos por não seguir com a pipeline de extração de metadados estilométricos como bloco principal: o ganho de acurácia que ela adiciona é pequeno frente ao esforço de engenharia necessário para extraí-la e validá-la, especialmente quando comparado ao ganho trazido pelos n-gramas de caractere.

Validação. Não há conjunto de validação fixo: a validação durante o treino é a validação cruzada acima.

Teste. 20% do dataset (1.440 textos), separados com `random_state=42`, mantidos fora do treino, da validação cruzada e da seleção de configurações, com a mesma normalização e truncamento do treino. Cada modelo é avaliado uma única vez sobre ele.

Notebook de preparação e separação (necessário para executar os demais): https://colab.research.google.com/drive/1mW8e7tKF_w-ntmOOpB3vcUxF74CKMT0h?usp=sharing

Hold-out versus K-Fold (03_SVM_training)

O notebook compara a validação por K-Fold e por Hold-out (80/20 do treino) e justifica $k=5$ testando $k \in \{2, 3, 5, 10, 20\}$, cada um repetido 5 vezes (config. word, LinearSVC):

k	treino/fold	validação/fold	F1 médio	std entre folds	std entre repetições	tempo (s)
2	2880	2880	0,9033	0,0033	0,0034	2,9
3	3840	1920	0,9109	0,0050	0,0036	2,6
5	4608	1152	0,9139	0,0051	0,0022	4,1
10	5184	576	0,9152	0,0087	0,0021	8,0
20	5472	288	0,9165	0,0156	0,0007	16,8

De $k=2$ para $k=5$ o F1 sobe 0,011 (o viés pessimista diminui); de 5 para 10 o ganho (0,0013) é menor que a variação entre repetições ($\sim 0,002$). Com $k=5$ cada validação tem 1.152 notícias, suficiente para métricas

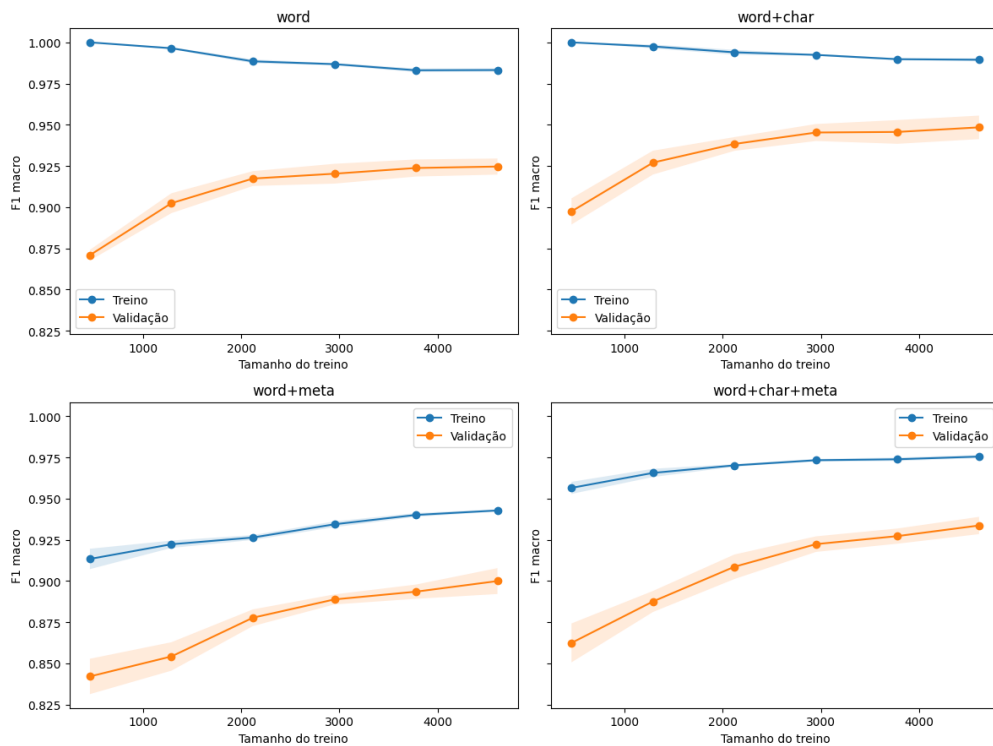
por fold; com $k=20$ são 288 e o F1 varia $3\times$ mais entre folds. O Hold-out custa $5\times$ menos, mas depende da divisão sorteada e usa 20% dos dados apenas para validar.

Curva de aprendizado

F1-macro de treino e de validação em função do tamanho do conjunto de treino (Regressão Logística; validação cruzada estratificada de 5 folds; faixas indicam ± 1 desvio-padrão).

Configuração	F1 treino	F1 validação	Gap
word,	$\approx 0,983$	$\approx 0,925$	$\approx 0,058$
word+char	$\approx 0,989$	$\approx 0,948$	$\approx 0,041$
word+meta	$\approx 0,943$	$\approx 0,900$	$\approx 0,043$
word+char+meta	$\approx 0,976$	$\approx 0,933$	$\approx 0,043$

Valores no maior ponto da curva (≈ 4.600 amostras), coerentes com a Tabela de desempenho da Regressão Logística.



Curva de aprendizado da Regressão Logística (F1 macro de treino e validação, 5 folds; protocolo anterior).

Fonte: notebook 02_LR_training.

Configurações de texto Em word e word+char, a curva de validação praticamente estabiliza e o gap de 4 a 6 pontos não se fecha com mais dados. Isso indica variância moderada e retorno decrescente de acrescentar exemplos do mesmo corpus. O F1 de treino próximo de 1,0 é esperado em um espaço de dezenas de milhares de atributos para milhares de documentos. O padrão não sugere vazamento, já que a validação fica em torno de 0,95 e coincide com a validação cruzada.

Metadados Na regressão logística, word+meta e word+char+meta ficam abaixo das versões sem metadados ($-2,4$ e $-1,5$ ponto de F1), e suas curvas ainda crescem ao final. O efeito é específico desse estimador: no LinearSVC e no Random Forest a diferença é de no máximo $\approx 0,4$ ponto, dentro do desvio-padrão. A hipótese

em investigação é a diferença de escala entre o bloco denso padronizado e o TF-IDF sob regularização L2. Essa decisão reforça a opção de não adotar os metadados como bloco principal.

Comparação com o LinearSVC No SVM, o F1 de treino é $\approx 1,0$ em todas as configurações. A curva de validação em função de C (config. word) atinge o máximo em $C = 1$ ($\approx 0,937$) e não melhora em $C = 10$. Reduzir C diminui o gap, mas piora a validação, e por isso os hiperparâmetros padrão foram mantidos.

Dependência de convenções de redação

Termos como *feira*, *nesta*, *quarta* e *segundo* concentram-se nas notícias verdadeiras (por exemplo, "nesta" aparece em 36% das verdadeiras e 3,5% das falsas), e *lula* e *dilma* nas falsas. Remover os marcadores temporais reduz o F1 em 0,4 a 0,9 ponto, e remover os termos políticos não altera o resultado. A análise por subgrupo (LinearSVC, config. word) revela algo que o F1 global não mostra: 27,4% das notícias falsas com marcador temporal são classificadas como verdadeiras, contra 6,1% nas demais ($4,5\times$). Com treino e teste mascarados, a razão cai para $2,5\times$ (21,1% vs 8,3%), o que indica que parte do efeito vem de convenções jornalísticas e parte de estilo formal mais amplo.

Observação. A curva de aprendizado e a análise de convenções de redação acima têm números do **protocolo anterior** (por exemplo, LR word+char com F1 de validação $\approx 0,948$, enquanto o pipeline atual dá 0,9182). As conclusões qualitativas (gap que não fecha com mais dados; efeito dos marcadores temporais) são plausíveis, mas os valores precisam ser recalculados.

[PENDENTE] Reexecutar a curva de aprendizado e a análise de subgrupo (marcadores temporais) no pipeline atual.

5.11 Modelo de produção

Distinção das métricas. As métricas do notebook 12 são do pipeline sem calibração. O manifesto registra F1 interno calibrado de 0,9208, AUC de 0,9789 e Brier de 0,0569, antes do reajuste final em 7.200 notícias. Elas não são avaliação independente do artefato final. O contrato de pesquisa usa **confidence**; o serviço HTTP do app usa **fakeProbability**, **fakeScore** e **analysisStatus**, com scores nulos para menos de 30 palavras.

O modelo adotado para o app é o $M2 + \chi^2$ (10 mil) + SVD(500), definido em `modelo_olimpico.py` e exportado por `exportar_modelo.py` (`modelVersion = "svm-spacy-chi2k10k-svd500-v1"`). O word+char completo sem spaCy (B0) é mantido como baseline de referência.

Por que este modelo. Na validação cruzada ele iguala o F1 do B0 (0,9295) com gap de treino-validação bem menor, e o teste interno sem calibração registra F1 0,9243. A versão calibrada registra 0,9208; isso não demonstra ganho estatisticamente significativo. Além disso, seus atributos de estilo podem ser calculados para qualquer texto novo, sem os metadados do corpus, e sua pontuação se decompõe exatamente em n-gramas e atributos, o que alimenta as justificativas exibidas ao jogador.

Critério	B0 (referência)	$M2 + \chi^2$ 10 mil + SVD(500) (produção)
F1 CV (5 folds)	0,9295	0,9295
Gap treino—CV	0,0705	0,042
F1 teste interno, sem calibração	0,9222	0,9243
F1 teste interno, calibrado	não reportado	0,9208
AUC externo (FakeRecognia, títulos)	0,707	0,677
Explicação	coeficientes por n-grama	decomposição exata por n-grama e por atributo spaCy

Limite conhecido. A redução do gap não melhora a generalização entre fontes: fora do Fake.br o AUC é de ~0,68 (contra 0,707 do B0) e o F1 de ~0,64, em títulos. O modelo deve, portanto, ser apresentado ao jogador como **indício**, e não como veredito.

Pipeline (entrada: texto bruto da notícia):

1. **PreparadorEntrada:** aplica **normalizar** e **truncar** (idênticos ao `01_Data_Prep`) e extrai os 24 atributos spaCy. O modelo de linguagem é carregado sob demanda e não é gravado no arquivo exportado.
2. Bloco de texto: $\text{TF-IDF word+char} \rightarrow \text{SelectKBest}(\chi^2, k=10 \text{ mil}) \rightarrow \text{TruncatedSVD}(500) \rightarrow \text{Normalizer}$.
3. Bloco spaCy: imputação por mediana e padronização, com peso $w = 0,03$.
4. `LinearSVC(C=1, class_weight="balanced")` com calibração sigmoide (`CalibratedClassifierCV, StratifiedKFold(5), ensemble=False`), que dá $P(\text{fake})$.

Explicação por notícia (`explicar`): como o bloco de texto é $z = V \cdot x / \|V \cdot x\|$, o score de texto se decompõe exatamente em contribuições por n-grama selecionado, e o bloco spaCy em contribuições por atributo. `analisar` devolve o contrato histórico da pesquisa (`classification, confidence, reasons, modelVersion`): **reliable** se $P(\text{fake}) \leq 0,35$, **unreliable** se $\geq 0,65$ e **uncertain** entre os dois. Os limiares são decisão de produto. Os motivos combinam os n-gramas de maior contribuição e frases descritivas dos atributos de estilo mais influentes.

Exportação (`exportar_modelo.py`): confere que o pré-processamento reproduz o split oficial e que o pipeline não calibrado reproduz o F1 de teste do notebook 12 (0,9243, tolerância 0,002); avalia o modelo calibrado; treina o modelo final em treino+teste (7.200 notícias; `--somente-treino` evita isso); salva o `.joblib` com um `.json` de metadados (versões de Python, scikit-learn, numpy, scipy, spaCy e `pt_core_news_sm`, hiperparâmetros, métricas e sha256) e confere que o arquivo recarregado dá as mesmas probabilidades. O artefato principal fica em `machine-learning/supervised-learning/modelos/`, com cópia congelada em `model-engine/models/supervised/assets/` e só carrega se `modelo_olimpico.py` estiver importável e as versões forem as registradas.

6. Aprendizado não supervisionado

A linha não supervisionada está documentada no repositório em `machine-learning/unsupervised-learning/` (branch `dev`): sete métodos de detecção de anomalias, agrupamento e aprendizado semi-supervisionado, uma comparação externa com o Jev (LLM) e, como método final de descoberta, a **mineração de padrões frequentes com FP-Growth**. As seções a seguir seguem o relatório técnico do instituto, com as referências [n] listadas ao final desta seção.

6.1 Análise de classificadores não supervisionados

Analisamos os algoritmos *K-Means*, HDBSCAN, DBSCAN, One-Class SVM, LOF, FP-Growth, PU (um algoritmo semi-supervisionado), LDA e *Isolation Forest*.

Enquanto os demais focam em agrupar temas ou contagem de palavras (pecando em custo computacional ou na perda do contexto semântico), o *Isolation Forest* destaca-se como um excelente detector de anomalias (*outliers*) focado no padrão de escrita.

Isolation Forest:

- Treinamento no padrão "normal": O modelo aprende o estilo de escrita de notícias legítimas e bem estruturadas.
- Identificação de *Outliers*: Notícias apelativas (com excesso de caixa alta, pontuação ou termos de urgência como *"BOMBA!"*) ficam nas bordas da distribuição e são isoladas rapidamente, gerando um alto score de anomalia.

<i>Classificador</i>	<i>Descrição</i>	<i>Vantagens</i>	<i>Desvantagens</i>
<i>K-Means</i>	Agrupar notícias em K clusters com base na similaridade de vetores de texto (<i>embeddings</i>).	· Alta velocidade de execução; · Ideal para organizar notícias já salvas do banco de dados em categorias ou “temas de fake news”.	· Exige definir o número fixo de clusters (K) previamente; · Não lida bem com ruídos ou formatos de clusters muito irregulares.
<i>HDBSCAN</i>	Algoritmo baseado em densidade que agrupa textos similares e identifica pontos isolados (<i>outliers</i>).	· Descobre o número de clusters automaticamente; · Identifica com precisão notícias inéditas/estranhas (<i>outliers</i>) inseridas pelo usuário.	· Requer maior processamento computacional e ajuste de parâmetros de densidade; · Pode classificar muitos dados como ruído se descalibrado (pode dar falsos positivos).
<i>LDA (Latent Dirichlet Allocation)</i>	Algoritmo probabilístico de modelagem de tópicos (<i>Topic Modeling</i>) que extrai temas ocultos nos textos.	· Excelente para identificar os assuntos recorrentes das notícias falsas; · Fornece contexto à LLM para gerar as perguntas socráticas.	· Requer pré-processamento pesado de texto; · Os tópicos gerados podem ser difíceis de interpretar sem intervenção humana (não é ideal no nosso caso, pois o humano não vai intervir nisso).
<i>Isolation Forest</i>	Detecta anomalias isolando pontos de dados discrepantes no espaço vetorial.	· Adequado para sinalizar conteúdos inéditos do usuário que usam padrões textuais anômalos, sensacionalistas ou atípicos.	· Não explica por que o texto é anômalo, o que não atende à metodologia de ensino que queremos seguir; · Necessita da LLM para analisar e desdobrar a anomalia em perguntas.

6.1.1 Isolation Forest: funcionamento

Dois modos de uso.

- **Deteção de novidade** (o modo usado no projeto): o modelo é treinado só com exemplos de um rótulo (aqui, notícias verdadeiras), e qualquer dado que caia fora dessa região é tratado como anomalia.
- **Deteção de outliers**: o modelo é treinado com tudo misturado (por exemplo, 99% de um rótulo e 1% do outro) e encontra as anomalias naturalmente, por ficarem fora do centro. É útil quando os rótulos do conjunto não são conhecidos.

Treinamento. O algoritmo constrói árvores de isolamento. Cada árvore parte de uma amostra aleatória de registros (256 por padrão no scikit-learn). Em cada nó sorteia-se uma *feature* (coluna) e, em seguida, um valor entre o menor e o maior daquela feature; os valores menores vão para a esquerda e os maiores para a direita. O processo se repete até não sobraem pontos para separar ou até a profundidade máxima. Todo o procedimento é repetido para formar a floresta (100 árvores por padrão; os experimentos do projeto usam `n_estimators=300`).

Previsão. Um dado novo percorre as árvores. Se ele é isolado por um caminho curto na maior parte delas, tem maior chance de ser anomalia; se percorre um caminho longo, tende a ser normal. O score de anomalia resume o comprimento médio desses caminhos.

Vantagens	Desvantagens
Tempo linear e alta velocidade: depende do tamanho das amostras, e não do tamanho do dataset	Não entende contexto
Pouco processamento computacional	Não explica por que o dado é anômalo

6.1.2 Por que a detecção de anomalias foi a primeira abordagem

A hipótese inicial foi identificar padrões linguísticos e estilísticos atípicos, frequentes em notícias falsas, que recorrem a gatilhos sensacionalistas, desvios gramaticais, pontuação ostensiva e termos apelativos para provocar reação emocional imediata. O modelo seria treinado com notícias legítimas de veículos de imprensa consolidados, o que estabelece o padrão textual esperado do jornalismo; ao receber uma notícia nova, mede o desvio em relação a esse padrão e devolve um score de anomalia.

Exemplos de pontos aberrantes esperados:

- **Gatilhos de urgência e alarmismo:** termos em caixa alta como "BOMBA", "URGENTE", "ATENÇÃO", "COMPARTILHE ANTES QUE APAGUE".
- **Anomalias sintáticas e ortográficas:** erros gramaticais recorrentes, frases truncadas e pontuação excessiva ("!!!", "???").
- **Viés emocional extremo:** vocabulário carregado de subjetividade, que apela ao medo, à raiva ou à indignação.

Pontos positivos	Pontos negativos
Independência de checagem de fatos: analisa forma e estilo, sem consultar bases em tempo real	Não detecta mentiras bem redigidas: notícias falsas em estilo jornalístico formal podem passar
Padrões inéditos (zero-day): sinaliza novas narrativas assim que surgem, se houver discrepância estilística	Falsos positivos: opinião, sátira, crônicas e títulos legítimos propositalmente chamativos podem ser marcados
Escalabilidade e agilidade: o cálculo do score leva milissegundos e permite grande volume	Dependência do baseline: exige notícias legítimas atualizadas e sem vícios de escrita
Sintonia com a proposta socrática e gamificada: o score indicativo é ponto de partida para instigar a investigação, sem resposta conclusiva	Sensibilidade à evolução da linguagem: gírias, neologismos e mudanças de estilo podem ser lidos como desvio

Observação. O Isolation Forest foi escolhido na etapa inicial, mas a revisão crítica da seção 6.6 mostrou que suas acurácias altas (0,97) refletem a autoria do corpus e não a capacidade de detectar falsidade. Por isso o método final de descoberta passou a ser o FP-Growth (seções 6.7 e 6.8). A limitação "não explica por que o texto é anômalo" é justamente a que o projeto resolve com atributos de estilo interpretáveis e padrões do FP-Growth.

6.2 Resumo do relatório

O modelo principal atual é o FP-Growth sem autoria da raiz de `machine-learning/unsupervised-learning/: fp_growth_principal.py` e `fp_growth_principal_sem_autoria.ipynb`, variante `sintaxe ampliada`. No run `fp-growth-metadados-ampliados-20261008T002311Z`, são 22 atributos ativos, 44 itens, 523 regras direcionais elegíveis e 345 padrões distintos. Na validação, 422 regras mantêm os filtros e 347 também têm redescoberta $\geq 80\%$ em 100 reamostragens dos grupos de treino. O teste conhecido é exploratório. O método descreve coocorrências e não classifica a veracidade de notícias.

Palavras-chave: aprendizado não supervisionado; mineração de padrões; FP-Growth; notícias falsas; viés de autoria.

6.3 Introdução e objetivo

Os tópicos abordados nesta documentação são: o comparativo de pelo menos três modelos, a indicação de quando utilizá-los, vantagens e desvantagens, a descrição de um baseline, do método final e da extração de regras [1]. Este estudo organiza essas exigências com os notebooks, scripts, manifestos e resultados desta trilha [2–8].

O objetivo que orientou a decisão metodológica é investigar regularidades observáveis em notícias do corpus e transformá-las em perguntas verificáveis por analistas. "Notícia falsa" é o rótulo do corpus, não uma propriedade que uma medida de estilo possa demonstrar por si. Por essa razão, as pontuações de anomalia e as associações estatísticas são tratadas como sinais exploratórios.

6.4 Dados e protocolo experimental

O conjunto utilizado é o Fake.br-Corpus em revisão fixa, validada por SHA-256 nos manifestos de execução. Ele contém 7.200 notícias, com 3.600 em cada classe. Na partição canônica por grupos, 4.320 registros compõem o treino, 1.440 a validação e 1.440 o teste. Os pares alinhados e duplicatas textuais normalizadas são mantidos no mesmo grupo, sem sobreposição de IDs entre partições [3, 4].

As seis medidas de estilo são calculadas nos primeiros 300 caracteres: presença de autor, razão de tipos por tokens, densidade de links, densidade de pontuação, proporção de palavras em maiúsculas e diversidade lexical. Para descoberta temática, os notebooks usam TF-IDF de palavras e caracteres sobre textos truncados a 200 palavras. São representações diferentes e suas métricas não formam uma escala única [3–5].

Na trilha de novidade, os algoritmos aprendem sobretudo a partir de notícias True; o limiar de alerta é o percentil 95 dos scores de True na validação. Na trilha temática, o agrupamento é exploratório. O FP-Growth usa apenas o treino canônico e não consulta o rótulo na mineração; os rótulos aparecem numa avaliação posterior dos padrões congelados [3, 6, 7].

6.5 Jornada dos modelos avaliados

A sequência registrada começa com variantes de detecção de anomalias, segue por agrupamento e uma linha complementar PU, e termina na mineração de padrões frequentes. Os experimentos antigos variaram texto integral, prefixo, janelas e política de autoria; permanecem como histórico e não foram misturados ao ranking canônico [2, 8].

Método	Quando é indicado	Vantagem no projeto	Limitação observada
Isolation Forest	Triar desvios em atributos compactos.	Treino em True; baseline de novidade rápido e comparável.	Alerta de estilo não verifica fatos; sensível à autoria.
LOF	Identificar baixa densidade local em relação a vizinhos.	Bom ranking de novidade no split compartilhado.	Depende da vizinhança e da validação com rótulos.
One-Class SVM	Aprender fronteira de normalidade em escala adequada.	Comparação de uma hipótese geométrica distinta.	Baixo recall Fake no corte canônico.
K-means	Explorar grupos ou distância a centroides.	Fornece perfis temáticos e baseline de novidade.	Grupos forçados; separação temática fraca.
DBSCAN	Encontrar grupos densos e ruído.	Distingue cobertura, ruído e alerta.	Muitos itens sem grupo; parâmetros sensíveis.
HDBSCAN	Explorar densidades heterogêneas no treino.	Agrupamento sem fixar número de grupos.	Grupos extremamente enviesados para o nosso contexto, que não agregam em nada o valor dos dados.

Método	Quando é indicado	Vantagem no projeto	Limitação observada
PU Learning	Estudar positivos conhecidos com não rotulados.	Linha complementar para contraste.	Simulação constrói U com True conhecidos; outro protocolo.

Quadro 1 Aplicabilidade e limites dos métodos examinados. Fonte: [2–5, 8].

O quadro distingue comparação metodológica de escolha final. HDBSCAN não fornece decisão fora da amostra no ambiente registrado; PU Learning é semissupervisionado e seu teste histórico não é diretamente comparável ao dos detectores [2–5].

6.5.1 Resultados quantitativos no protocolo canônico

Método	Acurácia	Recall Fake	FPR True	ROC-AUC
Isolation Forest	0,9729	0,9847	0,0389	0,9786
LOF	0,9479	0,9347	0,0389	0,9766
One-Class SVM	0,5563	0,1500	0,0375	0,8738
K-means novidade	0,5382	0,1111	0,0347	0,8471
DBSCAN novidade	0,9667	0,9847	0,0514	0,9598

Tabela 1 Teste canônico compartilhado, com corte q95. Fonte: [3].

No holdout temporal secundário, a acurácia do Isolation Forest cai para 0,8592 e a do DBSCAN para 0,8155; o LOF registra 0,9189. Isso reforça que o número obtido num recorte específico não basta para sustentar uma regra geral [3, 4]. Na descoberta temática canônica, K-means teve silhouette 0,0083 e HDBSCAN 0,0398 nos itens atribuídos; os valores baixos e os índices externos modestos não autorizam tratar grupos como classes de veracidade [3–5].

6.5.2 Índice de Jaccard como medida complementar

O grupo poderia ter usado o índice de Jaccard para quantificar a sobreposição entre conjuntos de termos ou atributos binários das notícias, ou entre conjuntos de registros cobertos por agrupamentos. Essa comparação faria sentido para os métodos de agrupamento e novidade quando representados como conjuntos; os vetores contínuos de estilo e TF-IDF exigiriam discretização ou binarização prévia. Jaccard não substituiria automaticamente as distâncias usadas nos experimentos. A lacuna de análise de sobreposição foi corrigida na etapa final: o FP-Growth calcula Jaccard entre as coberturas do antecedente e do consequente de cada regra e entre as coberturas de padrões para sinalizar redundância [6, 11].

6.6 Revisão crítica das acurácias

Houve uma interpretação intermediária equivocada: as altas acurácias dos detectores sugeriram utilizá-los como classificadores Fake/True. O experimento, porém, aprende desvio em relação ao padrão True e seleciona ou calibra componentes com a validação; o rótulo atribuído no teste é uma convenção de avaliação. Um texto atípico pode ser verdadeiro, e uma notícia falsa pode preservar o estilo de uma fonte legítima [3].

O corpus é limitado para essa inferência. A autoria é praticamente separada por classe, as fontes e rótulos estão confundidos, a avaliação histórica consultou o teste durante o desenvolvimento e parte dos splits antigos difere do canônico. A acurácia também muda com a prevalência e o limiar. O resultado mais forte é a capacidade de reconhecer particularidades deste conjunto, não a validação factual de notícias novas [3, 8].

O controle de autoria anterior já apontava o problema: no split por notícia, o recall anotado caía de 0,9733 com autoria para 0,6700 sem autoria; no split pareado, de 0,9750 para 0,7069. Esses números pertencem a protocolos diferentes e servem como evidência histórica de sensibilidade à feature, sem comparação direta com a Tabela 1 [8].

6.7 Mineração FP-Growth e extração de regras

O modelo principal atual é o FP-Growth sem autoria da raiz de `machine-learning/unsupervised-learning/`: `fp_growth_principal.py` e `fp_growth_principal_sem_autoria.ipynb`, variante `sintaxe_ampliada`. No run `fp-growth-metadados-ampliados-20261008T002311Z`, são 22 atributos ativos, 44 itens, 523 regras direcionais elegíveis e 345 padrões distintos. Na validação, 422 regras mantêm os filtros e 347 também têm redescoberta $\geq 80\%$ em 100 reamostragens dos grupos de treino. O teste conhecido é exploratório. O método descreve coocorrências e não classifica a veracidade de notícias.

6.7.1 Preparação e execução

O corpus congelado contém 7.200 notícias. A partição canônica mantém pares e duplicatas juntos: 4.320 notícias de treino, 1.440 de validação e 1.440 de teste. A extração normaliza em NFKC, remove BOM inicial e usa os primeiros 300 caracteres. As taxas POS/DEP usam tokens lexicais; a pontuação usa tokens não espaciais. Ausência de tokens elegíveis não equivale a taxa zero.

Os quantis 25% e 75% e as regras são aprendidos somente no treino. O suporte mínimo é 0,08, com até três itens; os filtros são $\text{confidence} \geq 0,50$, $\text{lift} \geq 1,05$ e $\text{Jaccard} \geq 0,10$. Autoria e classe não entram na mineração nem no ranking dos candidatos; entram nas descrições posteriores. As famílias de atributos foram informadas por análises exploratórias anteriores.

A referência corrigida tem 152 regras elegíveis e 93 padrões; POS ampliado tem 189 e 118; `sintaxe ampliada`, o principal, tem 523 e 345; a ablação ampla tem 4.350 e 2.546. Esses números vêm de `variant_summary.csv`, conferidos com os CSVs de cada variante. A versão antiga de estilo (18 itemsets, 20 regras brutas e oito elegíveis) permanece apenas no histórico.

6.7.2 Como ler as regras

Support mede a frequência da combinação; confidence mede a frequência do consequente entre os registros com antecedente; lift compara essa frequência com a frequência marginal do consequente; Jaccard mede a interseção dividida pela união das coberturas. Regras inversas podem representar o mesmo padrão com confidences diferentes: 523 regras não são 523 padrões independentes.

A fila de revisão contém 20 candidatos selecionados sem classe ou autoria no ranking e cobre 1.346 notícias da validação. A união de todos os 345 padrões cobre 1.439/1.440, incluindo padrões instáveis; isso não mede eficácia pedagógica. A combinação `DEP_acl:relcl_rate_baixo + POS_PRON_rate_baixo`, por exemplo, ocorre em 441 notícias da validação e sua direção representativa tem redescoberta de 100%. `baixo` pode incluir zero: a anotação automática no trecho não comprova ausência do fenômeno no documento completo.

6.8 Avaliação externa e controles

Os valores de composição e controles de autoria abaixo pertencem ao experimento histórico de estilo. Para o principal atual, as tabelas separam treino, validação e teste: na validação, 105 padrões têm maior frequência em Fake, 235 em True e cinco empatam. Não há validação externa nem intervalos de associação por classe. O catálogo de pesquisa permanece `research_only`; o catálogo do app é uma cópia distinta com status `experimental_descriptive_comparison`, sem aprovação editorial humana.

No experimento histórico de estilo, o notebook aplica os limites textuais congelados ao corpus, confere se a cobertura do treino reproduz o membership persistido e só então calcula as proporções por classe. O notebook histórico avaliou, em separado, o baseline com autoria. As avaliações abrangem as 7.200 notícias, inclusive as 4.320 usadas para descobrir padrões: são descrições do mesmo corpus, não testes independentes de generalização [6, 7, 9].

Na avaliação histórica dos padrões de estilo sem autoria, `punctuationDensity_alto` com `typeTokenRatio_baixo` aparece em 1.007 notícias, com 70,3% Fake, perante baseline global de 50%. `Diversidade_baixo` com `typeTokenRatio_baixo` aparece em 1.242 notícias, com 61,6% Fake. Em sentido oposto, `diversi-`

dade_alto com typeTokenRatio_alto tem 31,2% Fake em 1.199 ocorrências. Esse lift Fake de classe difere do lift de associação calculado durante a mineração [6, 11].

6.8.1 Controle da autoria

O controle de autoria mostra que 3.528 de 3.601 notícias sem autor são Fake, enquanto 3.527 de 3.599 com autor são Real; o Cramér V é 0,960. Dos 31 padrões do baseline, 21 contêm autoria. Portanto, um padrão com sem_autor pode herdar quase toda a associação ao rótulo da composição do corpus. A análise principal exclui essa variável da mineração textual [6, 9, 11].

No notebook histórico de estilo, a mineração sem autoria preserva IDs e limites textuais do baseline, e as coberturas são examinadas dentro dos estratos com_autor e sem_autor. O padrão punctuationDensity_alto com typeTokenRatio_baixo tem 70,3% Fake no agregado; no estrato com autor, tem 0,7% perante baseline de 2,0%; no estrato sem autor, 98,9% perante baseline de 98,0%. A associação agregada não pode ser atribuída isoladamente ao texto [6, 11].

Há associações que merecem nova investigação, como punctuationDensity_baixo com uppercaseRatio_baixo entre notícias com autor: 32 Fake em 524 ocorrências, contra 72 Fake em 3.599 notícias desse estrato. O número de exemplos minoritários é pequeno; o resultado não estabelece causalidade, robustez por fonte ou validade fora do Fake.br [9].

6.9 Discussão e aplicação pelo método socrático

A aplicação pretendida é um roteiro de perguntas orientado por evidências. Cada padrão informa quais atributos coocorrem, em quantas notícias e com que diferença observada entre estratos. O analista então testa explicações alternativas: a ausência de autoria decorre do conteúdo ou da forma como a fonte foi coletada? A associação textual persiste quando a autoria é mantida fixa? O mesmo padrão aparece em outras fontes, temas e períodos? Notícias verdadeiras que exibem o padrão têm características contextuais semelhantes?

Esse processo é socrático porque usa as regras para questionar uma hipótese, procurar contraexemplos e delimitar o que os dados permitem concluir. A implementação atual gera padrões, coberturas e controles; a transformação desses achados em diálogo com o usuário é uma proposta de uso analítico, ainda não um módulo conversacional implementado (a interface do jogo exibe apenas cartões socráticos fixos, seção 8.4).

A prioridade seguinte de validação é aplicar critérios previamente congelados a um corpus externo com fontes e épocas distintas, registrar cobertura e efeitos por estrato e submeter os achados a revisão humana. Também convém ampliar atributos menos dependentes de metadados de publicação, mantendo a distinção entre estilo, fonte e checagem factual.

6.10 Conclusão

Os experimentos de anomalia e agrupamento mapearam métodos e revelaram sinais fortes, mas alta acurácia neste corpus não equivale a detecção confiável de falsidade. O FP-Growth sem autoria foi escolhido para investigar a pergunta científica: quais combinações textuais são frequentes, quais regras direcionais elas geram e como sua leitura muda ao controlar a autoria. O Jaccard acrescentou uma medida explícita de sobreposição na etapa final. O resultado é uma base reproduzível para investigação guiada, com limites quanto a causalidade e generalização [6, 11].

Observação. As avaliações do FP-Growth (7.200 notícias) incluem o treino de descoberta e portanto **descrevem o corpus, sem medir generalização**. As acurácias altas de Isolation Forest e DBSCAN (0,97) foram reinterpretadas no próprio relatório: os detectores aprendem desvio em relação às notícias verdadeiras, e a autoria separa as classes quase por completo. Por isso o Isolation Forest, escolhido na primeira análise (seção 6.1), deixou de ser o método final.

Observação. Os arquivos `fp_growth*.ipynb`, `REGRAS_FP_GROWTH.md` e `history/RESULTADOS.md` estão em `main`, incorporados pelo *pull request* #69 (*merge* de `dev`, em 07/10/2026), que incluiu o #54. Os números do relatório foram conferidos contra `history/RESULTADOS.md` e `REGRAS_FP_GROWTH.md`.

Referências do relatório não supervisionado

- [1] APRENDIZADO NÃO SUPERVISIONADO. Briefing em PDF, uma página. Arquivo `AprendizadoNaoSupervisionado.pdf`, projeto Olimpo Fake News AI.
- [2] OLIMPO FAKE NEWS AI. Experimentos não supervisionados. `machine-learning/unsupervised-learning/README.md`. Consulta em: 30 set. 2026.
- [3] OLIMPO FAKE NEWS AI. Comparação histórica dos experimentos. `machine-learning/unsupervised-learning/history/RESULTADOS.md`. Consulta em: 30 set. 2026.
- [4] OLIMPO FAKE NEWS AI. Documentação de DBSCAN e K-means. `machine-learning/unsupervised-learning/docs/modelos/dbscan.md` e `history/modelos/kmeans.md`. Consulta em: 30 set. 2026.
- [5] OLIMPO FAKE NEWS AI. Documentação de HDBSCAN e PU Learning. `machine-learning/unsupervised-learning/docs/modelos/hdbscan.md` e `history/modelos/pu-learning.md`. Consulta em: 30 set. 2026.
- [6] OLIMPO FAKE NEWS AI. Notebook histórico FP-Growth sem autoria e controles. `machine-learning/unsupervised-learning/history/mineracao-de-padroes/fp-growth-legado/fp_growth_principal_sem_autoria_controles.ipynb`; run `fp-growth-controls-20260929T215211Z`. Consulta em: 30 set. 2026.
- [7] OLIMPO FAKE NEWS AI. Notebook histórico de avaliação do FP-Growth com autoria. `machine-learning/unsupervised-learning/history/mineracao-de-padroes/avaliacao_fp_growth_com_autoria.ipynb`; run `fp-growth-evaluation-20260929T213741Z`. Consulta em: 30 set. 2026.
- [8] OLIMPO FAKE NEWS AI. Modelos e experimentos anteriores. `machine-learning/unsupervised-learning/history/consulta-modelos-anteriores/README.md`. Consulta em: 30 set. 2026.
- [9] OLIMPO FAKE NEWS AI. Notebook baseline FP-Growth com autoria. `machine-learning/unsupervised-learning/history/mineracao-de-padroes/fp-growth-legado/fp_growth_baseline_com_autoria.ipynb`; run `fp-growth-20260929T212058Z`. Consulta em: 30 set. 2026.
- [10] INSTITUTO DE PESQUISAS ELDORADO. Logotipo institucional. Disponível em: https://www.eldorado.org.br/wp-content/themes/eldorado/assets/img/logo_eldorado.svg. Acesso em: 30 set. 2026.
- [11] OLIMPO FAKE NEWS AI. Regras extraídas pelo FP-Growth e utilidade para o projeto. `machine-learning/unsupervised-learning/REGRAS_FP_GROWTH.md`. Consulta em: 30 set. 2026.

7. Resultados e discussão integrada

7.1 Quadro-resumo

Linha de trabalho	Protocolo	Resultado principal	Como ler
LinearSVC word+char, C=1	CV 5 folds, treino do Fake.br	F1 $0,9295 \pm 0,0099$	referência interna atual
Regressão Logística word+char	CV 5 folds	F1 $0,9182 \pm 0,0136$ (C=1) $\rightarrow$ 0,9281 (C=10)	baseline interpretável
Random Forest word+char	CV 5 folds, protocolo anterior	F1 0,9285	[PENDENTE]
Redução (χ^2 , L1, SVD)	CV 5 folds	nenhuma supera a base	reexecutar seção 5.8

Linha de trabalho	Protocolo	Resultado principal	Como ler
Kernels RBF/polinomial	CV 5 folds	sem ganho (notebook, sem CSV)	seção 5.8
Teste externo FakeRecogna	treino Fake.br, títulos	F1 0,6122, AUC 0,7072, 70,7% previsto fake	viés de domínio
Metadados do corpus	LR (protocolo anterior) / SVM (grid search, 3 folds)	-2,4 pt (LR); +1,4 pt (SVM)	em conflito
Metadados spaCy (M2)	CV 5 folds, C=1, w=0,03	F1 0,9344 (+0,005; 5/5 folds); teste interno +0,0014; AUC externo 0,707 → 0,662	ganho pequeno e frágil; piora a transferência
M2 + χ^2 10 mil + SVD(500) (modelo de produção)	CV 5 folds	F1 0,9295, gap 0,042 (B0: 0,9295, 0,0705); AUC externo 0,677	menos ajuste ao treino, sem ganho de generalização
Isolation Forest / LOF / DBSCAN	split canônico, q95	acurácia 0,97 / 0,95 / 0,97	descreve o corpus; reinterpretado
FP-Growth principal, sintaxe ampliada	treino canônico e validação por grupos	523 regras elegíveis, 345 padrões; 422 sustentadas, 347 também estáveis	descrição, não classificador

7.2 O que os resultados sustentam

1. **Dentro do Fake.br, o problema é bem resolvido por um modelo linear simples.** O LinearSVC sobre TF-IDF word+char chega a ~0,93 de F1 em validação cruzada; arquiteturas mais complexas (RF, kernels) e reduções de dimensionalidade não melhoram.
2. **Esse desempenho não se transfere.** Em títulos do FakeRecogna o F1 cai para ~0,61, com forte viés para "fake". A distância entre os dois corpora é de domínio e de formato, e nenhuma técnica de dimensionalidade a resolveu.
3. **Parte do sinal é artefato da fonte.** Autoria (Cramér V 0,96), tipografia e marcadores temporais (27,4% das falsas com marcador temporal são classificadas como verdadeiras, contra 6,1% nas demais; protocolo anterior) explicam parte dos resultados altos. Daí a normalização, o truncamento e a decisão de tratar o FP-Growth como gerador de hipóteses. A normalização eliminou o artefato de espaços (SPACE de 2,93% para 0,00% nas falsas), mas resta um sinal de tamanho: 17% das falsas têm menos de 100 palavras, contra ~0% das verdadeiras.
4. **Os dois métodos respondem a perguntas diferentes.** O supervisionado classifica; o não supervisionado descreve padrões de estilo que podem alimentar perguntas socráticas. Nenhum dos dois verifica fatos.
5. **Os metadados agregam pouco e atrapalham fora do domínio.** O bloco spaCy soma +0,005 na CV (só com peso w=0,03), não melhora o teste interno e reduz o AUC externo;
6. **Reduzir a dimensão não melhora a generalização.** SVD só diminui o gap por *underfitting*; χ^2 + SVD(500) reduz o gap sem perder F1 na CV, mas piora o AUC externo (0,677 contra 0,707).

7.3 Limitações

- Um único corpus de treino, rotulado por fonte; rótulo e fonte estão confundidos.
- Partição aleatória no supervisionado (pares temáticos podem cruzar treino e teste) e por grupos no não supervisionado; os dois não são comparáveis entre si.
- Resultados de teste interno com a configuração já escolhida; cada configuração externa foi avaliada uma vez.
- Nenhum modelo foi calibrado nem testado em notícias reais submetidas por usuários.

8. MVP: plataforma Olimpo

Estado: em desenvolvimento ativo. O backend de jogo está implementado e testado; a interface já cobre o fluxo principal do jogo (seção 8.3); **o modelo de ML ainda não está integrado** e as perguntas socráticas são cartões fixos, não geradas dinamicamente.

8.1 Visão do produto

O Olimpo é uma plataforma gamificada, educativa e colaborativa para treinar o reconhecimento de notícias falsas. Em salas multiplayer ou solo, os participantes analisam manchetes sob tempo limite e votam entre **É FAKE NEWS**, **É FATO REAL** e **TENHO DÚVIDA**. Há ainda desafios autorais (um jogador cadastra uma notícia falsa convincente e ganha pontos de blefe por cada colega induzido ao erro), feedback educativo ao fim da rodada e ranking com conquistas.

8.2 Arquitetura

Stack. Next.js 16 (App Router, Server Components e Route Handlers), React 19, TypeScript em modo estrito, Tailwind CSS v4; PostgreSQL com Drizzle ORM; Redis para publicação e assinatura de eventos; validação com Zod v4. A ingestão de notícias usa JSDOM (sem execução de scripts), [@mozilla/readability](#), cliente **undici** com *DNS pinning* contra DNS rebinding, `ipaddr.js` para bloquear IPs privados e a API Jina Reader como fallback.

Camadas. O backend segue Clean Architecture (domínio → aplicação → infraestrutura → apresentação), e o frontend segue Atomic/Feature-Sliced (**views, widgets, features, entities, components**). Fluxo de uma ação:

flowchart LR

```
UI[Interface] --> Action[Server Action]
Action --> Zod[Validação Zod]
Zod --> Controller
Controller --> UC[Caso de uso]
UC --> RepoIF[Interface do repositório]
RepoInf[Repositório Drizzle] -.implementa.-> RepoIF
RepoInf --> PG[(PostgreSQL)]
UC --> Redis[(Redis pub/sub)]
Redis --> SSE[SSE: eventos em tempo real]
SSE --> UI
```

Regras de dependência: aplicação → domínio; infraestrutura → aplicação e domínio; domínio não importa Next.js, Drizzle nem React; o frontend nunca acessa a infraestrutura.

Observação. O `docs/codigo/arquitetura-web-app.md` e o README descrevem as camadas em `src/server/`. No código atual, cada módulo vive em `src/app/api/<módulo>/` (**entities, repositories, usecase, actions, tests**) e `src/server/shared` guarda apenas banco, Redis e logger. Essa organização por funcionalidade é a descrita corretamente em `docs/codigo/arquitetura.tex` (documentação de arquitetura do repositório), usada como fonte das decisões abaixo; o `architecture.md` e o README devem ser atualizados.

Decisões de arquitetura e trade-offs

Cada decisão segue o formato contexto, pontos positivos, pontos negativos e resolução. Fonte: `docs/codigo/arquitetura.tex` e o registro de decisões do grupo.

Next.js como plataforma full-stack Escolhido pela capacidade full-stack integrada: **Server Actions** executam as mutações no servidor sem rotas de API dedicadas para cada operação, o que reduz a

complexidade e a superfície de ataque, e a hospedagem gratuita para a volumetria prevista (Vercel) viabiliza o projeto.

- **Positivos:** desenvolvimento rápido (mesmo framework e mesma linguagem no frontend e no backend, com tipagens e funções compartilhadas e segurança de tipos de ponta a ponta); segurança (componentes e módulos **server-only** mantêm dados sensíveis e segredos no servidor, e importá-los no cliente quebra o build); sem negociação de CORS, por operar na mesma origem; React Server Components reduzem o JavaScript enviado ao navegador; hospedagem gratuita.
- **Negativo:** a arquitetura **serverless não permite WebSocket nativamente**, o que afetaria o tempo real das salas e do chat em grupo.
- **Resolução adotada:** tempo real por **Server-Sent Events (SSE)** com **Redis Pub/Sub** (item seguinte). Um serviço complementar de WebSocket gerenciado, como o **Aby**, foi considerado apenas como **alternativa**, caso o produto passasse a exigir comunicação bidirecional; **não foi adotado e não existe no código**.

Tempo real: SSE em vez de WebSocket

- **Unidirecionalidade:** as ações dos jogadores (entrar, votar, avançar rodada) são mutações pontuais, autenticadas e validadas, que já passam por Server Actions; o canal em tempo real só precisa levar eventos do servidor ao cliente.
- **Compatibilidade com serverless e HTTP/2:** WebSocket exige servidores com conexões abertas e processos ininterruptos (ou serviços externos pagos); SSE funciona sobre HTTP comum.
- **Reconexão automática:** o **EventSource** do navegador reconecta sozinho e preserva o último evento recebido, dispensando bibliotecas como Socket.io.
- **Transparência em redes corporativas:** SSE trafega como HTTP na porta 443, sem o cabeçalho Upgrade que firewalls e proxies costumam bloquear.
- **Implementação:** rota GET /api/rooms/[pin]/events com Content-Type: text/event-stream, cliente **ioredis** dedicado como assinante do canal `room:<pin>`, **ReadableStream** para emitir os eventos (`MEMBER_JOINED`, `VOTE_CAST`, `ROUND_COMPLETED`, `MATCH_FINISHED`) e encerramento da assinatura quando o cliente desconecta, para evitar vazamento de conexões.

Serverless Custo proporcional ao uso (o tráfego de treinamentos e aulas oscila entre picos e inatividade), elasticidade e menor custo operacional. Para operar bem nesse modelo: conexões de banco e Redis reutilizadas via `globalThis`, `runtime = "nodejs"` e `dynamic = "force-dynamic"` nas rotas de SSE, e nenhum estado mutável em memória ou disco (o estado fica no JWT em cookie, no Redis ou no PostgreSQL).

Banco de dados: PostgreSQL no Supabase, com Redis

- **Contexto:** PostgreSQL oferece um motor relacional confiável, de alto rendimento e de fácil manutenção, e a hospedagem no **Supabase** atende, sem custo, ao volume de acessos previsto. O padrão de acesso de um jogo em tempo real tem dois perfis opostos, então a persistência é híbrida.
- **PostgreSQL (dados duráveis e relacionais):** usuários, salas, notícias, votos e análises, com integridade referencial; a notícia extraída fica em uma coluna **JSONB** (`news_articles.article`), que absorve a variação de metadados sem migrações destrutivas; enums nativos (`ml_target_type`, `vote_option_type` com `reliable / uncertain / unreliable`).
- **Redis (dados efêmeros e de alta frequência):** busca de sala por PIN em $O(1)$ (`room:pin:<pin>`), placar em *sorted set* (`ZADD/ZREVRANGE`) e canais Pub/Sub para o SSE.
- **Supabase:** operação gerenciada (sem administrar servidores nem backups), *connection pooling* (Supervisor) adequado ao serverless, e console para inspeção. A conexão exige TLS (`sslmode=require`); o app usa o pooler, e as migrações usam a conexão direta.
- **Sem vendor lock-in:** o Olimpo conecta-se como cliente PostgreSQL padrão, por meio do Drizzle e do `postgres.js`, e não pelo SDK do Supabase.
- **Positivos:** hospedagem gratuita, conexão simples com o projeto, adequado ao uso.

Drizzle ORM Camada de acesso a dados com entidades tipadas e consultas construídas no código, o que traz robustez e agiliza o desenvolvimento.

- **Segurança:** a manipulação direta no código com consultas parametrizadas evita SQL injection.
- **Migrations:** esquemas em TypeScript geram SQL determinístico (**drizzle-kit**), versionado no Git.
- **Objetos e tipagem forte:** facilita declarar entidades e reduz erros.

Camadas, DDD e Feature-Sliced Design

Observação. O registro de decisões do grupo agrupava este item sob o título "FSD", mas o texto descreve a arquitetura em camadas do backend. São duas decisões distintas, separadas abaixo.

- **Backend: Clean Architecture orientada a funcionalidades.** Domínio (entidades e regras) isolado de HTTP, Next.js, Drizzle e Redis; casos de uso orquestram o fluxo; repositórios são interfaces com implementações de infraestrutura; Server Actions são a porta de entrada e validam com Zod. O motivador é desacoplar as regras de domínio dos serviços de infraestrutura, de modo que um componente externo possa ser trocado (por exemplo, migrar de um SGBD relacional para NoSQL) com impacto limitado ao adaptador da camada de dados. A **injeção de dependência** permite testar os casos de uso em milissegundos com repositórios em memória.
- **Frontend: Feature-Sliced Design** (**views, widgets, features, entities, components**), em que uma camada só importa de camadas abaixo dela, o que evita dependências circulares.
- **Positivos:** manutenibilidade, legibilidade e possibilidade de trocar ferramentas (como o banco) sem comprometer o restante do código. **Negativos:** complexidade e maior tempo de implementação.

Padrões complementares

- **Validação com Zod v4** em todas as entradas ("nunca confie na entrada do usuário").
- **Autenticação própria** (e-mail e senha), com senha em hash (**bcryptjs**) e sessão por JWT em cookie `httpOnly`, que não é legível por scripts e mitiga o roubo de sessão por XSS.
- **Contrato uniforme de respostas:** as actions devolvem `ActionResult<T>({ success: true, data } ou { success: false, error, code? })`, e os Route Handlers seguem o mesmo formato, com mensagens amigáveis que não expõem detalhes internos.
- **OpenSpec:** especificações vivas por capacidade (**auth, rooms, news-voting, realtime-events, ai-feedback, gamification**), com proposta, design e tarefas para cada mudança, que mantém humanos e agentes de IA alinhados a contratos verificáveis.
- **Padrão de código** com linters, formatação e Conventional Commits (Husky e Commitlint).
- **Pirâmide de testes:** testes unitários de domínio e casos de uso (Vitest), testes de Server Actions com simulação de ponta a ponta e logs estruturados, e testes E2E de interface (Playwright), todos no CI.

Conexão da IA com o backend O ponto de integração é a interface de domínio `IAIAnalysisService`, injetada no caso de uso de feedback. Hoje a implementação é um mock por palavras-chave; a implementação com o modelo de produção é descrita na seção 8.5. Como o domínio só conhece o contrato, a troca não altera as regras do jogo.

Diagrama entidade-relacionamento Gerado a partir dos schemas do Drizzle (`src/server/shared/database/schemas/`):

```
erDiagram
    users ||--o| user_avatars : possui
    users ||--o{ rooms : "é host de"
    users ||--o{ room_members : participa
    rooms ||--o{ room_members : tem
```

```

rooms ||--o{ room_playlist_items : "tem playlist"
news_articles ||--o{ room_playlist_items : "compõe"
news_articles ||--o{ news_analyses : "recebe análise"
room_playlist_items ||--o{ news_votes : "recebe votos"
rooms ||--o{ news_votes : registra
users ||--o{ news_votes : emite
news_articles ||--o{ global_challenges : "base de"
global_challenges ||--o{ global_challenge_answers : "recebe respostas"
users ||--o{ global_challenge_answers : responde

users {
    text id PK
    text email UK
    text password_hash
    enum role "participant | admin"
    int xp
}
user_avatars {
    text user_id PK
    enum gender
    enum skin
    enum outfit
    enum headwear
}
rooms {
    text id PK
    varchar pin UK
    enum status "waiting | in_progress | finished"
    int round_duration_seconds
    int current_round
    int total_rounds
    text host_id FK
}
room_members {
    text id PK
    text room_id FK
    text user_id FK
    enum role "host | participant"
    int score
}
news_articles {
    text id PK
    jsonb article
    enum target_classification "reliable | uncertain | unreliable"
}
room_playlist_items {
    text id PK
    text room_id FK
    text article_id FK
    int round_order
}
news_votes {
    text id PK
    text room_id FK

```

```

    text playlist_item_id FK
    text user_id FK
    enum vote "reliable | uncertain | unreliable"
    bool is_correct
    int points_awarded
}
news_analyses {
    text id PK
    text article_id FK
    enum classification
    jsonb reasons
    numeric confidence
    text model_version "default mock-v1"
}
global_challenges {
    text id PK
    text title
    text article_id FK
    int xp_reward
    bool is_active
}
global_challenge_answers {
    text id PK
    text challenge_id FK
    text user_id FK
    enum answer
    bool is_correct
    int xp_awarded
}

```

Restrições de unicidade relevantes: um usuário por sala (**room_members**), uma notícia por posição de rodada (**room_playlist_items**), um voto por usuário em cada item da playlist (**news_votes**) e uma resposta por usuário em cada desafio global.

8.3 Módulos implementados

Módulo	Função
auth	cadastro, login com JWT em cookies, resolução de sessão, avatar
rooms	ciclo de vida da sala, código PIN, playlist de notícias
news-voting	votação em três níveis, avaliação da rodada e pontuação
realtime-events	rota SSE e publicador via Redis pub/sub
global-challenges	desafios globais e respostas
news/extract	extração de título e conteúdo a partir de URL (ingestão)
ai-feedback	análise de credibilidade de cada notícia (hoje <i>mock</i>)

Dados: **users**, **user_avatars**, **rooms**, **room_members**, **room_playlist_items**, **news_articles** (com carga **jsonb**), **news_analyses**, **news_votes**, **global_challenges**, **global_challenge_answers**. A construção seguiu o fluxo de especificação com OpenSpec (specs arquivadas e ativas por módulo) e TDD nos casos de uso.

Qualidade. Aproximadamente 40 arquivos de teste (Vitest) e testes ponta a ponta de ações com logs `pino`; Playwright para a interface; CI com `pnpm audit`, `pnpm check` e testes E2E; CodeQL, *dependency review*, Dependabot e um revisor automatizado de *pull requests* (Acrux).

Interface (Next.js, Feature-Sliced Design). As telas implementadas cobrem o fluxo principal: página inicial com os modos de jogo, criação e entrada em sala por PIN, lobby, **jogo por etapas** (checagem da notícia, aguardando o veredito com a reflexão socrática, placar da rodada e placar final), ranking, submissão de notícia ou desafio autoral, cadastro/login e extração de notícia por URL. Os eventos da sala chegam à tela por SSE.

8.4 Estado do componente de IA

Os dois componentes de IA são executados pelo `model-engine/`, um serviço HTTP em Python separado do app Next.js e acessado apenas pelo servidor. `machine-learning/` continua responsável por experimentos, treinamento e resultados; o motor contém só o que é necessário para analisar um texto. Os componentes têm funções e saídas distintas e **não são combinados em um único escore**:

Componente	Função	Onde aparece no jogo
Observação de escrita (FP-Growth, POST/analyze)	Mostra padrões gramaticais presentes no trecho e a frequência da mesma combinação em cada classe do corpus	Painel “Observe a escrita”, durante a leitura
Modelo supervisionado (POST/supervised/analyze)	Estima P(fake) com explicação por n-grama e atributo	“Análise do modelo”, após o fechamento comum da rodada
Reflexão socrática	Perguntas gerais fixas, não pontuação dos jogadores	Devolutiva da rodada, distinta da previsão

Modelo supervisionado

O serviço recebe somente o corpo da notícia extraído pelo parser (até 100 mil caracteres) e não registra o conteúdo. Devolve `classification` (`reliable` | `uncertain` | `unreliable`), `fakeProbability`, escore canônico $100 \times P(\text{fake})$, `reasons`, `modelVersion`, hash do artefato e versão do código de inferência. A política `olimp-decision-policy-v1` define `reliable` para $P \leq 0,35$, `unreliable` para $P \geq 0,65$ e `uncertain` entre os dois valores; textos com menos de 30 palavras não são classificados, e a análise usa as primeiras 100 palavras, como no treino.

- **Verificação de integridade.** Antes de carregar o `.joblib`, o motor confere o SHA-256 do artefato, o patch exato do Python (3.14.2) e as versões de scikit-learn, NumPy, SciPy, pandas, spaCy, `pt_core_news_sm` e `joblib` registradas no manifesto. Se algo divergir, apenas o supervisionado fica indisponível (`/health/supervised` devolve 503) e as observações de escrita continuam funcionando.
- **Artefato final.** O modelo foi reajustado em treino + teste (7.200 notícias). As métricas internas da seção 5.11 vêm de outro ajuste, treinado só no treino, e não são uma avaliação independente deste artefato.
- **Desempenho do motor** (execução local em Docker Linux, 20 análises aquecidas): carga e aquecimento de 8,3 s, mediana de 140,6 ms, p95 de 188,3 ms e pico de 438 MiB. Com oito requisições simultâneas, três foram atendidas e cinco recusadas, pois a fila é intencionalmente curta (uma execução e duas vagas de espera). A medição não representa um teste de carga em produção.
- **Cache e identidade.** A análise é guardada em PostgreSQL (migração `0005_supervised_analysis_identity`) com chave composta por hash do corpo, artefato, código e política; trocar a versão gera nova análise. Em teste com duas instâncias do caso de uso e quatro consultas concorrentes, houve uma inferência e um registro.

- **Acesso.** POST `/api/news-prediction` exige sessão, participação na sala e rodada encerrada coletivamente; votar, avançar ou concluir individualmente não libera a previsão. Votos, placar e eventos SSE fecham sem chamar o modelo, e a pontuação continua vindo do gabarito. Se o modelo falhar, a partida segue e só a previsão fica indisponível. O marcador de fechamento da rodada fica no Redis e expira em 24 horas; depois disso a previsão é bloqueada, mesmo em partidas finalizadas.

Observação de escrita (FP-Growth)

O catálogo de produto vem da variante `sintaxe_ampliada` do run `fp-growth-metadados-ampliado s-20261008T002311Z` (22 atributos, 523 regras direcionais elegíveis, 345 padrões distintos) e contém 20 padrões candidatos, selecionados sem classe ou autoria no ranking. Para um texto novo, o motor normaliza (NFKC), recorta os 300 primeiros caracteres, extrai as medidas com limites congelados e indica os padrões que correspondem **integralmente**, sem recalcular quantis nem receber gabarito ou autor. Cada padrão traz uma explicação em linguagem simples e a frequência da combinação em cada classe da partição de validação (720 falsas e 720 verdadeiras): $\text{frequência} = \text{contagem} / 720$. Essas frequências são **dentro de cada classe** do corpus, e não a probabilidade de a notícia ser falsa. O catálogo tem status `experimental_descriptive_comparison` e **não passou por aprovação editorial humana**; o catálogo científico permanece `research_only`. A análise devolve `ok`, `no_match`, `invalid_text` (HTTP 400) ou `unavailable` (503).

Validação da integração

Um teste ponta a ponta (`web-app/scripts/check-news-prediction.ts`) usa banco e Redis descartáveis, parser HTML real, o modelo real e duas sessões de navegador. Verifica bloqueio antes do fechamento coletivo, recusa de texto arbitrário, cinco consultas concorrentes com um registro, recarga, telas desktop e celular, próxima rodada sem resultado antigo e encerramento da partida com falha induzida do modelo; passou com $P(\text{fake}) = 0,99$ em uma notícia de teste e versão `svm-spacy-chi2k10k-svd500-v1`. A suíte do web app reportou 419 testes aprovados (cinco condicionais ignorados) e a suíte Python 20, que inclui a reprodução numérica das features e das probabilidades do código original. O teste não cobre extração de URL externa ao vivo. Evidências em `web-app/validation/news-prediction/` e relatório em `docs/machine-learning/validacao-integracao-supervisionado.md`.

Observação. Os textos fixos que a interface exibia sem origem no modelo (a dica "78% mais chances" e a margem " $\pm 1.8\%$ ") não existem mais na `dev`. A reflexão socrática, porém, continua sendo um conjunto de perguntas gerais fixas.

8.5 Como o MVP constrói sobre as descobertas

1. **Contrato de saída alinhado.** O adaptador HTTP implementa `IAIAnalysisService` e devolve o formato do backend, sem mudar o domínio.
2. **Três níveis a partir de um classificador binário.** O jogo tem *reliable* / *uncertain* / *unreliable* e o modelo tem duas classes; a solução usa a probabilidade calibrada com limiares 0,35 e 0,65, decisão de produto.
3. **Entrada heterogênea.** O app recebe o texto extraído da página (`news/extract`). O motor aplica a mesma normalização e o mesmo truncamento do treino. Risco conhecido: os atributos de estrutura de sentença saem da distribuição de treino em textos curtos, o que também explica parte da queda externa (seção 5.6).
4. **Explicação.** Os `reasons` vêm da decomposição exata do score em n-gramas e atributos de estilo, em linguagem para o jogador. Os padrões do FP-Growth chegam ao jogo como observações descritivas, não como argumento de classificação.
5. **Separação de papéis.** A previsão só é revelada após o fechamento coletivo, para não influenciar o voto; a pontuação segue o gabarito. A interface usa "previsão do modelo" e "estimativa", e o classificador não consulta fontes nem confirma acontecimentos.

6. **Registro.** `modelVersion`, hash do artefato e versão do código ficam gravados em cada análise (`news_analyses`).

O que ainda falta. Medir o modelo com notícias reais coletadas pelo `news/extract`; submeter o catálogo de observações a revisão editorial humana e a um estudo de compreensão com jogadores; gerar dinamicamente as perguntas socráticas; persistir o fechamento da rodada de forma durável; e dimensionar a fila de inferência para carga real.

9. Conclusão

O projeto percorreu o caminho de uma ideia de produto (avaliador socrático de notícias) para uma plataforma de jogo com backend funcional, apoiada por duas linhas de pesquisa complementares e por um modelo de produção já integrado ao jogo.

O que está estabelecido. (i) Um classificador linear sobre TF-IDF word+char resolve o Fake.br com $F1 \approx 0,93$ e é difícil de superar: nem reduções de dimensionalidade, nem kernels, nem modelos de árvore, nem metadados morfosintáticos trouxeram ganho relevante de F1. (ii) O regime $d \gg N$ e a esparsidade favorecem modelos lineares, e $C=1$ é o ponto ótimo do LinearSVC. (iii) A normalização do texto e o truncamento são decisões metodológicas necessárias: eliminaram o artefato de formatação (SPACE), embora reste um sinal de tamanho. (iv) O FP-Growth, com controle de autoria, foi escolhido para gerar candidatos a perguntas; sua utilidade pedagógica ainda não foi avaliada, com a ressalva de que descreve o corpus. (v) O modelo de produção, $M2 + \chi^2 + SVD(500)$, mantém o F1 de 0,9295 com gap de 0,042 em vez de 0,0705.

Trade-offs assumidos. Interpretabilidade e custo contra capacidade (linear em vez de kernel ou árvore); descoberta de padrões contra classificação (FP-Growth em vez de detector de anomalias); controle de vazamento de fonte contra desempenho aparente (normalização e truncamento reduziram o F1 interno); menor ajuste ao treino e explicação detalhada contra generalização externa (o modelo de produção tem AUC externo de 0,677, contra 0,707 do word+char completo); e escopo (a previsão só é revelada após o fechamento coletivo e não altera a pontuação).

Limites. A generalização fora do Fake.br é fraca ($F1 \approx 0,61\text{--}0,64$, $AUC \approx 0,68\text{--}0,71$, em títulos), por viés de domínio que nenhuma das técnicas testadas resolveu; por isso a saída do modelo é apresentada como indício. O método socrático está implementado apenas como perguntas gerais fixas na interface, e o modelo ainda não foi medido em notícias reais de usuários.

Próximos passos. (1) Medir o modelo com notícias reais; (2) revisar editorialmente o catálogo de observações e testar sua compreensão com jogadores; (3) coletar um conjunto externo com texto completo de fontes e épocas distintas; (4) tornar dinâmicas as perguntas socráticas, a partir dos **reasons** e dos padrões do FP-Growth; (5) persistir o fechamento da rodada e dimensionar a fila de inferência.

Referências

<https://www.ibm.com/br-pt/think/topics/logistic-regression>

<https://www.scirp.org/reference/referencespapers?referenceid=3092942>

<https://www.bbc.com/future/article/20240509-the-sift-strategy-a-four-step-method-for-spotting-misinformation>

<https://www.productplan.com/glossary/moscow-prioritization>

<https://scikit-learn.org/stable/modules/tree.html>

<https://github.com/GiovanniGatti/socratic-llm/tree/main>

<https://www.datacamp.com/pt/tutorial/introduction-to-shap-values-machine-learning-interpretability>

<https://huggingface.co/tasks/feature-extraction>

<https://www.ibm.com/think/topics/feature-extraction>

<https://fernandafperes.com.br/blog/interpretacao-boxplot/>

https://shap.readthedocs.io/en/latest/example_notebooks/api_examples/plots/beeswarm.html

<https://shap.readthedocs.io/en/latest/>

https://shap.readthedocs.io/en/latest/example_notebooks/overviews/An%20introduction%20to%20explainable%20AI%20with%20Shapley%20values.html

<https://www.geeksforgeeks.org/data-analysis/what-is-exploratory-data-analysis/>

<https://www.ibm.com/br-pt/think/topics/exploratory-data-analysis>

Bibliografia:

<https://blog.runrun.it/metodologia-crystal/>

<https://www.dataphi.net/post/epic-story-e-tarefas-como-fazer-uma-gest%C3%A3o-eficiente-de-escopo-de-projeto>

<https://www.efficientlyconnected.com/pmi-ai-project-management-standard-enterprise-governance/>

https://www-datascience--pm-com.translate.goog/crisp-dm-2/?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc

https://www-6sigma-us.translate.goog/lean-six-sigma-articles/crystal-agile-methodology/?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc

https://youtu.be/G_ZETFAYxSQ?si=54Zrs0QEfuAEeGTi

https://interfacing-com.translate.goog/what-is-rasci-raci?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc

<https://www.atlassian.com/br/agile/scrum/ceremonies>

https://www-scrum--institute-org.translate.goog/Effort_Estimations_Planning_Poker.php?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc

<https://aws.amazon.com/pt/what-is/mlops/>

<https://www.atlassian.com/br/work-management/project-management/raci-chart>

<https://www.atlassian.com/br/agile/project-management/epics-stories-themes>

<https://www.iso.org/home/insights-news/resources/iso-42001-explained-what-it-is.html>

<https://www.gov.br/mds/pt-br/acao-a-informacao/governanca/integridade/campanhas/lgpd>

Apêndices

Apêndice A: Feature extraction e atributos derivados propostos

A etapa de **Feature Extraction** transforma os dados disponíveis no corpus em características mais representativas para o modelo de classificação. Embora o Fake.br Corpus já forneça métricas como número de verbos, adjetivos, advérbios, pronomes e palavras em maiúsculo, valores absolutos podem ser influenciados diretamente pelo tamanho da notícia.

Por isso, algumas dessas variáveis podem ser convertidas em **proporções e razões**, como `num_adverbios / num_palavras` ou `num_verbos_modais / num_verbos`. Dessa forma, textos de tamanhos diferentes podem ser comparados de maneira mais adequada, permitindo representar características como intensidade de linguagem descritiva, modalização, pessoalidade e densidade lexical.

Também é possível combinar diferentes atributos para criar indicadores mais significativos. A densidade lexical, por exemplo, relaciona substantivos, verbos, adjetivos e advérbios ao total de palavras, enquanto relações como adjetivos por substantivos ou advérbios por verbos ajudam a representar aspectos específicos do estilo de escrita.

Essas características não determinam diretamente se uma notícia é verdadeira ou falsa. Elas fornecem sinais linguísticos para que o algoritmo aprenda quais padrões e combinações possuem maior poder de classificação. Posteriormente, técnicas como **SHAP** podem indicar quais desses atributos tiveram maior influência em cada resultado, permitindo gerar justificativas mais rastreáveis para a análise apresentada ao usuário.

Atributos derivados propostos

<i>Atributo</i>	<i>Cálculo / origem</i>	<i>O que representa</i>
typeTokenRatio	$\text{num_types} / \text{num_tokens}$	Diversidade do vocabulário em relação ao tamanho do texto
adjectiveRatio	$\text{num_adjetivos} / \text{num_palavras}$	Intensidade relativa de linguagem descritiva ou avaliativa
adverbRatio	$\text{num_advverbios} / \text{num_palavras}$	Frequência relativa de modificadores e intensificadores
pronounRatio	$\text{num_pronomes} / \text{num_palavras}$	Presença relativa de referências pessoais no discurso
modalVerbRatio	$\text{num_verbos_modais} / \text{num_verbos}$	Grau de modalização, possibilidade, necessidade ou incerteza
imperativeSubjunctiveRatio	$\text{num_verbos_subj_imp} / \text{num_verbos}$	Presença relativa de construções imperativas ou subjuntivas
personalPronounRatio	$(\text{num_pron_1_2_sing} + \text{num_pron_1_plural}) / \text{num_pronomes}$	Grau de aproximação com autor, leitor ou grupo social
linkDensity	$\text{num_links} / \text{num_palavras}$	Quantidade relativa de referências externas presentes no texto
punctuationDensity	$(\text{num_tokens} - \text{num_palavras}) / \text{num_tokens}$	Aproximação da densidade de elementos de pontuação

As características já fornecidas pelo próprio corpus, como pausalidade, `tam_medio_sentenca`, `tam_medio_palavra`, `pct_erro_ortograficos`, emotividade e diversidade, podem ser mantidas juntamente com esses atributos derivados. Posteriormente, análises de correlação, importância de atributos e desempenho do modelo podem determinar quais variáveis realmente agregam informação e quais apresentam redundância.

Observação. Os atributos derivados (razões como `adjectiveRatio`, `modalVerbRatio` e `typeTokenRatio`) são proposta de engenharia de atributos e não fazem parte do pipeline treinado; os atributos usados estão listados na seção 4.3.

Apêndice B: Capturas dos resultados do FP-Growth

As figuras deste apêndice apresentam resultados históricos do FP-Growth de estilo, sem autoria e com controles. Não representam o principal atual, sintaxe ampliada, descrito na seção 6.7.

history/RESULTADOS.md comparação no mesmo split			
Modelo	Acurácia	Recall Fake	FPR True
Isolation Forest	0,9729	0,9847	0,8389
LOF	0,9479	0,9347	0,8389
One-Class SVM	0,5563	0,1500	0,8375
K-means (novidade)	0,5382	0,1111	0,8347
DBSCAN (novidade)	0,9667	0,9847	0,8514
Fonte: history/RESULTADOS.md, teste canônico congelado.			

Figura 1 Comparação resumida do teste canônico. Fonte: history/RESULTADOS.md [3].

FP-Growth principal resultados persistidos	
Notebook principal: FP-Growth sem autoria e controles	
Treino: 4.320 notícias	rótulos fora da descoberta
18 itemsets frequentes	20 regras brutas
8 regras destacadas	10 padrões textuais
Maior lift Fake agregado: 1,406	
Interpretação depende da comparação por autoria.	
Fonte: run fp-growth-controls-202609291215211Z.	

Figura 2 Síntese do experimento histórico sem autoria. Fonte: notebook histórico de estilo e run de controles [6].

Controle de autoria notebook principal				
Estado	Fake	Real	Total	P(fake estado)
com_autor	72	3.527	3.599	0,020
sem_autor	3.528	73	3.601	0,980
Cramér V = 0,960 21/31 padrões baseline contêm autoria				
Fonte: comparison_summary.md, run de controles.				

Figura 3 Controle de autoria no corpus. Fonte: notebook principal [6].

Apêndice C: Trechos do código do FP-Growth

Os trechos abaixo vêm de células de código do notebook Jupyter principal. Eles documentam a extração das features, a discretização, o cálculo de Jaccard, a mineração e a conferência da cobertura antes da avaliação por classe. Linhas longas foram quebradas visualmente nas figuras.

```

Notebook principal | célula style()

1 def style(text, author):
2     """Mesmas seis definições de estilo do notebook HDBSCAN, primeiros 300 caracteres."""
3     text = unicodedata.normalize("NFKC", text).rstrip("\uffff")[:300]
4     words, tokens = WORDS.findall(text), TOKENS.findall(text)
5     nw, nt = len(words), len(tokens)
6     distinct = len({word.casefold() for word in words})
7     links = len(re.findall(r"https?://\S+", text, flags=re.IGNORECASE))
8     uppercase = sum(word.isupper() and len(word) > 1 for word in words)
9     ratio = lambda a, b: a / b if b else np.nan
10    return {
11        "tem_autor": int(str(author).strip().casefold() not in {"", "none", "null", "nan"}),
12        "typeTokenRatio": ratio(distinct, nt),
13        "linkDensity": ratio(links, nw),
14        "punctuationDensity": ratio(nt - nw, nt),
15        "uppercaseRatio": ratio(uppercase, nw),
16        "diversidade": ratio(distinct, nw),
17    }

```

Figura 4 Extração das medidas de estilo. Fonte: célula style() do notebook principal [6].

```

Notebook principal | célula transactions()

1 def transactions(frame, low_q, high_q):
2     if not 0 < low_q < high_q < 1:
3         raise ValueError("Quantis devem satisfazer 0 < baixo < alto < 1")
4     result = pd.DataFrame(index=frame.index)
5     thresholds = {}
6     result["con_autor"] = frame["tem_autor"].eq(1)
7     result["sen_autor"] = frame["tem_autor"].eq(0)
8     for feature in FEATURES:
9         series = frame[feature].replace([np.inf, -np.inf], np.nan)
10        lo, hi = float(series.quantile(low_q)), float(series.quantile(high_q))
11        thresholds[feature] = {"low": lo, "high": hi, "missing": int(series.isna().sum())}
12        if lo >= hi: # distribuição degenerada; não fabricar faixas indistinguíveis
13            thresholds[feature]["omitted"] = True
14            continue
15        result[f"{feature}_baixo"] = series.le(lo) & series.notna()
16        result[f"{feature}_alto"] = series.ge(hi) & series.notna()
17    return result.astype(bool), thresholds

```

Figura 5 Conversão de medidas em itens discretos. Fonte: célula transactions() do notebook principal [6].

```

Notebook principal | célula mine()

6 frequent = fpgrowth(matrix, min_support=min_support, use_colnames=True, max_len=max_len)
7 frequent["features"] = frequent.itemsets.map(lambda x: sorted(x))
8 frequent["itemset_size"] = frequent.features.map(len)
9 frequent["pattern_id"] = frequent.features.map(lambda x: hashlib.sha256(json.dumps(x).encode()).hexdigest()[
:12])
10 frequent = frequent.sort_values(["support", "itemset_size", "pattern_id"], ascending=[False, False, True])
11 if frequent.empty or frequent.itemset_size.max() < 2:
12     rules = pd.DataFrame()
13 else:
14     rules = association_rules(frequent[["support", "itemsets"]], metric="confidence", min_threshold=0)
15     rules["jaccard"] = rules["support"] / (rules["antecedent support"] + rules["consequent support"] -
rules["support"])
16     rules["passes_filters"] = ((rules["lift"] >= min_lift) & (rules["confidence"] >= min_confidence)
17                               & (rules["jaccard"] >= min_jaccard))

```

Figura 6 Mineração, regras e Jaccard. Fonte: célula mine() do notebook principal [6].

```

Notebook principal | célula evaluate_frozen()

1 def evaluate_frozen(features, labels, patterns, criteria, train_ids, train_membership):
2     items = apply_criteria(features, criteria)
3     matches = {}
4     membership_rows = []
5     for row in patterns.iterTuples():
6         ids = set(items.index[items.features.all(axis=1)])
7         matches[row.pattern_id] = ids
8         membership_rows.extend((record_id, row.pattern_id, row.display_id) for record_id in ids)
9     membership = pd.DataFrame(membership_rows, columns=["record_id", "pattern_id", "display_id"])
10    expected = set(map(tuple, train_membership[["record_id", "pattern_id"]].iterTuples(index=False, name=None)))
11    observed = set(map(tuple, membership.loc[membership.record_id.isin(train_ids),
12                                           ["record_id", "pattern_id"]].iterTuples(index=False, name=None)))
13    if observed != expected:
14        raise ValueError(f"Membership do treino sem autoria divergiu: {len(observed ^ expected)} pares")

```

Figura 7 Conferência da cobertura congelada. Fonte: célula evaluate_frozen() do notebook principal [6].

Apêndice D: Catálogo completo das regras extraídas

A tabela reúne as 20 regras do experimento histórico de estilo sem autoria. Oito passaram conjuntamente pelos filtros de confidence, lift e Jaccard; as outras doze permanecem no CSV bruto e são apresentadas para transparência. A direção da seta altera a confidence, mesmo quando support, lift e Jaccard são iguais. Nenhuma regra estabelece falsidade factual [6, 11].

Legenda: TTR = razão de tipos por tokens; DIV = diversidade lexical; PONT = densidade de pontuação; MAIÚSC = proporção de maiúsculas. ↑ e ↓ indicam alto e baixo segundo os quantis do treino. S/C/L/J = support, confidence, lift de associação e Jaccard, nessa ordem.

Tabela 2 Todas as regras de associação extraídas pelo FP-Growth histórico de estilo sem autoria. Fonte: [6, 11].

Regra	S / C / L / J	Filtro	Como pode ajudar
DIV↓ → TTR↓	0,172 / 0,684 / 2,692 / 0,516	Sim	Investigar repetição e menor variedade lexical; as duas medidas são próximas.

Regra	S / C / L / J	Filtro	Como pode ajudar
TTR↓ →	0,172 / 0,677 / 2,692	Sim	Testar a direção inversa e comparar a confiança da associação.
DIV↓	/ 0,516		
TTR↑ →	0,166 / 0,657 / 2,557	Sim	Contrastar textos com maior variedade lexical com o perfil anterior.
DIV↑	/ 0,482		
DIV↑ →	0,166 / 0,645 / 2,557	Sim	Testar a direção inversa e comparar a confiança da associação.
TTR↑	/ 0,482		
TTR↑ →	0,138 / 0,548 / 2,163	Sim	Explorar textos com menos pontuação e maior variedade de tokens.
PONT↓	/ 0,376		
PONT↓ →	0,138 / 0,546 / 2,163	Sim	Testar a direção inversa e comparar a confiança da associação.
TTR↑	/ 0,376		
TTR↓ →	0,138 / 0,545 / 2,137	Sim	Priorizar revisão humana e teste externo do maior liftFake agregado.
PONT↑	/ 0,374		
PONT↑ →	0,138 / 0,543 / 2,137	Sim	Testar a direção inversa e comparar a confiança da associação.
TTR↓	/ 0,374		
TTR↑ →	0,119 / 0,471 / 1,080	Não	Comparar variedade de tokens com menor uso de maiúsculas.
MAIÚSC↓	/ 0,209		
MAIÚSC↓	0,119 / 0,273 / 1,080	Não	Testar a direção inversa e comparar a confiança da associação.
→ TTR↑	/ 0,209		
PONT↓ →	0,116 / 0,459 / 1,053	Não	Examinar perfil de pontuação e maiúsculas, inclusive por estrato de autoria.
MAIÚSC↓	/ 0,203		
MAIÚSC↓	0,116 / 0,267 / 1,053	Não	Testar a direção inversa e comparar a confiança da associação.
→ PONT↓	/ 0,203		
DIV↑ →	0,115 / 0,449 / 1,030	Não	Investigar variedade lexical sem ênfase por maiúsculas.
MAIÚSC↓	/ 0,200		
MAIÚSC↓	0,115 / 0,265 / 1,030	Não	Testar a direção inversa e comparar a confiança da associação.
→ DIV↑	/ 0,200		
DIV↓ →	0,105 / 0,419 / 0,961	Não	Verificar repetição lexical sem maiúsculas no conjunto.
MAIÚSC↓	/ 0,181		
MAIÚSC↓	0,105 / 0,242 / 0,961	Não	Testar a direção inversa e comparar a confiança da associação.
→ DIV↓	/ 0,181		
TTR↓ →	0,106 / 0,418 / 0,960	Não	Comparar baixa variedade de tokens com menor uso de maiúsculas.
MAIÚSC↓	/ 0,182		
MAIÚSC↓	0,106 / 0,244 / 0,960	Não	Testar a direção inversa e comparar a confiança da associação.
→ TTR↓	/ 0,182		
PONT↑ →	0,105 / 0,412 / 0,946	Não	Inspecionar pontuação alta combinada a poucas maiúsculas.
MAIÚSC↓	/ 0,179		
MAIÚSC↓	0,105 / 0,241 / 0,946	Não	Testar a direção inversa e comparar a confiança da associação.
→ PONT↑	/ 0,179		

Leitura prática: as regras destacadas ajudam a formular perguntas sobre perfis de escrita. As doze regras que não passaram nos filtros servem como registro exploratório. Antes de qualquer uso preditivo, compare as associações dentro dos estratos de autoria e valide-as em notícias externas [11].

Apêndice E: Planejamento (roadmap) e cumprimento

O grupo definiu, na primeira semana, um roadmap de 7 semanas para a Residência em IA (Eldorado / PUC), com 6 integrantes organizados em frentes rotativas, sem papéis fixos (princípio do *Crystal Clear*). O quadro abaixo compara cada etapa com o que o repositório evidencia. A coluna "Entregue" só contém o que foi verificado no código e nos documentos.

Etapa	Objetivo planejado	Entregue (evidência)
1	Metodologia e organização	Metodologias documentadas (seção 2): Crystal Clear, RACI, MoSCoW, Planning Poker, AIPGF, CRISP-DM, MLOps, testes e pesquisa. GitHub com <i>pull requests</i> numerados, branches por funcionalidade, Conventional Commits e revisão automatizada
2	Arquitetura, tecnologias e dataset	Arquitetura documentada (<code>docs/codigo/arquitetura.tex</code> , OpenSpec) e tecnologias escolhidas (seção 8.2); Fake.br analisado e limpo (seção 4); variável-alvo identificada; Regressão Logística como baseline
3	Estudo e experimentação de técnicas	TF-IDF e n-gramas; Regressão Logística, SVM e Random Forest comparados; várias técnicas não supervisionadas (Isolation Forest, LOF, One-Class SVM, K-means, DBSCAN, HDBSCAN, PU Learning, FP-Growth); SVD/PCA; métricas iniciais
4	Construção e avaliação do modelo; preparação para o backend	Pipeline de treino em Python; split controlado; accuracy, precisão, recall, F1 e matriz de confusão; várias visualizações; modelo principal documentado (seções 5 e 7); módulo de inferência isolado (<code>modelo_olimpico.py</code>); contrato de entrada e saída (<code>AIAnalysisResult</code>)
5	Motor de análise, backend e fundação do frontend	Backend com autenticação, salas, votação em três níveis (equivalente ao semáforo), SSE, desafios globais e extração de notícia com proteção contra SSRF; entradas validadas com Zod; estrutura base do frontend; a tela já consome o contrato <code>analysis</code>
6	Experiência do usuário, integração e início da frente de negócio	Tela inicial, criação e entrada em sala, lobby, jogo por etapas, placares, ranking, submissão de notícia e extração por URL; semáforo de três níveis; reflexão socrática (cartões fixos)
7	Validação, negócio, refinamento e pitch	Documentação técnica (este documento, <code>docs/codigo/arquitetura.tex</code> , especificações OpenSpec); cerca de 40 arquivos de teste automatizados e CI com auditoria, verificação estática, E2E e CodeQL

Diretrizes de escopo do roadmap versus o construído. O roadmap previa como *Must Have* a classificação com score, o semáforo, a explicação por sinais, o modo desafio, a entrada manual de notícia, o modelo supervisionado, um experimento não supervisionado, as métricas e a validação com usuários. Desses, o repositório evidencia o semáforo, o modo desafio, a entrada de notícia (por URL), o modelo supervisionado, os experimentos não supervisionados e as métricas; **a classificação com score e a explicação vindas do modelo, e a validação com usuários, não estão evidenciadas.** Itens que o roadmap classificava como *Could Have* (login, ranking global e funcionalidades sociais) e como *Won't Have* (infraestrutura avançada de multiplayer) **foram implementados**, o que mostra que a prioridade do produto passou do avaliador individual para o jogo multiplayer, antes da integração do modelo.